

Appunti di Core Java 21

Emanuele Romano

Indice

1	Fondamenta	1
1.1	Editing, Compilazione ed Esecuzione	1
1.2	Cenni alla JShell: L'approccio REPL	1
1.3	Tipi Primitivi	2
1.3.1	Il tipo char e Unicode	2
1.3.2	Il tipo boolean	3
1.3.3	Quadro generale dei tipi primitivi	3
1.4	Stringhe	4
1.4.1	Eccezioni allo String Pool: Stringhe Dinamiche	4
1.4.2	L'Immutabilità in Java: Oltre le Stringhe	5
1.5	La Classe StringBuilder	6
1.6	Input e output su console	7
1.6.1	Scanner	7
1.6.2	Output su Console	7
1.7	Lettura e Scrittura su File	8
1.7.1	Lettura da File	8
1.7.2	Scrittura su File	8
1.7.3	Gestione dei Percorsi e Eccezioni	9
1.8	Il Costrutto Switch	9
1.8.1	Lo Switch Tradizionale (Statement)	9
1.8.2	La Sintassi a Freccia (Modern Switch Statement)	9
1.8.3	Switch Expressions	10
1.8.4	Evoluzioni Avanzate: Pattern Matching e Null	10
1.8.5	L'Utilizzo della Clausola <code>when</code> negli <code>switch</code>	11
1.9	Big Numbers: BigInteger e BigDecimal	11
2	Classi e oggetti	13
2.1	I Costruttori	13
2.1.1	Ulteriori note su <code>this</code>	14
2.2	Overloading e Firma dei Metodi	14
2.3	Costruzione degli Oggetti	15
2.3.1	Inizializzazione e Valori di Default	16
2.3.2	Blocchi di Inizializzazione	16
2.3.3	L'Ordine di Esecuzione	16
2.4	L'inferenza del tipo: la parola chiave <code>var</code>	16
2.5	Ulteriori note sull'incapsulamento	17
2.5.1	Il rischio della fuga di riferimenti (<i>Leaking References</i>)	17
2.6	I Metodi Privati	18
2.7	Campi di istanza <code>final</code>	19
2.8	Membri Statici: Campi e Costanti	19
2.9	Il Metodo <code>main</code>	20
2.10	Il passaggio dei parametri: Call-by-Value	21
2.11	Metodi con numero variabile di argomenti (Varargs)	21

2.12	Tipi Enumerati (Enum)	23
2.13	I Records: classi come contenitori di dati	25
2.14	Gestione della Memoria: Stack vs Heap	29
2.14.1	Confronto Sintetico	29
3	Aspetti di Gestione del Codice	31
3.1	Anatomia di un Progetto Java: Convenzioni e Struttura	31
3.2	I Package e il Naming Convention	31
3.2.1	Collaborazione e Conflitti	32
3.3	Creazione e Gestione Manuale dei Package	33
3.4	L'Accesso a livello di Package	34
3.4.1	Sicurezza e Moduli	35
3.5	Gli Import	35
3.5.1	Risoluzione dei Conflitti di Omonimia	36
3.5.2	Gli Import Statici	36
4	Ereditarietà e Polimorfismo	38
4.1	Classi, Superclassi e Sottoclassi	38
4.2	Sovrascrivere i Metodi (Overriding)	39
4.2.1	L'importanza di <code>@Override</code>	40
4.3	I Costruttori delle Sottoclassi	41
4.4	Polimorfismo	42
4.4.1	Il Legame Dinamico (Dynamic Binding)	43
4.4.2	Sintesi Mnemonica: Il Senso di Marcia	44
4.4.3	Covarianza dei Tipi di Ritorno	44
4.5	Prevenire l'Ereditarietà: Il modificatore <code>final</code>	45
4.6	Il Casting degli Oggetti	47
4.7	Evoluzione di <code>instanceof</code> : Pattern Matching	47
4.7.1	Scoping e Operatori Logici	48
4.7.2	Il Record Pattern e la Decostruzione (Java 21+)	49
4.8	Accesso Protetto: Il modificatore <code>protected</code>	50
4.9	La Classe <code>Object</code>	51
4.9.1	Il Metodo <code>equals()</code>	51
4.9.2	Il Metodo <code>hashCode()</code>	55
4.9.3	Il Metodo <code>toString()</code>	57
4.9.4	Il caso particolare dei Records	57
4.10	Classi Astratte	58
4.11	Classi Sigillate (<i>Sealed Classes</i>)	59
5	Interfacce, Lambdas e Inner Classes	61
5.1	Interfacce in Java	61
5.1.1	L'interfaccia <code>Comparable</code>	62
5.1.2	Interfacce vs Classi Astratte	65
5.1.3	Metodi Default: Evoluzione e Compatibilità	66
5.2	Il Pattern Callback: Passare Comportamento	66
5.2.1	Un esempio di Callback: l'interfaccia <code>Comparator</code>	67
5.3	Lambda Expressions	68
5.3.1	Introduzione e Motivazioni	68
5.3.2	Sintassi delle Lambda Expressions	69

5.3.3	Usso corretto delle Lambda Expressions	70
5.3.4	Interfacce Funzionali Standard	72
5.3.5	Method e Constructor References	73
5.3.6	Accesso alle variabili (Variable Capture)	74
5.4	Inner Classes	75
5.4.1	Struttura e Accesso allo Stato	75
5.4.2	Local Inner Classes	77
5.4.3	Meccanica della Variable Capture	78
5.4.4	Classi Anonime	79
5.4.5	Classi Statiche	79
5.4.6	Il Pericolo del Memory Leak nelle Inner Classes	80
5.5	Approfondimento: le Chiusure (Closures)	81
6	Eccezioni, Asserzioni e Logging	83
6.1	Gestione delle Eccezioni	83
6.1.1	La Gerarchia delle Eccezioni	85
6.1.2	Dichiarare e Lanciare Eccezioni	86
6.1.3	Il Blocco <code>try-catch</code> : gestione locale	86
6.1.4	La clausola <code>throws</code> : gestione delegata	88
6.1.5	Ereditarietà ed Eccezioni	89
6.1.6	Creare Classi di Eccezione Personalizzate	90
6.1.7	Exceptions Chaining	90
6.1.8	<code>try-with-resources</code>	91
6.1.9	Costruttori e Metodi di Eccezioni	94
6.2	Le Asserzioni (Assertions)	94
6.2.1	Attivare/Disattivare Asserzioni	95
6.2.2	Quando scegliere le Asserzioni	97
7	Generics	99
7.1	Introduzione	99
7.2	Struttura di una Classe Generica	100
7.3	Metodi Generici	101
7.4	Variabili di Tipo Vincolate (Bounded Type Variables)	101
7.5	Funzionamento interno dei Generics	102
7.5.1	Type Erasure (Cancellazione del Tipo)	102
7.5.2	Traduzione delle Espressioni Generiche	103
7.5.3	Type Erasure e Polimorfismo: i Metodi Ponte (Bridge Methods)	103
7.6	Interoperabilità con Codice Legacy	105
7.7	Record Patterns e Generics	106
7.8	Ereditarietà nei Tipi Generici	107
7.9	Wildcard Types (Tipi Jolly)	108
7.9.1	Usi consentiti	108
7.9.2	Wildcard con vincolo inferiore: <code>? super T</code>	110
7.9.3	Applicazioni in Supertipi Generici	110
7.9.4	Il Principio PECS (Producer Extends, Consumer Super)	112
7.9.5	Unbounded Wildcards	113
7.9.6	Wildcard Capture	113
7.10	Restrizioni e Limitazioni dei Generics	114
7.10.1	Verifiche di Tipo a Runtime	114

7.10.2	Divieto di Creazione di Array Generici	114
7.10.3	Limiti sui Varargs	115
7.10.4	Divieto di Istanziamento di Variabili di Tipo	116
8	Collections	117
8.1	Separazione tra Interfaccia e Implementazione	117
8.2	L'Interfaccia Collection	118
8.2.1	Il Concetto di Operazioni Opzionali	120
8.3	Iteratori	120
8.3.1	L'Interfaccia <code>ListIterator</code>	123
8.3.2	Sicurezza e Iterazione: Perché l'Indice è Pericoloso	124
8.4	Le Classi Astratte "Ponte"	125
8.5	Interfacce della Gerarchia Collection	126
8.5.1	L'Interfaccia List	126
8.5.2	L'Interfaccia Set (Insiemi)	127
8.5.3	SequencedCollection (Java 21+)	128
8.5.4	Queue e Deque	129
8.6	Classi Collection Concrete	130
8.6.1	Il Motore ad Array (Contiguità Fisica)	130
8.6.2	Il Motore a Nodi: <code>LinkedList</code>	130
8.6.3	Il Motore Hash (Tabelle Hash)	131
8.6.4	<code>EnumSet</code> : Prestazioni a Livello di Bit	132
8.6.5	Il Motore ad Albero: <code>TreeSet</code>	132
8.6.6	Il Motore a Heap: <code>PriorityQueue</code>	132
8.7	Mappe	133
8.7.1	L'Interfaccia Map	133
8.7.2	Iterazione e le "Viste" della Mappa	134
8.7.3	Pattern Operativi di Aggiornamento	136
8.7.4	La Classe Astratta "Ponte" per le Mappe	138
8.7.5	SequencedMap e Mappe Ordinate (Java 21+)	139
8.7.6	Classi Map Concrete	140
8.8	Copie e Viste	141
8.8.1	Factory Method per Collezioni Leggere e Immodificabili	142
8.8.2	Copie e Viste Immodificabili	143
8.8.3	Viste di Sotto-Intervalli (Sub-Ranges)	145
A	Appendice A: Nozioni Accessorie	147
A.1	Il Concetto di Hashing	147
A.2	Tabelle Hash in Java	148
A.3	L'Array Circolare (Circular Array)	149
A.4	La Struttura Dati Heap	150
A.5	La Coda a Priorità (Priority Queue)	151
B	Bozze e Argomenti in Sospeso	153
B.1	Argomenti in sospeso da riprendere	153
B.2	Il Class Path	153
B.2.1	La "Ricerca Intelligente" del Compilatore	153
B.2.2	I file JAR (Java Archive)	154
B.2.3	L'Algoritmo di Ricerca	154

B.2.4	Uso delle Wildcard	154
B.3	La Classe <code>java.lang.System</code>	154
B.3.1	Caratteristiche Strutturali	154
B.3.2	I Flussi di I/O Standard	155
B.3.3	Gestione del Tempo e Misurazioni	155
B.3.4	Interazione con l'Ambiente Operativo	155
B.3.5	Utility di Sistema e Memoria	156
B.3.6	Integrazione con il Logging Moderno	156
B.4	Snippets vari da inserire	156
B.5	Logging [BOZZE DA RIPRENDERE]	156
B.5.1	Architettura del Logging: Facciate e Implementazioni	157
B.5.2	L'interfaccia <code>System.Logger</code>	158
B.5.3	Ottenimento e Utilizzo del Logger	159

Fondamenta

1.1 Editing, Compilazione ed Esecuzione

Il processo di sviluppo in Java si fonda sulla distinzione tra codice sorgente e codice eseguibile, mediata dalla Java Virtual Machine (JVM).

Si definiscono tre stadi sequenziali per la messa in funzione di un software:

1. **Editing:** Scrittura del codice sorgente in un file di testo con estensione `.java`.
2. **Compilazione:** Traduzione del sorgente in *bytecode* (formato intermedio indipendente dall'architettura) tramite il comando `javac`. L'output è un file `.class`.
3. **Esecuzione:** Caricamento e interpretazione del bytecode da parte della JVM tramite il comando `java`.

Osservazione 1.1. La padronanza della riga di comando è essenziale per operare in ambienti server (sprovvisti di interfaccia grafica), per la gestione manuale del *Classpath* e per comprendere i meccanismi di risoluzione delle dipendenze che gli IDE tendono ad automatizzare e occultare.

A partire da Java 11, è stata introdotta la funzionalità definita dalla JEP 330, che permette di eseguire un programma contenuto in un singolo file sorgente senza la creazione esplicita di un file `.class`.

Listing 1.1: Esecuzione rapida senza compilazione manuale

```
1 # Sintassi valida da Java 11+
2 java NomeFile.java
```

In questo caso, il bytecode viene generato esclusivamente in memoria e rimosso al termine dell'esecuzione, rendendo Java efficace anche per la scrittura di script rapidi e prototipazione.

1.2 Cenni alla JShell: L'approccio REPL

Introdotta con Java 9, lo strumento `jshell` fornisce un ambiente interattivo per l'esplorazione del linguaggio.

JShell è un sistema *Read-Eval-Print Loop*. A differenza del ciclo standard (*edit-compile-run*), il REPL legge l'input dell'utente, valuta l'espressione, stampa il risultato e attende l'istruzione successiva.

Caratteristiche principali :

- **Assenza di Boilerplate:** Non è necessario definire classi o il metodo `main` per eseguire istruzioni.
- **Variabili Implicite:** Le espressioni valutate senza assegnazione vengono memorizzate in variabili generate automaticamente (es. `$1`, `$2`).
- **Comandi di Controllo:** I comandi che iniziano con lo slash (es. `/list`, `/vars`, `/exit`) permettono di gestire lo stato della sessione.

Osservazione 1.2. L'utilità di JShell risiede nella riduzione dell'attrito cognitivo durante l'apprendimento. Permette di isolare il comportamento di specifiche API o costrutti sintattici (come il comportamento dei tipi primitivi in condizioni di overflow) senza la distrazione della struttura formale del progetto.

Listing 1.2: Esempio di sessione interattiva

```
1 jshell> 2 + 2
2 $1 ==> 4
3
4 jshell> int x = $1 * 3
5 x ==> 12
6
7 jshell> /vars
8 |      int $1 = 4
9 |      int x = 12
```

1.3 Tipi Primitivi

1.3.1 Il tipo char e Unicode

A differenza del linguaggio C, dove il tipo `char` occupa solitamente 1 byte (ASCII), in Java il tipo `char` occupa **2 byte** (16 bit) e segue lo standard **UTF-16**.

Osservazione 1.3 (L'illusione dei 16 bit). Originariamente, Java era stato progettato con l'idea che 65.536 caratteri fossero sufficienti. Poiché lo standard Unicode è cresciuto oltre questo limite, Java ha dovuto adottare le **Surrogate Pairs**: alcuni simboli (emoji, caratteri esotici) vengono rappresentati usando **due char** consecutivi (4 byte totali).

Differenza chiave: Code Point vs Code Unit

- **Code Point**: Il valore numerico effettivo assegnato a un simbolo nello standard Unicode (es. U+1F4E9).
- **Code Unit**: L'unità di memoria minima in Java (16 bit).

Listing 1.3: Esempio di conteggio con caratteri supplementari

```
1 // Usiamo la codifica esadecimale per l'emoji del puzzle
2 // (che in UTF-16 richiede due unita': \uD83E e \uDDE9)
3 String s = "\uD83E\uDDE9";
4
5 System.out.println(s.length()); // Restituisce 2, non 1!
```

In definitiva, la funzione `length()` non restituisce il numero di simboli grafici "umani", ma il numero di **code units** (unità da 16 bit) necessarie a rappresentarli:

- **Caso Standard**: Per i caratteri comuni (Basic Multilingual Plane), 1 simbolo = 1 `char` (2 byte). In questo caso, la lunghezza coincide con il numero di caratteri.
- **Caso Esotico**: Per i caratteri supplementari (come le emoji o simboli matematici rari), 1 simbolo = 2 `char` (4 byte). In questo caso, la lunghezza risulta **doppia** rispetto al numero di simboli reali.



Nonostante le stringhe siano logicamente sequenze di `char` a 2 byte (UTF-16), le versioni moderne di Java (a partire da Java 9 in poi) le memorizzano come sequenze di singoli byte se il contenuto lo permette, risparmiando spazio senza cambiare il comportamento del codice.

1.3.2 Il tipo boolean

Il tipo `boolean` esprime i valori di verità dell'algebra di Boole: `true` e `false`.

Osservazione 1.4 (Rappresentazione in memoria). Nonostante logicamente basti 1 bit, la dimensione effettiva non è definita rigorosamente dalla specifica JVM. Nella pratica:

- Una variabile singola occupa solitamente **1 byte** per motivi di allineamento della memoria e velocità di accesso della CPU.
- Non esiste alcuna conversione (casting) tra tipi numerici e il tipo `boolean`, a differenza di quanto avviene in linguaggio C.

Listing 1.4: Rigidità del tipo boolean

```

1 // Errore in Java (ma valido in C)
2 // int x = 1; if (x) { ... }
3
4 // Forma corretta in Java
5 int x = 1;
6 if (x != 0) {
7     System.out.println("Vero");
8 }

```

1.3.3 Quadro generale dei tipi primitivi

La seguente tabella riassume gli 8 tipi primitivi di Java, evidenziando le caratteristiche tecniche e le limitazioni algebriche discusse.

Tipo	Memoria	Intervallo / Specifica	Note di Rilievo
<code>byte</code>	1 byte	$[-2^7, 2^7 - 1]$	Uso limitato (stream di dati).
<code>short</code>	2 byte	$[-2^{15}, 2^{15} - 1]$	Ormai raramente utilizzato.
<code>int</code>	4 byte	$[-2^{31}, 2^{31} - 1]$	Tipo standard per gli interi.
<code>long</code>	8 byte	$[-2^{63}, 2^{63} - 1]$	Richiede suffisso L (es. 42L).
<code>float</code>	4 byte	IEEE 754 (6-7 cifre dec.)	Richiede suffisso F (es. 3.14F).
<code>double</code>	8 byte	IEEE 754 (15 cifre dec.)	Default per decimali; approssimato.
<code>char</code>	2 byte	Unità di codice UTF-16	16-bit; gestisce <i>surrogate pairs</i> .
<code>boolean</code>	~1 byte	{true, false}	Nessuna conversione da/verso <code>int</code> .

Tabella 1.1: Quadro sinottico dei tipi primitivi in Java.



Promemoria:

- **Aritmetica Modulare:** L'overflow degli interi è silenzioso e ciclico ($max + 1 = min$).
- **Precisione Binaria:** Non usare mai `double` o `float` per calcoli monetari o di precisione assoluta (usare `BigDecimal`).

1.4 Stringhe

In linguaggi come Python è fondamentale la distinzione tra tipi mutabili e immutabili, resa piuttosto evidente dal fatto che oggetti mutabili hanno spesso un equivalente immutabile (si pensi a `list/tuple`). In Java questo aspetto è meno immediato sintatticamente, ma altrettanto cruciale.

In particolare, gli oggetti **String** sono **immutabili**: una volta creati, il loro stato non può essere alterato. Ogni tentativo di modifica produce, di fatto, un nuovo oggetto con un riferimento diverso in memoria.

Apparentemente questa scelta progettuale potrebbe sembrare inefficiente sotto il profilo delle performance (si pensi a cicli iterativi che concatenano stringhe). In realtà, si tratta di una scelta strategica per bilanciare sicurezza, gestione della memoria e velocità. Ecco i quattro motivi principali:

1. String Pool (Efficienza di Memoria)

Le stringhe sono tra i dati più frequenti in un'applicazione. Grazie all'immutabilità, Java può utilizzare lo *String Pool*: una zona di memoria dove viene conservata una sola copia di ogni stringa letterale. Se definiamo `String a = "Ciao"` e `String b = "Ciao"`, entrambe le variabili punteranno allo **stesso identico oggetto**. Se le stringhe fossero mutabili, la modifica di `a` altererebbe inavvertitamente anche `b`. L'immutabilità permette quindi un enorme risparmio di RAM.

2. Sicurezza (Il motivo critico)

Le stringhe gestiscono dati sensibili come:

- Percorsi di file (es. `C:\dati\segreti.txt`);
- Credenziali di database (URL, username, password);
- Parametri di rete (Indirizzi IP e porte).

Se una stringa fosse mutabile, un thread malevolo potrebbe cambiare il contenuto del percorso di un file un istante dopo che il sistema di sicurezza ha autorizzato l'accesso, ma un istante prima dell'apertura effettiva. Questo bug è noto come **Time-of-Check to Time-of-Use (TOCTOU)**.

3. Caching dell'Hashcode (Performance)

Poiché il contenuto di una `String` non cambia mai, Java ne calcola l'hashcode una sola volta e lo memorizza in *cache*. Questo rende le stringhe estremamente veloci quando usate come chiavi nelle `HashMap` o nei `HashSet`. Se la stringa fosse mutabile, l'hash andrebbe ricalcolato ad ogni operazione di ricerca o inserimento.

4. Thread Safety

In contesti multi-core, gli oggetti immutabili sono *thread-safe* per definizione. Non è necessario implementare blocchi o meccanismi di sincronizzazione complessi (che rallentano l'esecuzione) per evitare che due thread modifichino la stessa stringa contemporaneamente, corrompendo i dati.

Osservazione 1.5. Nei casi in cui l'immutabilità comporti una reale inefficienza (come la costruzione di stringhe complesse in cicli intensivi), Java mette a disposizione la classe `StringBuilder`, che permette manipolazioni mutabili e che analizzeremo in un paragrafo a seguire.

1.4.1 Eccezioni allo String Pool: Stringhe Dinamiche

È fondamentale distinguere tra stringhe *letterali* e stringhe *dinamiche*:

- **Stringhe Letterali:** Definite direttamente nel codice (es. `"Ciao"`). Il compilatore le inserisce automaticamente nello *String Pool*.

- **Stringhe Dinamiche:** Create a runtime (es. tramite `new String()`, input dell'utente o metodi come `substring()`). Queste **non** risiedono nello String Pool ma vengono allocate come nuovi oggetti nello Heap.

Esempio 1.1 (Confronto di Riferimenti). Se scriviamo:

```

1 String s1 = "Test"; // Pool
2 String s2 = new String("Test"); // Heap (nuovo oggetto)
3 System.out.println(s1 == s2); // Risultato: false

```

Anche se il contenuto è identico, i riferimenti di memoria sono diversi.

Il metodo `intern()`

Esiste un modo per forzare una stringa dinamica nel Pool: il metodo `s.intern()`. Se invocato, Java controlla se esiste già una stringa uguale nel Pool: se sì, restituisce il riferimento esistente; altrimenti, aggiunge la stringa al Pool e ne restituisce il riferimento. Il metodo `intern()` è una scelta vincente in scenari di **Data Deduplication**:

- **Dati Altamente Ripetitivi:** Quando carichi grandi dataset (es. da file CSV o database) che contengono migliaia di occorrenze delle stesse stringhe (es. nomi di città, categorie merceologiche, stati "SÌ/NO").
- **Ottimizzazione RAM:** Invece di avere 10.000 oggetti "Milano" diversi nello Heap, avrai 10.000 riferimenti che puntano all'unica istanza "Milano" nel Pool.
- **Confronti Massivi:** Se devi fare milioni di confronti tra stringhe, una volta "internate", il confronto tramite `==` è istantaneo (confronto di indirizzi) rispetto alla scansione carattere per carattere di `.equals()`.

Attenzione: Evita `intern()` su stringhe ad alta cardinalità (es. ID univoci, timestamp, codici fiscali) perché satureresti lo String Pool inutilmente, degradando le prestazioni invece di migliorarle.

1.4.2 L'Immutabilità in Java: Oltre le Stringhe

È un errore comune pensare che l'immutabilità riguardi esclusivamente la classe `String`. In realtà, Java adotta questo pattern per molte altre classi fondamentali, garantendo coerenza dei dati e facilità di debug. Un oggetto è definito immutabile se il suo stato non può essere alterato dopo la costruzione.

Oltre alle stringhe, i principali tipi immutabili in Java includono:

- **Classi Wrapper dei Primitivi:** Tutte le classi che "incapsulano" i tipi primitivi (`Integer`, `Long`, `Double`, `Float`, `Short`, `Byte`, `Character`, `Boolean`) sono immutabili. Se hai un `Integer` con valore 10, non puoi "sovrascrivere" quel 10; ogni operazione aritmetica produrrà un nuovo oggetto `Integer`.
- **Numeri a Precisione Arbitraria:** Le classi `BigInteger` e `BigDecimal` (utilizzate per calcoli matematici e finanziari di alta precisione) sono immutabili. Questo è cruciale perché garantisce che i valori calcolati non vengano alterati accidentalmente da altri moduli del programma.
- **Nuova API Date e Time (`java.time`):** A partire da Java 8, le classi per la gestione di date e orari (come `LocalDate`, `LocalTime`, `ZonedDateTime`) sono state rese immutabili. Questo ha risolto i gravi problemi di sicurezza e thread-safety della

vecchia classe `java.util.Date`, che essendo mutabile era soggetta a manipolazioni impreviste.

Osservazione 1.6. L'immutabilità funge da **invariante logica**: in un sistema complesso, sapere che certi oggetti non cambieranno mai il proprio stato permette all'informatico di ragionare sul codice con la stessa certezza con cui si ragiona su una costante in una formula.

1.5 La Classe `StringBuilder`

Come analizzato nella paragrafo precedente, l'immutabilità delle stringhe garantisce sicurezza e risparmio di memoria tramite lo *String Pool*. Tuttavia, questa caratteristica diventa un limite critico in termini di performance quando è necessario modificare ripetutamente un testo (ad esempio all'interno di un ciclo).

La classe `StringBuilder` (definita in `java.lang`) è progettata per gestire stringhe ****mutabili****. Internamente gestisce un array di caratteri (un buffer) che può essere modificato direttamente senza creare nuovi oggetti.

Osservazione 1.7 (Capacity vs Length). `StringBuilder` distingue tra:

- **Length**: Il numero di caratteri effettivamente contenuti.
- **Capacity**: La dimensione del buffer interno. Se la lunghezza supera la capacità, l'array viene automaticamente ridimensionato (solitamente raddoppiato), garantendo un'efficienza ammortizzata.

Listing 1.5: Esempio di utilizzo efficiente in un ciclo

```
1 StringBuilder sb = new StringBuilder();
2 for (int i = 0; i < 100; i++) {
3     sb.append("Elemento ").append(i).append("; ");
4 }
5 String risultato = sb.toString(); // Conversione finale in String
```

A differenza di `String`, questa classe offre metodi per la modifica *in-place*:

- `append(valore)`: Aggiunge qualsiasi tipo di dato alla fine della sequenza.
- `insert(offset, valore)`: Inserisce testo in un punto specifico spostando i caratteri successivi.
- `delete(start, end)`: Rimuove una porzione di testo.
- `reverse()`: Inverte l'ordine dei caratteri (operazione estremamente efficiente).



Ottimizzazione del Compilatore: Nelle versioni moderne di Java, il compilatore trasforma automaticamente le concatenazioni semplici (es. `s = a + b`) in istruzioni `StringBuilder`. Tuttavia, all'interno dei **cicli**, il compilatore non è in grado di applicare questa ottimizzazione in modo efficiente, rendendo l'uso manuale di `StringBuilder` un requisito fondamentale per scrivere codice professionale. Non conviene invece usarlo se devono solo unire due o tre stringhe una sola volta: l'overhead per istanziare l'oggetto `StringBuilder` renderebbe il codice inutilmente complesso e leggermente più lento rispetto ad una semplice concatenazione con l'operatore `+`.

1.6 Input e output su console

1.6.1 Scanner

Per leggere l'input dalla console (standard input), Java mette a disposizione la classe `Scanner`, definita nel pacchetto `java.util`.

Per utilizzare lo `Scanner`, bisogna creare un'istanza collegata allo stream `System.in`:

Listing 1.6: Inizializzazione dello Scanner

```
1 import java.util.Scanner;
2
3 Scanner in = new Scanner(System.in);
```

I metodi principali per acquisire dati sono:

- `nextLine()`: legge un'intera riga di testo (inclusi gli spazi).
- `next()`: legge una singola parola (si ferma al primo spazio bianco).
- `nextInt()`, `nextDouble()`: leggono rispettivamente un intero o un numero decimale.

Osservazione 1.8. A differenza del C, dove la gestione dell'input con `scanf` può essere complessa e rischiosa per la sicurezza (buffer overflow), la classe `Scanner` gestisce internamente l'allocazione della memoria, rendendo l'acquisizione dei dati più astratta e sicura.

1.6.2 Output su Console

In Java, l'output standard viene gestito principalmente attraverso l'oggetto `System.out`. Esistono tre metodi fondamentali per inviare dati alla console:

- `print()`: Stampa il contenuto senza aggiungere un ritorno a capo alla fine.
- `println()`: Stampa il contenuto e aggiunge automaticamente un carattere di *newline* (`\n`). È il metodo più utilizzato per il debug rapido.
- `printf()`: Introdotto per la compatibilità con lo stile del linguaggio C, permette la stampa di testo formattato tramite l'uso di *placeholder* (specificatori di formato).

Lo specificatore printf

La sintassi di `printf` segue lo schema: `System.out.printf(formato, argomenti)`. Ecco i simboli più comuni:

Simbolo	Tipo di dato
<code>%d</code>	Intero decimale
<code>%s</code>	Stringa
<code>%f</code>	Numero a virgola mobile
<code>%n</code>	Ritorno a capo (piattaforma-indipendente)

Listing 1.7: Esempio di output formattato

```
1 double quota = 1000.0 / 3.0;
2 // Stampa con 2 cifre decimali e un totale di 8 spazi di
  larghezza
3 System.out.printf("%8.2f", quota);
```

Osservazione 1.9. Mentre in C potevi accidentalmente passare un argomento del tipo sbagliato a `printf` causando crash o comportamenti indefiniti, in Java il compilatore e la JVM effettuano controlli più rigorosi, lanciando un'eccezione (`IllegalFormatException`) se il tipo dell'argomento non corrisponde allo specificatore.



Se hai bisogno di formattare una stringa senza stamparla immediatamente, puoi usare `String.format("...", args)`, che restituisce una nuova stringa formattata seguendo le stesse regole di `printf`.

1.7 Lettura e Scrittura su File

Per operare con i file, Java estende l'uso della classe `Scanner` per la lettura e introduce la classe `PrintWriter` per la scrittura. Entrambe richiedono la gestione dei percorsi tramite la classe `Path`.

1.7.1 Lettura da File

Per leggere un file, non è sufficiente passare una stringa con il nome del file allo `Scanner`. È necessario creare un oggetto `Path`.

Listing 1.8: Lettura corretta di un file

```
1 import java.nio.file.Path;
2 import java.util.Scanner;
3 import java.nio.charset.StandardCharsets;
4
5 // Creazione del percorso e apertura dello scanner
6 Scanner in = new Scanner(Path.of("miofile.txt"), StandardCharsets
    .UTF_8);
```



Se scrivi `Scanner in = new Scanner("miofile.txt")`, lo `Scanner` non aprirà il file, ma leggerà la stringa stessa come se fosse il contenuto dei dati. L'uso di `Path.of` è obbligatorio per puntare al file fisico.

1.7.2 Scrittura su File

Per scrivere testo su un file, si utilizza la classe `PrintWriter`. Se il file non esiste, verrà creato; se esiste, il contenuto verrà sovrascritto.

Listing 1.9: Scrittura su file

```
1 import java.io.PrintWriter;
2
3 PrintWriter out = new PrintWriter("output.txt", StandardCharsets.
    UTF_8);
4 out.println("Risultato del calcolo: " + risultato);
5 out.close(); // Fondamentale per svuotare il buffer e salvare
```

1.7.3 Gestione dei Percorsi e Eccezioni

Quando si lavora con i file, bisogna considerare due aspetti critici:

- **Percorsi:** Puoi usare percorsi relativi o assoluti (es. `C:\\Users\\test.txt`). Ricorda che nei percorsi Windows inseriti nel codice Java il backslash va raddoppiato.
- **Eccezioni:** L'accesso ai file può fallire (file mancante, permessi negati). Per ora, Java ti obbliga a dichiarare che il metodo `main` può generare un errore aggiungendo `throws IOException` alla firma del metodo.



Evoluzione del Charset (JEP 400): Sebbene a partire da Java 18 l'encoding predefinito sia diventato ufficialmente `UTF_8`, è comunque consigliabile specificare `StandardCharsets.UTF_8` (spesso considerato ridondante) in modo esplicito. Questo garantisce la portabilità del codice su versioni precedenti di Java e rende l'intento del programmatore inequivocabile, evitando che il comportamento del software dipenda implicitamente dalle impostazioni della JVM.

1.8 Il Costrutto Switch

Lo `switch` permette di selezionare un ramo di esecuzione basandosi sul valore di un'espressione. Java 21 ha trasformato radicalmente questo costrutto.

1.8.1 Lo Switch Tradizionale (Statement)

La forma classica (stile C) utilizza i due punti (`:`) e richiede il `break` per evitare il *fall-through*.

Listing 1.10: Switch tradizionale

```
1 switch (scelta) {  
2     case 1:  
3         System.out.println("Uno");  
4         break;  
5     case 2:  
6         System.out.println("Due");  
7         break;  
8     default:  
9         System.out.println("Altro");  
10 }
```

1.8.2 La Sintassi a Freccia (Modern Switch Statement)

Introdotta per risolvere l'intrinseca fragilità del `break`, questa sintassi utilizza l'operatore `->`. In questa forma, lo `switch` è ancora uno **statement** (esegue un'azione ma non restituisce un valore).

- **Niente break:** Viene eseguito solo il codice a destra della freccia. Non c'è rischio di cadere nel caso successivo.
- **Casi multipli:** Si possono raggruppare più valori con la virgola: `case 1, 2, 3 ->`
....

Listing 1.11: Switch moderno con freccia

```
1 switch (scelta) {
2     case 1 -> System.out.println("Uno");
3     case 2 -> System.out.println("Due");
4     case 3, 4 -> {
5         // Se serve piu di una riga, si usano le graffe
6         System.out.println("Tre o Quattro");
7     }
8     default -> System.out.println("Altro");
9 }
```

1.8.3 Switch Expressions

Quando lo `switch` viene usato per assegnare un valore a una variabile, diventa una **expression**. In questo caso, il costrutto **deve** essere esaustivo (coprire tutti i casi o avere un default).

Listing 1.12: Switch come espressione

```
1 String etichetta = switch (scelta) {
2     case 1 -> "Uno";
3     case 2 -> "Due";
4     default -> "Altro";
5 }; // Nota il punto e virgola finale, obbligatorio nelle
    espressioni
```

L'istruzione yield

Se un ramo della *switch expression* richiede un blocco logico complesso, si usa `yield` per "emettere" il valore finale del blocco.

Listing 1.13: Uso di yield

```
1 int calcolo = switch (modo) {
2     case 1 -> 10;
3     default -> {
4         int temp = eseguiCalcolo();
5         yield temp * 2;
6     }
7 };
```

1.8.4 Evoluzioni Avanzate: Pattern Matching e Null

Solo una volta compresa la sintassi a freccia, possiamo sfruttare la potenza di Java 21 per testare i **tipi** degli oggetti e gestire i valori `null`.

Listing 1.14: Pattern Matching e gestione null

```
1 switch (obj) {
2     case null -> System.out.println("Nessun oggetto");
3     case Integer i -> System.out.println("Intero: " + i);
4     case String s -> System.out.println("Stringa: " + s);
5     default -> System.out.println("Tipo non supportato");
6 }
```


💡 Sebbene esistano diverse combinazioni sintattiche per lo **switch** (incrociando l'uso di `:` o `->` con la natura di *statement* o *expression*), la buona pratica in Java moderno è orientata verso la massima sicurezza. Si tende a utilizzare quasi esclusivamente la **sintassi a freccia** (`->`) per eliminare alla radice il rischio di *fall-through* e a preferire, laddove possibile, le **switch expressions** per rendere il codice più dichiarativo e meno incline a errori di stato.

1.8.5 L'Utilizzo della Clausola **when** negli **switch**

La clausola **when** (o *Guards*) estende la potenza dell'istruzione **switch** permettendo di aggiungere condizioni booleane specifiche a ogni **case**.

Invece di limitarsi a confrontare un valore costante, **when** permette di filtrare il match in base a variabili dinamiche.

Listing 1.15: Esempio di Pattern Matching con clausola **when**

```

1 switch (dispositivo) {
2     case Smartphone s when s.getLivelloBatteria() < 10 ->
3         pulisciMemoria();
4     case Smartphone s when s.isModalitaAereo() ->
5         disabilitaRete();
6     default ->
7         attendiSegnale();
8 }

```

L'introduzione di questa clausola è particolarmente vantaggiosa in tre scenari principali:

- **Rimozione dei rami condizionali annidati:** Evita di dover inserire blocchi **if-else** all'interno di un **case**, rendendo il codice più lineare e leggibile (cosiddetto *Flattening*).
- **Pattern Matching Avanzato:** Permette di gestire lo stesso tipo di oggetto in modi diversi nello stesso **switch**. Ad esempio, posso avere due **case** per la classe **Ordine**, uno **when total > 1000** e uno per i restanti.
- **Espressività Semantica:** Sposta la logica di controllo direttamente nella firma del caso, rendendo immediatamente chiaro sotto quali condizioni quel ramo viene eseguito.

1.9 Big Numbers: **BigInteger** e **BigDecimal**

Quando la precisione dei tipi primitivi interi (**long**) o a virgola mobile (**double**) non è sufficiente, il pacchetto **java.math** mette a disposizione due classi fondamentali: **BigInteger** e **BigDecimal**. Queste classi permettono di manipolare numeri con una sequenza di cifre arbitrariamente lunga.

Esistono tre modi principali per creare un "Big Number":

- **Metodo statico `valueOf`:** Ideale per convertire numeri ordinari.

```
1 BigInteger a = BigInteger.valueOf(100);
```

- **Costruttore con `String`:** Indispensabile per numeri che superano la capacità dei tipi primitivi.

```
1 BigInteger reallyBig = new BigInteger("
    222322446294204455297398934619099672066669390964000099");
```

- **Costanti predefinite:** `BigInteger.ZERO`, `ONE`, `TWO` e `TEN`.



Per `BigDecimal`, si deve **sempre** usare il costruttore che accetta una `String`. Il costruttore `BigDecimal(double)` è intrinsecamente pronò a errori di arrotondamento dovuti alla rappresentazione binaria dei floating-point.

Ad esempio, `new BigDecimal(0.1)` non produce esattamente 0.1, ma una sequenza imprecisa (com'è noto da nozioni di architettura dei calcolatori, 0.1 in binario è un numero periodico infinito, quindi viene approssimato):

```
0.10000000000000000055511151231257827021181583404541015625
```

L'uso della stringa `"0.1"` garantisce invece la precisione esatta richiesta.

Poiché Java non supporta l'overloading degli operatori (a differenza di C++), non è possibile usare `+`, `-`, `*` o `/`. È necessario utilizzare i metodi d'istanza dedicati.

Operazione	Metodo Java
Somma (+)	<code>a.add(b)</code>
Sottrazione (−)	<code>a.subtract(b)</code>
Moltiplicazione (*)	<code>a.multiply(b)</code>
Divisione (/)	<code>a.divide(b)</code>
Modulo (%)	<code>a.mod(b)</code>

Tabella 1.2: Metodi aritmetici per `BigInteger` e `BigDecimal`.

Osservazione 1.10 (Immutabilità). Proprio come le `String`, anche i Big Numbers sono **immutabili**. Metodi come `add` non modificano l'oggetto originale nello **Heap**, ma restituiscono una nuova istanza con il risultato.

Classi e oggetti

2.1 I Costruttori

Un costruttore è un metodo speciale che viene invocato esclusivamente tramite l'operatore `new`. Esso rappresenta il blocco di codice che viene eseguito nel momento esatto in cui un nuovo oggetto di una classe viene istanziato.

- **Scopo:** L'obiettivo primario di un costruttore è impostare lo stato iniziale dell'oggetto, ovvero inizializzare i suoi campi di istanza affinché l'oggetto nasca in uno stato valido.
- **Nome e Ritorno:** Per convenzione del linguaggio, il costruttore deve avere lo stesso identico nome della classe e **non deve specificare alcun tipo di ritorno** (nemmeno `void`).

Listing 2.1: Esempio di costruttore base

```
1 public class Employee {
2     private String name;
3     private double salary;
4
5     // Costruttore
6     public Employee(String n, double s) {
7         name = n;
8         salary = s;
9     }
10 }
```

Osservazione 2.1. È importante ricordare che un costruttore non può essere invocato su un oggetto già esistente per "re-inizializzarlo". Viene eseguito una sola volta per ogni istanza, all'atto della creazione.

Spesso, per rendere il codice più leggibile ed evitare la proliferazione di identificatori, si tende a dare ai parametri del costruttore lo stesso nome dei campi di istanza della classe. In questi casi, il nome del parametro "nasconde" (*shadowing*) il campo di istanza. Per risolvere l'ambiguità e indicare esplicitamente che ci si riferisce alla variabile dell'oggetto e non al parametro locale, si utilizza la parola chiave `this`.

In Java, `this` è un riferimento all'oggetto corrente su cui il metodo o il costruttore è stato invocato. Scrivere `this.nomeCampo` indica in modo univoco al compilatore di accedere alla variabile memorizzata nello *heap* per quell'istanza specifica.

Listing 2.2: Uso di `this` nel costruttore

```
1 public class Employee {
2     private String name;
3     private double salary;
4
5     public Employee(String name, double salary) {
6         // this.name si riferisce al campo istanza
7         // name si riferisce al parametro del costruttore
8         this.name = name;
9     }
10 }
```

```
9         this.salary = salary;
10     }
11 }
```

2.1.1 Ulteriori note su `this`

Oltre alla necessità tecnica di risolvere conflitti di nomi, molti programmatori utilizzano `this` all'interno dei metodi e dei costruttori anche laddove non sarebbe strettamente necessario (ovvero quando i nomi dei parametri sono diversi da quelli dei campi).

Questa pratica ha un valore puramente **visivo e documentale**: anteporre `this` permette di distinguere immediatamente, a colpo d'occhio, quali variabili appartengono allo stato permanente dell'oggetto e quali sono invece variabili locali o parametri temporanei. In progetti di grandi dimensioni, questa convenzione aiuta a ridurre il carico cognitivo di chi legge il codice e riduce drasticamente il rischio di errori accidentali durante la manutenzione dello stesso.

In Java, il riferimento `this` punta sempre all'oggetto corrente (il cosiddetto *parametro implicito*). Il suo utilizzo si divide in tre casi principali:

- **`this.nomeCampo`**: fondamentale, come si è visto poco sopra, per risolvere lo *shadowing*, ovvero quando un parametro del metodo o del costruttore ha lo stesso nome di un campo della classe.
- **`this.nomeMetodo(parametri)`**: utilizzato per invocare un altro metodo sulla stessa istanza.
- **`this(parametri)`**: Questa forma speciale permette a un costruttore di richiamarne un altro all'interno della stessa classe. Come vedremo in paragrafi a seguire, è la tecnica principe per gestire l'**overloading dei costruttori** senza duplicare il codice di inizializzazione. *Nota bene: deve essere obbligatoriamente la prima istruzione del costruttore.*

2.2 Overloading e Firma dei Metodi

L'overloading consente di definire più costruttori o metodi con lo stesso nome, purché abbiano parametri differenti.

- **Firma (Signature)**: È l'identità del metodo, composta da *nome del metodo*, *numero e tipo dei parametri* e *l'ordine in cui sono dichiarati*.
- **Esclusione del Ritorno**: Il tipo di ritorno **non** fa parte della firma. Non è possibile avere due metodi con parametri identici ma tipi di ritorno diversi.

In questo esempio, la classe `Employee` gestisce tre campi: `name`, `salary` e `hireDay`. Utilizziamo l'overloading per offrire diversi gradi di flessibilità nella creazione dell'oggetto, delegando sempre l'inizializzazione al costruttore più completo.

Listing 2.3: Overloading a cascata nella classe `Employee`

```
1 public class Employee {
2     private String name;
3     private double salary;
4     private LocalDate hireDay;
5
6     // 1. Costruttore "Master" (3 parametri)
7     // E' l'unico che contiene la logica di assegnazione
    effettiva.
```

```
8      public Employee(String name, double salary, LocalDate hireDay
9          ) {
10          this.name = name;
11          this.salary = salary;
12          this.hireDay = hireDay;
13      }
14
15      // 2. Costruttore con 2 parametri
16      // Imposta la data di assunzione a "oggi" per default.
17      public Employee(String name, double salary) {
18          this(name, salary, LocalDate.now());
19      }
20
21      // 3. Costruttore con 1 parametro
22      // Imposta lo stipendio a 0 e la data a "oggi" per default.
23      public Employee(String name) {
24          this(name, 0, LocalDate.now());
25      }
26
27      // 4. Costruttore vuoto (no-arg)
28      // Imposta valori generici di default.
29      public Employee() {
30          this("Sconosciuto", 0, LocalDate.now());
31      }
32  }
```

Osservazione 2.2. Il vantaggio della manutenzione. Se in futuro si volesse aggiungere un controllo di validità (ad esempio, impedire che lo stipendio sia negativo), basterebbe inserire la logica nel **costruttore master**. Tutti gli altri costruttori ne beneficerebbero automaticamente, garantendo la coerenza dello stato dell'oggetto indipendentemente da come viene creato.

Osservazione 2.3. Si parla dunque di overloading di un metodo se ci sono due metodi con lo stesso nome ma con firma diversa. Anche se il numero di parametri è lo stesso ma cambia l'ordine, sempre di overloading si tratta, perché cambia la firma. Di fatto, anche se l'identificatore del metodo è lo stesso, si tratta di metodi diversi. Quindi l'overloading non è altro che un modo per evitare il proliferare di identificatori di metodi ed è comodo anche per continuare a descrivere (tramite un nome appropriato) cosa fa un metodo anche se cambiano i parametri.



Ricorda che la chiamata `this(...)` deve essere tassativamente la **prima riga** del costruttore. Non puoi eseguire alcun calcolo o istruzione prima di delegare la costruzione all'altro costruttore.

2.3 Costruzione degli Oggetti

Java offre diversi meccanismi per inizializzare lo stato di un oggetto. Per evitare confusione, è utile osservare il processo come una sequenza di regole logiche.

2.3.1 Inizializzazione e Valori di Default

A differenza delle variabili locali (che devono essere inizializzate esplicitamente), i campi di istanza vengono sempre azzerati automaticamente se non specificato diversamente:

- Numeri (int, double, ecc.): 0.
- Booleani: `false`.
- Riferimenti a oggetti: `null`.



Il costruttore predefinito "gratis". Java fornisce implicitamente un costruttore senza argomenti (*no-arg constructor*) solo se nella classe non è stato definito **nessun** costruttore. Se ne definisci anche solo uno con parametri, quello predefinito scompare; se ti serve ancora, devi scriverlo esplicitamente. In ogni caso è importante ricordare che un costruttore **esiste sempre**, anche se non lo si dichiara.

2.3.2 Blocchi di Inizializzazione

Oltre all'assegnazione diretta (nelle dichiarazioni dei campi di istanza) e ai costruttori, Java mette a disposizione un terzo meccanismo per inizializzare i campi: i **blocchi di inizializzazione**. Si tratta di segmenti di codice racchiusi tra parentesi graffe inseriti direttamente nel corpo della classe.

Esistono due tipi di blocchi, con scopi e tempi di esecuzione differenti:

- **Blocchi di istanza:** Vengono eseguiti ogni volta che viene creata una nuova istanza della classe, subito dopo l'inizializzazione dei campi ai valori predefiniti e prima del corpo del costruttore.
- **Blocchi statici (`static { ... }`):** Vengono eseguiti una sola volta, nel momento in cui la classe viene caricata in memoria dalla JVM. Sono ideali per configurazioni complesse di variabili statiche (ad esempio, caricare dati da un file o inizializzare un generatore di numeri casuali).

Osservazione 2.4. Sebbene potenti, i blocchi di inizializzazione di istanza sono raramente necessari e possono rendere il codice meno leggibile. La pratica consigliata è quella di inserire la logica di inizializzazione direttamente nei costruttori o di usare la concatenazione tramite `this(...)`.

2.3.3 L'Ordine di Esecuzione

Quando viene invocato l'operatore `new`, Java segue un ordine tassativo:

1. Tutti i campi sono azzerati (valori di default).
2. Vengono eseguiti gli eventuali inizializzatori espliciti (es. `private int id = 1;`) e gli eventuali blocchi di inizializzazione (`{...}`) nell'ordine in cui appaiono nel file.
3. Viene infine eseguito il corpo del costruttore selezionato.

2.4 L'inferenza del tipo: la parola chiave `var`

A partire da Java 10, il linguaggio ha introdotto la parola chiave `var`, un meccanismo noto come *local variable type inference*. Per un programmatore abituato al rigore del C, questo potrebbe sembrare un passaggio verso la tipizzazione dinamica, ma la realtà è differente: Java rimane un linguaggio a tipizzazione statica.

Il senso del `var` è eliminare la ridondanza visiva. Spesso, in Java, il nome del tipo appare sia nella dichiarazione che nell'inizializzazione (es. `Employee e = new Employee()`).

Poiché il compilatore è in grado di dedurre il tipo osservando l'espressione alla destra dell'operatore di assegnazione, il `var` permette di omettere la dichiarazione esplicita del tipo senza perdere in sicurezza.

Listing 2.4: Confronto tra dichiarazione esplicita e `var`

```
1 // Approccio tradizionale
2 Employee harry = new Employee("Harry Hacker", 50000);
3
4 // Approccio con inferenza del tipo
5 var staff = new Employee("Harry Hacker", 50000);
```

Esistono tuttavia dei limiti tecnici e stilistici precisi per l'utilizzo di questo strumento:

- **Ambito locale:** Può essere utilizzato esclusivamente per le variabili locali all'interno dei metodi. Non è ammesso per i campi di istanza di una classe, dove il tipo deve essere sempre esplicito.
- **Inizializzazione immediata:** Una variabile dichiarata con `var` deve essere inizializzata nello stesso momento in cui viene dichiarata, altrimenti il compilatore non avrebbe elementi per dedurre il tipo.
- **Leggibilità:** Horstmann suggerisce di usare `var` solo quando il tipo è ovvio dalla lettura del lato destro. Ad esempio, `var in = new Scanner(System.in)` è chiaro, mentre `var x = calcola()` potrebbe rendere il codice criptico se non si conosce il valore di ritorno del metodo.

⚠ È fondamentale ricordare che il tipo viene fissato a tempo di compilazione. Una volta che `var staff` è stato interpretato come `Employee`, non potrà mai contenere un tipo diverso (come una stringa o un intero) durante l'esecuzione del programma.

2.5 Ulteriori note sull'incapsulamento

L'incapsulamento non consiste solo nel nascondere i dati, ma nel fornire un'interfaccia controllata per interagire con essi. I principali vantaggi sono:

1. **Debugging semplificato:** Se un campo viene modificato solo tramite un mutatore, la ricerca di eventuali bug legati a valori errati è circoscritta a quel metodo specifico.
2. **Trasparenza dell'implementazione:** La struttura interna dei dati può evolvere (es. dividere un campo `name` in `firstName` e `lastName`) senza che il codice esterno debba essere modificato, poiché l'interfaccia dei metodi rimane costante.
3. **Integrità dei dati:** I metodi mutatori possono eseguire controlli di validità (es. impedire stipendi negativi) prima di aggiornare lo stato interno.

2.5.1 Il rischio della fuga di riferimenti (*Leaking References*)

Un errore di progettazione sottile ma grave consiste nello scrivere metodi accessori che restituiscono riferimenti a oggetti **mutabili**.

⚠ Riferimenti mutabili e rottura dell'incapsulamento

Se un campo privato è un oggetto mutabile (come la vecchia classe `Date`), restituire quel riferimento tramite un getter permette al codice esterno di modificare lo stato interno dell'oggetto all'insaputa della classe stessa.

Si consideri il seguente esempio di “codice canaglia”:

Listing 2.5: Violazione dell’incapsulamento

```
1 Employee harry = ...;
2 Date d = harry.getHireDay(); // d punta all'oggetto interno di
   harry
3 double tenYearsInMilliseconds = 10 * 365.25 * 24 * 60 * 60 *
   1000;
4 d.setTime(d.getTime() - (long) tenYearsInMilliseconds);
5 // Lo stato di harry è cambiato senza usare i metodi di Employee!
```

⚠ Attenzione

Per prevenire questa vulnerabilità, esistono due strategie principali:

- **Utilizzare oggetti immutabili:** Classi come `LocalDate` o `String` non possiedono metodi mutatori. Una volta creati, il loro stato non può cambiare, rendendo sicuro il ritorno del loro riferimento.
- **Clonazione:** Se si deve assolutamente usare un oggetto mutabile, il getter dovrebbe restituire una copia (*clone*) dell’oggetto, non l’originale. Vedremo più avanti in dettaglio questo tipo di soluzione.

2.6 I Metodi Privati

Nella grande maggioranza dei casi, i metodi di una classe vengono dichiarati con il modificatore `public`, poiché costituiscono l’interfaccia attraverso cui il mondo esterno interagisce con l’oggetto. Tuttavia, esistono circostanze in cui è preferibile o necessario rendere un metodo `private`.

Helper Methods e Pulizia del Codice Spesso, l’implementazione di un compito complesso richiede di spezzare il codice in diverse sotto-operazioni. Questi “metodi di supporto” (*helper methods*) non dovrebbero far parte dell’interfaccia pubblica: potrebbero essere troppo legati all’attuale implementazione interna o richiedere un protocollo di chiamata specifico che non deve essere esposto all’utente della classe. Dichiarandoli privati, ci si assicura che vengano utilizzati solo internamente.

Libertà di Evoluzione del Software Il vantaggio principale della riservatezza dei metodi risiede nella libertà di manutenzione.

- **Metodi Pubblici:** Rappresentano un impegno verso chi utilizza la classe. Se un metodo è pubblico, non può essere rimosso o modificato drasticamente senza rischiare di “rompere” il codice esterno che vi fa affidamento.
- **Metodi Privati:** Finché un metodo rimane privato, il progettista della classe ha la certezza che non venga utilizzato altrove. Questo permette di rinominarlo, ottimizzarlo o addirittura eliminarlo completamente in futuro, qualora la rappresentazione interna dei dati dovesse cambiare, senza alcun impatto sul resto del programma.

Osservazione 2.5. L’uso di metodi privati è una forma di incapsulamento del comportamento. Proprio come i campi privati proteggono lo stato dell’oggetto da manipolazioni esterne, i metodi privati proteggono la logica di implementazione, garantendo che l’utente

della classe veda solo ciò che è strettamente necessario al corretto funzionamento del sistema.

2.7 Campi di istanza **final**

In Java, è possibile dichiarare un campo di istanza come **final**. Tale modificatore impone che la variabile venga inizializzata tassativamente entro la fine di ogni costruttore della classe. Una volta assegnato, il valore del campo non può più essere modificato.

Tipi primitivi e Classi Immutabili L'uso di **final** è particolarmente efficace con i tipi primitivi o con classi immutabili (ovvero classi i cui metodi non modificano mai lo stato dell'oggetto, come **String** o **LocalDate**). In questi casi, il campo diventa a tutti gli effetti una costante dopo la costruzione dell'oggetto.

La trappola delle Classi Mutabili È necessario prestare attenzione quando si usa **final** con oggetti mutabili.



Il modificatore **final** garantisce solo che il **riferimento** memorizzato nella variabile non punti mai a un oggetto diverso. Tuttavia, non impedisce la mutazione dell'oggetto stesso: per i mutabili **final** è solo il riferimento all'oggetto ad essere costante, ma tutto ciò a cui punta quel riferimento può essere cambiato.

Ad esempio, un campo `private final StringBuilder evaluations` impedisce di assegnare un nuovo **StringBuilder** alla variabile, ma permette comunque di invocare metodi come `evaluations.append()`, modificando di fatto lo stato interno dell'oggetto.

Osservazione 2.6. Un campo **final** può essere inizializzato con il valore **null** (magari a seguito di un controllo condizionale). In tal caso, la variabile rimarrà **null** per l'intero ciclo di vita dell'istanza e non potrà mai essere riassegnata a un oggetto reale.

2.8 Membri Statici: Campi e Costanti

Il modificatore **static** permette di definire membri che appartengono alla **classe** stessa piuttosto che alle singole istanze.

Campi Statici (Variabili di Classe) Se un campo è dichiarato **static**, ne esiste un'unica copia condivisa da tutti gli oggetti della classe. Anche se non viene istanziato alcun oggetto, il campo statico è presente in memoria.

Esempio 2.1. Nella classe **Employee**, un campo `private int id` è un campo di istanza (ogni dipendente ha il suo), mentre un campo `private static int nextId` è un campo di classe (condiviso da tutti per generare ID univoci).

Listing 2.6: Utilizzo di un campo statico nel costruttore

```
1 public Employee(...) {  
2     id = nextId; // Assegna l'ID corrente  
3     nextId++;   // Incrementa il contatore condiviso nella  
4                 classe  
5 }
```

Mentre le variabili statiche sono rare (e vanno usate con cautela!) le costanti statiche (`static final`) sono molto comuni. Esse permettono di accedere a valori fissi tramite il nome della classe, senza dover istanziare oggetti.

Osservazione 2.7. È buona norma accedere ai membri statici usando il nome della classe (es. `Employee.nextId`) invece che tramite un riferimento a un oggetto (es. `harry.nextId`), per rendere subito chiaro a chi legge che il campo non appartiene all'istanza.

I Metodi Statici Qui il concetto è del tutto analogo a quello dei campi di istanza: si tratta di metodi comuni a tutta la classe. A differenza dei metodi di istanza, un metodo statico non ha un parametro implicito `this`; in altre parole, non riceve alcun riferimento a un'istanza specifica della classe quando viene invocato.

Poiché non dispongono del riferimento `this`, i metodi statici presentano caratteristiche precise:

- **Accesso ai dati:** un metodo statico può accedere ai campi statici della classe, ma non può mai accedere ai campi di istanza. Non operando su un oggetto, il metodo non “saprebbe” a quale istanza riferirsi per leggere i dati non statici.
- **Invocazione:** come per i campi, sebbene il linguaggio permetta di invocare un metodo statico tramite il riferimento a un oggetto, la buona pratica prevede di utilizzare sempre il nome della classe. Questo chiarisce immediatamente che l'operazione non dipende dallo stato di un singolo oggetto.

È appropriato definire un metodo come `static` quando il metodo deve solo eseguire un calcolo basato su parametri forniti esplicitamente (come `Math.pow(x, a)`), oppure quando il metodo deve accedere esclusivamente a campi statici della propria classe (come un metodo che incrementa un contatore globale di istanze).

2.9 Il Metodo `main`

Il metodo `main` è il punto di ingresso di ogni applicazione Java. La sua natura riflette la necessità della Java Virtual Machine (JVM) di avviare l'esecuzione prima ancora che vengano creati gli oggetti.

Il metodo `main` deve essere dichiarato `static` perché deve poter essere invocato dalla JVM senza che sia necessaria un'istanza della classe. Quando il programma parte, non esiste ancora alcun oggetto: il `main` statico viene eseguito, "accende il motore" e inizia a creare gli oggetti necessari all'applicazione.

Una caratteristica potente di Java è che **ogni classe** può contenere un metodo `main`. Questo non significa che l'applicazione avrà più punti di ingresso, ma permette di inserire del codice di test o di dimostrazione direttamente all'interno della classe stessa.

- Se si lancia la classe specifica (es. `java Employee`), viene eseguito il suo `main` (utile per testare se i metodi funzionano correttamente).
- Se la classe è usata all'interno di un'applicazione più grande, il suo `main` viene semplicemente ignorato.

Novità Java 21: Instance Main Methods (Preview) A partire da Java 21, è stata introdotta una *preview feature* che semplifica l'avvio dei programmi. In questa nuova modalità:

- Il `main` non deve più essere obbligatoriamente `static` o `public`.
- Può non avere il parametro `String[] args`.

Se il `main` non è statico, la JVM crea automaticamente un'istanza della classe (usando il costruttore predefinito) e invoca il metodo su di essa. Questa evoluzione punta a ridurre il "boilerplate code" per i principianti e per piccoli script.

Osservazione 2.8. Per un programmatore C, il `main` di Java è l'equivalente della funzione globale `main()`, ma incapsulato in una classe per rispettare il paradigma ad oggetti. La versione "non statica" di Java 21 avvicina ancora di più il linguaggio alla semplicità dei linguaggi di scripting.

2.10 Il passaggio dei parametri: Call-by-Value

Esiste spesso confusione sulla modalità con cui Java passa i parametri ai metodi. La regola ferrea del linguaggio è una sola: **Java utilizza esclusivamente il passaggio per (copia del) valore (call-by-value)**. Ciò significa che il metodo riceve sempre una copia dei valori contenuti nelle variabili fornite come argomenti. Copie che, di fatto, sono quindi **variabili locali**.

Il comportamento osservato differisce in base a ciò che la variabile contiene:

- **Tipi Primitivi:** La variabile contiene il valore numerico o booleano. Il metodo riceve una copia di tale valore; ogni modifica effettuata all'interno del metodo rimane locale e non influenza la variabile originale.
- **Riferimenti a Oggetti:** La variabile contiene l'indirizzo di memoria dell'oggetto. Il metodo riceve una copia di questo indirizzo. Poiché sia la variabile originale che la copia puntano allo stesso oggetto, è possibile modificarne lo stato interno, ma **non è possibile** far sì che la variabile originale punti a un nuovo oggetto. In questo caso si parla di *passaggio per copia del valore del riferimento*.



Il fallimento dello swap

La dimostrazione definitiva che Java non usa il passaggio per riferimento è l'impossibilità di scrivere un metodo `swap(x, y)` che scambi i riferimenti di due oggetti esterni. Il metodo scambierà solo le sue copie locali, lasciando invariate le variabili originali.

Osservazione 2.9. Il caso degli oggetti immutabili. Sebbene per le classi immutabili (come `String`, `Integer`, `LocalDate`) valga la regola del passaggio per copia del valore del riferimento, il loro comportamento pratico è identico a quello dei tipi primitivi. Poiché l'oggetto non può essere modificato internamente, qualsiasi operazione di "modifica" effettuata nel metodo (es. `s = s + "!"`) non è altro che una riassegnazione della variabile locale a un nuovo oggetto, lasciando il riferimento originale del chiamante del tutto invariato.

Osservazione 2.10. Per un programmatore C, questo sistema è identico al passaggio di un puntatore per valore: si può manipolare il dato puntato, ma non si può modificare l'indirizzo di memoria memorizzato nel puntatore originale del chiamante.

2.11 Metodi con numero variabile di argomenti (Varargs)

Java permette di definire metodi che accettano un numero arbitrario di argomenti dello stesso tipo attraverso la sintassi dei **varargs** (puntini di sospensione `...`).

Un parametro dichiarato come `Type... name` viene trattato all'interno del metodo come un array di tipo `Type[]`. Il compilatore si occupa di raccogliere gli argomenti passati nella chiamata e di incapsularli in un array in modo trasparente per lo sviluppatore.

Listing 2.7: Esempio di funzione max con varargs

```
1 public static double max(double... values) {
2     double largest = Double.NEGATIVE_INFINITY;
3     for (double v : values) { // values e' trattato come double[]
4         if (v > largest) largest = v;
5     }
6     return largest;
7 }
```

 Il parametro varargs deve essere necessariamente l'**ultimo** nella firma del metodo. È possibile avere un solo parametro varargs per metodo.

Osservazione 2.11 (Efficienza e Varargs). È fondamentale distinguere tra l'utilizzo di *varargs* con tipi primitivi e con tipi oggetto (o classi wrapper):

- **Tipi Primitivi** (`double...`): Il compilatore crea un array di primitivi (`double[]`). Non avviene alcun autoboxing e i dati sono memorizzati in modo contiguo. Questa è la soluzione più efficiente.
- **Tipi Oggetto** (`Double...` o `Object...`): Se vengono passati valori primitivi, il compilatore deve eseguire l'autoboxing per **ogni singolo argomento** prima di inserirli nell'array.

In scenari di calcolo intensivo, la differenza di performance è significativa: il passaggio di oggetti comporta un overhead dovuto sia alla creazione di istanze temporanee sullo *heap*, sia alla gestione di un array di riferimenti invece di un array di valori grezzi.

Poiché i varargs sono implementati come array, è perfettamente lecito passare un array pre-esistente al posto dei singoli elementi:

```
1 double[] prezzi = {10.5, 20.0, 5.75};
2 double massimo = max(prezzi); // Legale: passa l'array
   direttamente
```

La trappola degli array di primitivi

Bisogna prestare estrema attenzione quando si passa un array a un metodo con *varargs* di tipo `Object...`:

- Se si passa un **array di oggetti** (es. `String[]`), Java lo "spacchetta" e tratta ogni elemento come un argomento distinto.
- Se si passa un **array di primitivi** (es. `int[]`), Java **non** può trattarlo come un `Object[]`. Di conseguenza, considera l'intero array come un **singolo oggetto** e lo inserisce come unico elemento della lista parametri.

Per ottenere lo spacchettamento di un array di primitivi, è necessario convertirlo preventivamente in un array della relativa classe wrapper (`Integer[]`).

Tip

Anche il metodo `main` può essere dichiarato usando i varargs: `public static void main(String... args)`. Questo non cambia la funzionalità del programma



ma rende la firma leggermente più moderna.

2.12 Tipi Enumerati (Enum)

In Java, un **Enum** non è una semplice lista di costanti intere (come in C o C++), ma una classe speciale che estende implicitamente la classe `java.lang.Enum`. Rappresenta un insieme fisso di costanti tipizzate, garantendo che una variabile di quel tipo possa assumere solo uno dei valori predefiniti.

L'utilizzo degli `enum` (laddove sensato) è considerato una *best practice* per diversi motivi:

- **Type Safety:** Impedisce il passaggio di valori non validi (evitando l'uso di "Magic Numbers" o costanti `int`).
- **Namespace:** Le costanti sono confinate nel tipo enumerato, evitando conflitti di nomi.
- **Potenzialità di Classe:** Possono avere campi, costruttori e metodi, permettendo di associare dati e comportamenti a ogni costante.
- **Immutabilità:** Le istanze di un `enum` sono create all'avvio e non possono essere modificate, rendendole intrinsecamente *thread-safe*.

Di seguito è riportato lo schema più completo per definire un `enum`, includendo attributi, costruttori e metodi personalizzati.

Listing 2.8: Sintassi generale di un Enum in Java

```
1 public enum NomeEnum {
2     // 1. Definizione delle costanti (devono apparire per prime)
3     COSTANTE_A("Descrizione A", 10),
4     COSTANTE_B("Descrizione B", 20),
5     COSTANTE_C; // Se non ci sono argomenti, il costruttore e' di
6                 // default
7
8     // 2. Campi di istanza (solitamente private final per
9     // immutabilità)
10    private final String descrizione;
11    private final int valore;
12
13    // 3. Costruttore (sempre private o package-private)
14    private NomeEnum(String descrizione, int valore) {
15        this.descrizione = descrizione;
16        this.valore = valore;
17    }
18
19    // Costruttore di default (opzionale)
20    private NomeEnum() {
21        this.descrizione = "Default";
22        this.valore = 0;
23    }
24
25    // 4. Metodi (Getter, Logica di business, Override)
26    public String getDescrizione() {
27        return descrizione;
28    }
29 }
```

```

26     }
27
28     public int getValore() {
29         return valore;
30     }
31
32     @Override
33     public String toString() {
34         return String.format("%s (Codice: %d)", descrizione,
35                               valore);
36     }
37 }

```

Un `enum` viene utilizzato come un qualsiasi altro tipo di dato, con la differenza fondamentale che non può essere istanziato tramite l'operatore `new`. L'accesso alle costanti avviene tramite il nome della classe enumerata.

Esempio di integrazione in una Classe

Listing 2.9: Esempio di utilizzo di un Enum in una classe di business

```

1 public class Ordine {
2     private int id;
3     private StatoOrdine stato; // Riferimento all'enum
4
5     public Ordine(int id) {
6         this.id = id;
7         this.stato = StatoOrdine.NUOVO; // Valore iniziale
8     }
9
10    public void avanzaStato() {
11        // Esempio di switch moderno (Java 21+)
12        this.stato = switch (this.stato) {
13            case NUOVO -> StatoOrdine.IN_ELABORAZIONE;
14            case IN_ELABORAZIONE -> StatoOrdine.SPEDITO;
15            case SPEDITO -> StatoOrdine.CONSEGNATO;
16            case CONSEGNATO -> StatoOrdine.CONSEGNATO;
17        };
18    }
19 }

```

Ogni `enum` eredita automaticamente dei metodi statici molto potenti:

- `values()`: Restituisce un array contenente tutte le costanti dell'enum nell'ordine in cui sono dichiarate. Utile per iterazioni.
- `valueOf(String name)`: Converte una stringa nel corrispondente valore enum. Lancia `IllegalArgumentException` se la stringa non corrisponde a nessuna costante.
- `ordinal()`: Restituisce la posizione intera (base 0) della costante nella dichiarazione. *Nota: se ne sconsiglia l'uso per logiche di business persistenti poiché fragile rispetto a cambiamenti dell'ordine.*

Osservazione 2.12 (Confronto e Memoria). Poiché le istanze di un `enum` sono **singleton** (ne esiste una sola copia nello **Heap** per tutta la durata dell'applicazione), il confronto tra

due variabili enum può essere effettuato in modo sicuro ed efficiente utilizzando l'operatore `==` invece di `.equals()`. Questo non solo è più leggibile, ma evita anche potenziali `NullPointerException`.

2.13 I Records: classi come contenitori di dati

A partire da Java 16, il linguaggio introduce i **Records**, una forma specializzata di classe progettata per modellare aggregati di dati immutabili in modo conciso. I records eliminano il cosiddetto *boilerplate code* tipico delle classi Java tradizionali (POJO - Plain Old Java Objects).

Un record si definisce dichiarandone i campi, in gergo chiamati **componenti** del record, tra parentesi tonde:

Listing 2.10: Dichiarazione di un Record

```
1 public record Point(double x, double y) { }
```

Questa singola riga di codice equivale a una classe completa che include:

- Campi privati e finali: `private final double x;`
- Un costruttore che inizializza i campi (*costruttore canonico del record*).
- Metodi accessori (getters): `p.x()` e `p.y()` (nota l'assenza del prefisso `get`).
- Implementazioni standard di `toString()`, `equals()` e `hashCode()`.

Per comprendere appieno la mole di codice risparmiata, ecco la classe Java tradizionale (POJO) che definisce a livello semantico lo stesso identico comportamento del record `Point`.

Listing 2.11: Classe Java equivalente al Record Point

```
1 import java.util.Objects;
2
3 public final class Point {
4     private final double x;
5     private final double y;
6
7     // Costruttore canonico
8     public Point(double x, double y) {
9         this.x = x;
10        this.y = y;
11    }
12
13    // Metodi accessori (senza prefisso 'get')
14    public double x() {
15        return x;
16    }
17
18    public double y() {
19        return y;
20    }
21
22    // Implementazione standard di equals
23    @Override
24    public boolean equals(Object o) {
25        if (this == o) return true;
```

```

26         if (o == null || getClass() != o.getClass()) return false
27         ;
28         Point point = (Point) o;
29         return Double.compare(point.x, x) == 0 &&
30             Double.compare(point.y, y) == 0;
31     }
32
33     // Implementazione standard di hashCode
34     @Override
35     public int hashCode() {
36         return Objects.hash(x, y);
37     }
38
39     // Implementazione standard di toString
40     @Override
41     public String toString() {
42         return "Point[x=" + x + ", y=" + y + "]";
43     }

```

Da notare che la classe equivalente è marcata come `final`, poiché i records in Java non supportano l'ereditarietà (non possono essere estesi da altre classi).

Le caratteristiche principali dei record sono:

- **Immutabilità superficiale (Shallow Immutability):** I record sono progettati per trasportare dati immutabili. Una volta istanziato il record, i riferimenti ai suoi campi (le componenti) sono `final` e non possono essere riassegnati.
- **Estensibilità e limiti:** Un record può implementare interfacce, definire metodi di istanza, metodi statici e campi statici. Tuttavia, **non può** aggiungere campi di istanza oltre a quelli dichiarati nell'istestazione. Inoltre, non può essere esteso (è implicitamente `final`) e non può estendere classi esplicite (poiché estende già nativamente `java.lang.Record`).
- **Personalizzazione dei metodi:** È possibile effettuare l'override dei metodi generati automaticamente (`equals()`, `hashCode()` o `toString()`) per fornirne un'implementazione specifica.
- **Costruttori espliciti e compatti:** Sebbene il compilatore generi automaticamente un costruttore *canonico* (che accetta tutte le componenti del record), il programmatore può sostituirlo dichiarandolo esplicitamente. Per facilitare l'aggiunta di logica di validazione o normalizzazione dei dati, Java introduce il **costruttore compatto** (*compact constructor*): un costruttore privo della lista dei parametri, che permette di omettere l'assegnazione finale ai campi, gestita implicitamente dal compilatore alla fine del blocco.
- **Sovraccarico dei costruttori (Overloading):** È possibile definire costruttori aggiuntivi con firme diverse, ma con un vincolo stringente: ogni costruttore secondario deve obbligatoriamente delegare l'inizializzazione al costruttore canonico, invocando `this(...)` come prima istruzione.



Attenzione alla mutabilità dei componenti

I componenti di un record sono implicitamente *final*, quindi va ricordato ciò che

 abbiamo detto a proposito di tali campi: se un record ha come componente un oggetto mutabile (es. un array), l'immutabilità del record è solo superficiale; il riferimento all'array non può cambiare, ma gli elementi interni all'array sì. Per garantire la sicurezza, è preferibile usare solo tipi primitivi o immutabili come componenti.

Un esempio base di record:

Listing 2.12: Dichiarazione ed istanza di un Record

```

1 // Dichiarazione del record (genera automaticamente campi,
2 // costruttore, getter, equals, hashCode e toString)
3 public record Persona(String nome, int eta) {}
4
5 // Utilizzo
6 public class Main {
7     public static void main(String[] args) {
8         // Creazione di un'istanza
9         Persona p = new Persona("Mario Rossi", 30);
10
11         // Accesso ai dati (notare l'assenza di "get")
12         System.out.println(p.nome());
13     }
14 }
```

In questo esempio personalizziamo il comportamento interno del record:

Listing 2.13: Record con logica personalizzata

```

1 public record Prodotto(String codice, double prezzo) {
2
3     // 1. Ridefinizione del costruttore (Compact Constructor)
4     // Usato per validazione o normalizzazione
5     public Prodotto {
6         if (prezzo < 0) {
7             throw new IllegalArgumentException("Il prezzo non puo
8                 ' essere negativo");
9         }
10        codice = codice.toUpperCase(); // Normalizzazione
11    }
12
13    // 2. Metodo proprio (aggiuntivo)
14    public double prezzoConIva() {
15        return prezzo * 1.22;
16    }
17
18    // 3. Override di un Getter (accessor)
19    @Override
20    public String codice() {
21        return "ID-" + codice;
22    }
23 }
```

```

23 // 4. Override di toString
24 @Override
25 public String toString() {
26     return "Prodotto: " + codice + " costa " + prezzo + "
27         Euro";
28 }

```

Come ulteriore esempio, mostriamo un record `Utente` che implementa sia un costruttore compatto per la validazione dei dati, sia un costruttore sovraccaricato per semplificare l'istanza dell'oggetto fornendo dei valori di default:

Listing 2.14: Costruttori espliciti e overloading in un Record

```

1 public record Utente(String username, String email, boolean
2     attivo) {
3
4     // 1. Costruttore compatto
5     // Sostituisce l'implementazione generata per il costruttore
6     // canonico.
7     // I parametri sono implicitamente disponibili e verranno
8     // assegnati
9     // ai campi finali in automatico al termine di questo blocco.
10    public Utente {
11        if (username == null || username.isBlank()) {
12            throw new IllegalArgumentException("L'username non
13                puo' essere vuoto");
14        }
15        // Normalizzazione del dato prima dell'assegnazione
16        // implicita
17        email = email.toLowerCase();
18    }
19
20    // 2. Sovraccarico del costruttore (Overloading)
21    // Permette di creare un utente omettendo lo stato 'attivo'.
22    public Utente(String username, String email) {
23        // Delega obbligatoria al costruttore canonico tramite
24        // this(...)
25        this(username, email, true);
26    }
27 }

```



Quando usare un Record?

Usa un **record** invece di una classe tradizionale quando l'oggetto è completamente definito dal suo stato (i suoi attributi) e tali attributi sono **costanti** e si è sicuri di non doverne aggiungere altri in futuro.

È la scelta ideale per DTO (Data Transfer Objects), chiavi di mappe o semplici contenitori di dati, a patto di poter accettare le restrizioni dei record (niente ereditarietà, campi final).

2.14 Gestione della Memoria: Stack vs Heap

Un obiettivo fondamentale per comprendere il comportamento di Java, specialmente durante l'analisi delle classi e degli oggetti, è la distinzione tra l'allocazione nello **Stack** e nello **Heap**.

Lo Stack è una sorta di memoria di un metodo in esecuzione, una sezione di memoria rapida e organizzata in modo lineare (LIFO).

- **Contenuto:** Memorizza le variabili locali e i **tipi primitivi** (`int`, `double`, `boolean`, ecc.).
- **Riferimenti:** Quando viene creato un oggetto, lo Stack non contiene l'oggetto stesso, ma solo l'indirizzo di memoria (riferimento) necessario per rintracciarlo nello Heap.
- **Ciclo di vita:** La memoria viene gestita automaticamente; quando un metodo termina l'esecuzione, il suo spazio nello Stack viene rimosso immediatamente.

Lo Heap è un'area di memoria molto più vasta e dinamica.

- **Contenuto:** Qui risiedono tutti i veri **oggetti** (istanze di classi) e gli **array**. Anche le stringhe letterali vengono conservate qui, all'interno del cosiddetto *String Pool*.
- **Accesso:** Gli oggetti nello Heap sono accessibili solo tramite i riferimenti memorizzati nello Stack.
- **Ciclo di vita:** Gli oggetti rimangono nello Heap finché sono raggiungibili da almeno un riferimento. Quando non sono più referenziati, il **Garbage Collector** si occupa di deallocarli per liberare spazio.

Osservazione 2.13. Comprendere questa distinzione chiarisce il comportamento dei parametri nei metodi:

1. **Passaggio per copia del valore (primitivi):** Viene copiata la variabile sullo Stack; le modifiche nel metodo non influenzano l'originale.
2. **Passaggio per copia del valore del riferimento (oggetti):** Viene copiato l'indirizzo memorizzato nello Stack. Poiché entrambi i riferimenti puntano allo **stesso oggetto nello Heap**, le modifiche apportate all'oggetto sono visibili globalmente.

2.14.1 Confronto Sintetico

Caratteristica	Stack	Heap
Tipo di accesso	Sequenziale (LIFO)	Casuale
Velocità	Molto Elevata	Più Lenta
Cosa ospita	Primitivi e Riferimenti	Oggetti, Array, String Pool
Gestione	Automatica (Compiler)	Garbage Collector
Visibilità	Privata al Thread	Condivisa (rischio thread-safety)

Tabella 2.1: Differenze chiave tra Stack e Heap in Java.



Connessione con la Concorrenza

In sistemi complessi o distribuiti, ogni thread possiede il proprio Stack privato, mentre lo Heap è condiviso tra tutti. Questa condivisione è la causa principa-



le dei problemi di visibilità e sincronizzazione che richiedono l'uso di blocchi (`synchronized`) o variabili atomiche.

Aspetti di Gestione del Codice

3.1 Anatomia di un Progetto Java: Convenzioni e Struttura

Nello sviluppo professionale, un progetto Java non è mai una semplice cartella piena di file sparsi. Esistono delle convenzioni universali (utilizzate da IDE come IntelliJ, Eclipse o sistemi come Maven e Gradle) che servono a separare ciò che il programmatore scrive da ciò che il computer esegue.

La **Root** (Radice) è la cartella principale che contiene l'intero progetto. Al suo interno, il lavoro viene diviso in due rami principali per garantire pulizia e sicurezza:

- **Cartella `src` (Source)**: Contiene esclusivamente i file sorgente (`.java`). È l'unica cartella che viene salvata nei sistemi di controllo versione (come Git).
- **Cartella `bin` oppure `out` (Binary/Output)**: Qui il compilatore deposita i file `.class` (il bytecode). Questa cartella è generata automaticamente e può essere eliminata in qualsiasi momento senza perdere il lavoro, poiché viene rigenerata alla successiva compilazione.

Osservazione 3.1 (Perché separare `src` da `bin`?). Separare i file è fondamentale per la distribuzione: se vuoi regalare il tuo programma a un amico, gli darai solo il contenuto di `bin`. Se gli dessi anche `src`, lui potrebbe vedere e modificare il tuo codice sorgente. Inoltre, permette a strumenti come Git di non tracciare file inutili che appesantirebbero solo il repository.

3.2 I Package e il Naming Convention

Nello sviluppo software moderno, la probabilità che due programmatori diversi scelgano lo stesso nome per una classe (ad esempio `Employee`, `Account` o `User`) è estremamente alta. Per evitare conflitti, chiamati **collisioni di nomi** (*namespace collisions*), Java introduce il concetto di package.

Un **package** è una collezione di classi correlate che formano un'unità logica separata dal resto del codice. Funge da "cognome" per la classe, garantendo che il suo nome sia unico all'interno dell'intero ecosistema Java.

Per garantire l'unicità dei package a livello globale, la comunità Java ha adottato una convenzione ingegnosa: utilizzare il proprio **nome di dominio Internet al contrario**. Poiché i domini sono assegnati in modo univoco dalle autorità internazionali, nessuno potrà mai creare un package che inizi con il tuo dominio se tu ne sei il legittimo proprietario.

Esempio 3.1 (Scomposizione di un nome). Se l'azienda `horstmann.com` sviluppa un progetto chiamato `corejava`, il package sarà `com.horstmann.corejava`. Una classe `Employee` al suo interno avrà come "nome legale completo" (**Fully Qualified Name**):

```
com.horstmann.corejava.Employee
```



Package vs Directory: Distinzione Concettuale

Un errore tipico dei principianti è identificare il concetto di **package** con quello di **directory** (cartella). Sebbene Java, come vedremo a breve, imponga una corrispondenza biunivoca tra il nome del package e il percorso fisico sul disco, i



due concetti rimangono distinti:

- Il **Package** è un'entità logica (*Namespace*): serve a raggruppare classi correlate e a prevenire conflitti di nomi.
- La **Directory** è un'entità fisica: è una struttura del *file system* utilizzata dal sistema operativo.

In sintesi: un package **non** è semplicemente una collezione di file contenuti nella stessa cartella. Prova ne è il fatto che file situati nella stessa cartella, ma privi della direttiva **package**, appartengono, come vedremo, all'“unnamed package” e non a un package che prende il nome dalla cartella stessa.

Un altro errore comune per chi inizia è pensare che i package siano "annidati" come le cartelle di un sistema operativo e che quindi condividano permessi o caratteristiche. In Java, questa gerarchia è puramente visiva per l'essere umano, ma non esiste per il compilatore: per quest'ultimo, i package sono tutti "allo stesso livello".

Non esiste dunque alcuna relazione tecnica tra package che sembrano annidati. Ad esempio, `java.util` e `java.util.jar` sono trattati come due entità **completamente distinte**.

- Una classe in `java.util` non ha alcun accesso privilegiato ai membri di una classe in `java.util.jar`.
- Importare `java.util.*` **non** importa le classi presenti in `java.util.jar`.



Best Practice per i Nomi

I nomi dei package dovrebbero essere sempre scritti in **minuscolo**. Questo aiuta a distinguerli immediatamente dalle classi, che per convenzione iniziano sempre con la lettera maiuscola.

Osservazione 3.2 (Titolarità dei Package). In ambito professionale, lo sviluppatore non utilizza un proprio dominio personale, bensì quello dell'organizzazione per cui lavora.

- **Aziende:** Si utilizza il dominio aziendale (es. `com.google.*`).
- **Open Source:** Si utilizza spesso il prefisso del repository:
`com.github.username.*`.

È importante notare che il compilatore Java **non verifica** l'esistenza reale del dominio; la convenzione serve esclusivamente a garantire l'unicità logica dei nomi su scala globale.

3.2.1 Collaborazione e Conflitti

Quando più programmatori lavorano sullo stesso dominio aziendale, l'organizzazione segue regole di compartimentazione rigorose per evitare sovrapposizioni:

- **Sotto-pacchetti Specifici:** Il progetto viene suddiviso in moduli funzionali (es. `ui`, `backend`, `security`), assegnando a ogni team o sviluppatore una porzione specifica dell'albero dei package.
- **Naming Convention del Ruolo:** Si prediligono nomi di classe che descrivano la funzione specifica (es. `UserValidator` invece di un generico `UserCheck`), riducendo il rischio di omonimie.
- **Sistemi di Controllo Versione:** Strumenti come **Git** gestiscono la presenza di più sviluppatori sugli stessi file, permettendo la fusione delle modifiche (*merging*) e la risoluzione dei conflitti in modo controllato.



L'Importanza del Prefisso

Il prefisso del dominio (es. `it.ferrari`) serve a proteggere il progetto dal **mondo esterno**. L'organizzazione interna dei sub-package serve invece a proteggere i **collegi tra loro**.

3.3 Creazione e Gestione Manuale dei Package

Per comprendere davvero come Java gestisce il codice, è utile simulare il lavoro che l'IDE svolge "dietro le quinte". La creazione di un package senza l'aiuto di IntelliJ richiede una sincronizzazione precisa tra il codice sorgente e la struttura delle cartelle del sistema operativo.

Fase 1: Dichiarazione e Struttura Fisica

Per creare un package, ad esempio `eu.dedalo.math`:

- si crea un percorso fisico nella cartella `src` che rifletta il nome del package, dunque `src/eu/dedalo/math/`,
- in tutti i file sorgenti `.java` che stanno alla fine del percorso si dichiara che appartengono a quel package: `package eu.dedalo.math;`. Questa direttiva si dice **dichiarazione di appartenenza al package (package statement)** e deve essere la prima istruzione del file, prima di qualsiasi `import` o dichiarazione di classe.

In Java, la regola è che ogni file sorgente che dichiara di appartenere a un package deve essere posizionato in una cartella che rispecchia esattamente la struttura del package. Se questa corrispondenza non viene rispettata, il compilatore genererà errori. Se invece la dichiarazione di package è assente, il file viene considerato parte dell'“unnamed package”, che è una sorta di spazio dei nomi predefinito per i file senza package.

Fase 2: Compilazione Professionale

Supponiamo ora di aver creato il file sorgente `Calcolatrice.java` nel package che abbiamo appena costruito. È tecnicamente possibile compilare il file puntando direttamente al suo percorso nel file system senza specificare una directory di output:

```
1 # Compilazione diretta dal terminale
2 javac src/eu/dedalo/math/Calcolatrice.java
```

In questo caso, però, il compilatore non crea una struttura separata, ma deposita il file `.class` generato nella stessa cartella del file `.java`.

Per questo motivo la best practice prevede l'uso del flag `-d` (*destination*).

Listing 3.1: Compilazione da terminale

```
1 # Posizionarsi nella cartella 'src'
2 javac -d ../bin eu/dedalo/math/Calcolatrice.java
```

Il comando `-d ../bin` istruisce il compilatore a:

- Leggere la riga `package eu.dedalo.math;` nel sorgente.
- Creare automaticamente la gerarchia di cartelle `eu/dedalo/math/` dentro la cartella `bin`.
- Depositare il file `Calcolatrice.class` nella cartella finale.

Fase 3: Esecuzione e il "Nome Legale"

L'errore più comune in questa fase è cercare di eseguire il programma entrando nella cartella dove risiede il file `.class`. In Java, questo non funziona.

Per lanciare il programma da terminale, la JVM deve essere posizionata sulla **base della gerarchia dei package** (comunemente chiamata *Classpath Root*).

- Se ti trovi fisicamente dentro la cartella `bin`, la base è la cartella corrente.
- Se ti trovi nella *Project Root*, devi specificare dove sono i binari:

```
java -cp bin eu.dedalo.math.Calcolatrice
```

In ogni caso, **non** devi mai entrare dentro le sottocartelle del package (come `math/`) per lanciare il comando, altrimenti la JVM "perderebbe il filo" della struttura logica: la JVM cercherebbe una classe che dichiara di non avere package (unnamed package); trovando invece scritto `package eu.dedalo.math;`, la JVM rifiuterebbe di caricarla per incoerenza tra posizione e dichiarazione.

⚠ Compilazione ed Esecuzione: File vs Classi

È fondamentale distinguere come il sistema si interfaccia con il codice a seconda della fase in cui ci troviamo. Assumendo di trovarsi nella **Project Root**:

- **Il Compilatore (javac)** lavora sui **File**:
 - Ragiona per **percorsi fisici** (usa lo *slash* `/`).
 - Richiede l'estensione `.java`.
 - *Best Practice*: `javac -d bin src/eu/dedalo/math/Calcolatrice.java`
- **La JVM (java)** lavora sulle **Classi**:
 - Ragiona per **nomi logici** (usa il *dot* `.`).
 - **Mai** usare l'estensione (niente `.class`).
 - Richiede la **Classpath Root**: deve sapere da dove inizia il package (es. la cartella `bin`).
 - *Best Practice*: `java -cp bin eu.dedalo.math.Calcolatrice`

💡 Regola Mnemonica

Usa lo **slash** (`/`) quando stai parlando con il Sistema Operativo (file sul disco).
Usa il **punto** (`.`) quando stai parlando con la Java Virtual Machine (logica dei package).

3.4 L'Accesso a livello di Package

Oltre ai modificatori `public` e `private`, Java prevede un terzo livello di visibilità che si attiva quando **non viene specificato alcun modificatore**. Questo livello è noto come *package-private* o accesso di default.

Una classe, un metodo o una variabile definiti senza modificatori di accesso sono visibili e utilizzabili da **tutte le altre classi appartenenti allo stesso package**, ma sono totalmente inaccessibili all'esterno.

Sebbene l'accesso di package sia accettabile per le classi di utilità interna, Horstmann sottolinea come sia una scelta pericolosa per i campi di istanza (variabili).

⚠ Regola d'oro dell'Incapsulamento

Le variabili di istanza dovrebbero essere **sempre** dichiarate come **private**. Dimenticare il modificatore non le rende private, ma le espone a tutte le classi del package, violando il principio di protezione dei dati.

3.4.1 Sicurezza e Moduli

In passato, era possibile "intrufolarsi" in un package altrui semplicemente dichiarando lo stesso nome di pacchetto nel proprio file sorgente.

- Il JDK oggi protegge i package di sistema (quelli che iniziano con `java.`).
- Per i progetti complessi, la soluzione moderna per isolare davvero i package è l'uso dei **Moduli** (introdotti in Java 9), che permettono di dichiarare esplicitamente quali package sono "esportati" e quali restano privati. Vedremo i moduli più avanti, ma è importante sapere fin da ora che rappresentano un'evoluzione significativa rispetto alla semplice organizzazione in package, offrendo un controllo molto più rigoroso sull'accesso e la visibilità del codice.

3.5 Gli Import

Una classe può utilizzare liberamente tutte le classi presenti nel proprio package e tutte le classi **public** provenienti da altri package. Per accedere a queste ultime, Java offre due strade: l'uso del nome completo o la scorciatoia dell'istruzione **import**.

Il modo più diretto (ma faticoso) per usare una classe di un altro package è scriverne il **Fully Qualified Name** ogni volta che la si invoca:

```
java.time.LocalDate today = java.time.LocalDate.now();
```

Poiché questo approccio è prolisso, si utilizza quasi sempre l'istruzione **import** in cima al file (sotto la dichiarazione del package). Questo agisce come un "alias" che permette di usare solo il nome semplice della classe.

Osservazione 3.3 (Tipi di Import). Esistono due modi per importare:

- **Specifico**: `import java.time.LocalDate;` (Importa solo quella classe).
- **Wildcard**: `import java.time.*;` (Importa potenzialmente tutte le classi del package).

Nota bene: L'uso del carattere `*` non ha alcun impatto negativo sulla dimensione del file compilato (*bytecode*), ma rende meno esplicito quali classi siano effettivamente utilizzate nel file.

⚠ La regola del pacchetto singolo

La wildcard `*` può "aprire" solo un singolo package. Non è possibile usare `import java.*` per importare tutti i sotto-pacchetti che iniziano con `java`. Ogni livello va importato singolarmente.

💡 L'eccezione `java.lang`

Il package `java.lang` (che contiene classi base come `System`, `String` o `Math`) è **sempre** importato automaticamente in ogni file Java. Non c'è mai bisogno di scriverlo esplicitamente.

3.5.1 Risoluzione dei Conflitti di Omonimia

Il problema principale dei package sorge quando due classi hanno lo stesso nome ma risiedono in pacchetti diversi. L'esempio classico è la classe `Date`, presente sia in `java.util` che in `java.sql`.

Esempio 3.2 (Gestire i conflitti). Se importiamo entrambi i package con la wildcard:

- `import java.util.*;`
- `import java.sql.*;`

Il compilatore andrà in errore se scriviamo semplicemente `Date`, perché non saprà quale scegliere. Per risolvere:

1. Si aggiunge un import specifico per "rompere il pareggio":
`import java.util.Date;`
2. Se servono **entrambe** le classi nello stesso file, si è obbligati a usare il nome completo (*Fully Qualified Name*) per almeno una delle due:
`var d1 = new java.util.Date();`
`var d2 = new java.sql.Date(...);`

Osservazione 3.4 (Dietro le quinte del compilatore). Si ricordi che gli `import` sono solo una comodità per il programmatore. Una volta compilato il codice, nel **bytecode** finale non esistono più nomi brevi: il compilatore sostituisce ogni riferimento con il **Fully Qualified Name** completo.

3.5.2 Gli Import Statici

Oltre alle classi, Java permette di importare direttamente **metodi e campi statici**. Una volta importati, questi membri possono essere utilizzati nel codice come se fossero definiti localmente, senza dover far precedere il nome della classe di appartenenza.

Esempio 3.3 (Sintassi). Per importare tutti i membri statici della classe `System`:

```
1 import static java.lang.System.*;
2
3 out.println("Ciao!"); // Invece di System.out.println
4 exit(0);             // Invece di System.exit(0)
```

Horstmann solleva un punto critico: non sempre abbreviare è un bene.

- **Sconsigliato:** Abbreviare `System.out` in `out` spesso rende il codice meno chiaro per chi legge, poiché `out` è un nome molto generico.
- **Consigliato:** L'uso con la classe `Math` è molto apprezzato. Scrivere:
`sqrt(pow(x, 2))`
è molto più vicino alla notazione matematica rispetto a:
`Math.sqrt(Math.pow(x, 2)).`



Import di Enumerazioni

Gli import statici sono estremamente comodi con le **Enum**. Se devi gestire molti giorni della settimana, importare `static java.time.DayOfWeek.*` ti permette di usare semplicemente `FRIDAY` invece di `DayOfWeek.FRIDAY`.

Osservazione 3.5 (Il bytecode non mente). Proprio come per gli import di classe, gli import statici non hanno alcun impatto sulle prestazioni o sulla dimensione del file `.class`. Il compilatore si occupa semplicemente di "risolvere" i nomi abbreviati sostituendoli con i nomi completi durante la compilazione.

Ereditarietà e Polimorfismo

4.1 Classi, Superclassi e Sottoclassi

L'ereditarietà è il meccanismo che permette di creare nuove classi basandosi su classi esistenti. L'idea fondamentale è quella di riutilizzare i metodi e i campi di una classe esistente, aggiungendo o modificando funzionalità per rispondere a esigenze più specifiche.

Il criterio fondamentale per stabilire se l'ereditarietà sia appropriata è la relazione "is-a" (è-un). Ogni istanza della sottoclasse deve poter essere considerata un'istanza della superclasse. Ad esempio, poiché ogni *Manager* è un *Dipendente*, la classe **Manager** può ereditare da **Employee**. Richiamiamo innanzitutto la classe **Employee** che abbiamo visto nei capitoli precedenti:

Listing 4.1: La superclasse Employee originale

```
1 import java.time.LocalDate;
2
3 public class Employee {
4     private String name;
5     private double salary;
6     private LocalDate hireDay;
7
8     public Employee(String name, double salary, int year, int
9         month, int day) {
10         this.name = name;
11         this.salary = salary;
12         this.hireDay = LocalDate.of(year, month, day);
13     }
14
15     public String getName() {
16         return name;
17     }
18
19     public double getSalary() {
20         return salary;
21     }
22
23     public LocalDate getHireDay() {
24         return hireDay;
25     }
26
27     public void raiseSalary(double byPercent) {
28         double raise = salary * byPercent / 100;
29         salary += raise;
30     }
31 }
```

In Java, l'ereditarietà si esprime tramite la parola chiave **extends**.

Listing 4.2: Esempio di ereditarietà

```

1 public class Manager extends Employee {
2     private double bonus; // Campo aggiuntivo
3
4     public void setBonus(double bonus) {
5         this.bonus = bonus;
6     }
7 }

```

Nonostante il prefisso "super", una superclasse non è "superiore" in termini di capacità. Al contrario, la sottoclasse è solitamente più complessa:

- La **Superclasse** contiene i metodi più generali.
- La **Sottoclasse** eredita tutto ciò che c'è nella superclasse e aggiunge campi o metodi specializzati (*campi e metodi propri*).

Precisazione sull'esempio del libro

Nel libro di testo, l'esempio viene proposto nell'ipotesi (puramente esemplificativa) che un `Employee` non possa mai diventare `Manager`. In una situazione reale questo non è in genere realistico e la modellazione delle classi dovrebbe tenere conto di questa possibilità.

Accesso di classe vs Accesso di istanza

In Java, il modificatore `private` garantisce la visibilità a livello di **classe** e non di singola istanza:

- Un'istanza di `Employee` può accedere ai membri privati di un'altra istanza della stessa classe.
- Una sottoclasse `Manager`, pur ereditando lo stato, è considerata **codice esterno** rispetto alla definizione della superclasse.

Conseguenza: La sottoclasse possiede i dati della superclasse (memoria), ma deve interagire con essi tramite l'interfaccia pubblica (metodi), preservando l'integrità dell'incapsulamento.

4.2 Sovrascrivere i Metodi (Overriding)

Spesso i metodi della superclasse non sono del tutto appropriati per la sottoclasse. Nel nostro esempio, il metodo `getSalary()` di `Employee` restituisce solo lo stipendio base, mentre per un `Manager` dovrebbe restituire la somma di stipendio e bonus. Tentare di implementare il metodo accedendo direttamente al campo della superclasse fallisce:

```

1 public double getSalary() {
2     return salary + bonus; // ERRORE: salary e' privato in
3     Employee
4 }

```

Anche se `Manager` è una sottoclasse, deve rispettare l'incapsulamento e usare l'interfaccia pubblica della superclasse. Un altro errore comune è chiamare il metodo per nome sperando che Java "capisca" di voler usare quello del padre:

```

1 public double getSalary() {
2     double baseSalary = getSalary(); // ERRORE: chiama se stesso
3     all'infinito
4 }

```

```

3     return baseSalary + bonus;
4 }

```

Questo codice causa un *crash* del programma perché il metodo richiama se stesso senza sosta (Stack Overflow).

Per istruire il compilatore a invocare il metodo della superclasse anziché quello corrente, si utilizza la parola chiave **super**.

Listing 4.3: Overriding corretto con super

```

1 public double getSalary() {
2     double baseSalary = super.getSalary();
3     return baseSalary + bonus;
4 }

```

! super non è un riferimento

A differenza di **this**, che è un riferimento all'oggetto corrente (e può essere assegnato a una variabile), **super** è solo una **parola chiave speciale** che dirige il compilatore verso la superclasse. Non puoi, ad esempio, scrivere `Object x = super;`.

! La Regola della Visibilità nell'Overriding

Questa è una regola ferrea: **una sottoclasse non può rendere "meno visibile" un metodo ereditato.** Se un metodo nella superclasse è **public**, nella sottoclasse **deve** rimanere **public**. Non puoi trasformare un metodo pubblico del padre in un metodo privato o protetto nel figlio.

4.2.1 L'importanza di @Override

Le annotazioni in Java sono metadati che forniscono informazioni aggiuntive al compilatore; saranno trattate più approfonditamente più avanti, ma è importante introdurre subito l'annotazione **@Override** in questo contesto. Sebbene non sia strettamente necessaria per il funzionamento del codice, è buona norma precedere ogni metodo sovrascritto con l'annotazione **@Override**.

Listing 4.4: Uso di @Override

```

1 @Override
2 public double getSalary() {
3     return super.getSalary() + bonus;
4 }

```

[Motivazioni e Vantaggi] L'uso di questa annotazione serve a istruire il compilatore sulle nostre intenzioni. Le motivazioni principali sono:

1. **Controllo degli errori di battitura:** Se scrivessimo per errore `getsalary()` (con la 's' minuscola), senza annotazione Java penserebbe semplicemente che stiamo creando un *nuovo* metodo. Con **@Override**, il compilatore segnalerebbe un errore perché non esiste alcun metodo `getsalary` nella superclasse da sovrascrivere.

2. **Verifica della firma:** Se cambiamo i parametri del metodo (es. aggiungiamo un argomento), non stiamo più facendo *overriding* ma *overloading*. L'annotazione ci avvisa immediatamente se stiamo rompendo il legame con il metodo del padre.
3. **Documentazione:** Rende immediatamente chiaro a chi legge il codice che quel metodo non è un'aggiunta originale della sottoclasse, ma una modifica di un comportamento ereditato.

Fail-Fast

Usare `@Override` è un esempio del principio "*fail-fast*": è meglio che il compilatore ci urli contro subito per un errore di distrazione, piuttosto che scoprire a runtime che il nostro `Manager` sta prendendo lo stipendio base perché il metodo di calcolo del bonus ha un nome leggermente sbagliato.

4.3 I Costruttori delle Sottoclassi

A differenza dei metodi, i costruttori non vengono "ereditati" nel senso stretto del termine: ogni sottoclasse deve definire i propri costruttori, ma ha l'obbligo di invocare un costruttore della superclasse per inizializzare i campi privati ereditati. Il meccanismo di *constructor chaining* (catena dei costruttori) impone infatti dei vincoli precisi a seconda della struttura della superclasse. Si possono verificare due scenari principali:

1. **Superclasse con costruttore no-args (esplicito o implicito)** Se nella superclasse esiste un costruttore senza argomenti, la sottoclasse non è obbligata a dichiarare un costruttore proprio. In questo caso, viene utilizzato il costruttore di default della sottoclasse che invoca implicitamente `super()`.
2. **Superclasse senza costruttore no-args** Se nella superclasse è presente almeno un costruttore con parametri e **non** è stato definito un costruttore senza argomenti, la sottoclasse ha un obbligo formale: bisogna dichiarare esplicitamente un costruttore nella sottoclasse e la prima istruzione di tale costruttore deve essere una chiamata esplicita a uno dei costruttori della superclasse tramite la sintassi `super(...parametri...)`.

Errore di compilazione comune

Se la superclasse richiede parametri e la sottoclasse non li fornisce esplicitamente (o tenta di farlo dopo la prima riga), il compilatore segnalerà l'errore: *"Implicit super constructor is undefined. Must explicitly invoke another constructor"*.

Invocazione Implicita e Campi Propri

Se nella superclasse non è definito alcun costruttore esplicito (ovvero è presente il costruttore *no-args* di default), la sottoclasse gode di maggiore flessibilità: è possibile definire un costruttore che inizializza esclusivamente i **campi propri** della sottoclasse. In questo scenario, la chiamata a `super()` può essere omessa (e di solito è considerata una buona pratica, per ridurre la verbosità), poiché il compilatore la inserirà automaticamente come prima riga invisibile.

⚠ La regola della prima posizione

Entrambe le istruzioni, `this()` e `super()`, hanno il vincolo categorico di dover essere la **prima istruzione** del costruttore. Di conseguenza, **non possono mai comparire nello stesso costruttore**.

Osservazione 4.1. Cosa succede se un costruttore nella sottoclasse usa `this(...)`? La responsabilità di invocare il padre viene semplicemente "passata" lungo la catena:

1. Il Costruttore A chiama `this(...)` (Costruttore B).
2. Il Costruttore B deve, a sua volta, chiamare `super(...)` (o un altro costruttore della stessa classe che alla fine lo faccia).

Indipendentemente da quanti `this()` vengano usati, l'ultimo anello della catena di costruttori di una classe **deve** sempre invocare (esplicitamente o implicitamente) un costruttore della superclasse. La "nascita" del padre non può essere evitata, può solo essere delegata a un altro costruttore "fratello".

4.4 Polimorfismo

Il polimorfismo è la capacità di un oggetto di essere trattato come un'istanza della sua superclasse. Questo concetto si basa sul **Principio di Sostituzione di Liskov**.

Il principio afferma che è sempre possibile utilizzare un oggetto di una sottoclasse in qualsiasi contesto in cui sia attesa una superclasse. Questo perché, per la relazione *is-a*, una sottoclasse possiede tutte le caratteristiche della superclasse.

Listing 4.5: Esempio di sostituzione

```

1 // Sintassi generale:
2 // SuperClasse oggetto = new SottoClasse(...);
3
4
5 Employee e = new Manager("Rossi", 50000, 2023, 10, 1);

```

Nella riga di codice polimorfica di sopra, dobbiamo distinguere due aspetti:

- **Tipo Dichiarato:** È il tipo della variabile (`Employee`). Determina quali metodi il compilatore ci permetterà di chiamare: *non è possibile invocare metodi specifici di Manager su una variabile dichiarata come Employee*; se provassimo a chiamare `e.setBonus(10000)`, otterremmo un errore di compilazione.
- **Tipo Effettivo:** È il tipo dell'oggetto creato in memoria (`Manager`). Determina quale versione del metodo verrà eseguita a runtime.

⚠ L'unidirezionalità della sostituzione

La sostituzione funziona solo verso l'alto nella gerarchia:

- **OK:** Assegnare un `Manager` a una variabile `Employee`.
- **ERRORE:** Assegnare un `Employee` a una variabile `Manager`.

Osservazione 4.2 (Array Polimorfici). Il principio di sostituzione si applica anche agli array. È possibile creare un array di una superclasse e riempirlo con oggetti di sottoclassi diverse:

```

1 Employee[] staff = new Employee[3];
2 staff[0] = new Manager("Boss", 80000, 2020, 1, 1);

```



```
3 | staff[1] = new Employee("Worker", 40000, 2021, 5, 1);
```

Questo permette di gestire gruppi eterogenei di oggetti con un unico ciclo di elaborazione.

4.4.1 Il Legame Dinamico (Dynamic Binding)

Il cuore pulsante del polimorfismo è il **dynamic binding**. È il meccanismo per cui la JVM decide, solo al momento dell'esecuzione, quale versione di un metodo invocare.

Consideriamo un array di tipo `Employee` che contiene sia dipendenti "standard" che manager:

Listing 4.6: Esempio di Dynamic Binding

```
1 Employee[] staff = new Employee[3];
2
3 // Sostituzione: Manager "travestito" da Employee
4 staff[0] = new Manager("Carl Cracker", 80000, 1987, 12, 15);
5 staff[1] = new Employee("Harry Hacker", 50000, 1989, 10, 1);
6 staff[2] = new Employee("Tony Tester", 40000, 1990, 3, 15);
7
8 // Ciclo polimorfico
9 for (Employee e : staff) {
10     System.out.println(e.getName() + " " + e.getSalary());
11 }
```

Quando il programma arriva alla riga `e.getSalary()`, avviene la magia del dynamic binding:

1. Il **Compilatore** controlla il tipo dichiarato (`Employee`) e verifica che esista un metodo `getSalary()`. Se esiste, dà il via libera.
2. La **JVM (a runtime)** guarda l'oggetto reale a cui punta `e`:
 - Se `e` punta a un `Employee`, viene eseguito `Employee.getSalary()` (quello base).
 - Se `e` punta a un `Manager`, viene eseguito `Manager.getSalary()` (il metodo sovrascritto, che aggiunge il bonus).

Osservazione 4.3 (Flessibilità ed Estensibilità). La bellezza del dynamic binding è che se domani aggiungessimo una nuova sottoclasse `Executive` con il suo metodo `getSalary()` specifico, il ciclo `for` qui sopra **non cambierebbe di una virgola**. Il codice diventa "aperto" a nuove forme senza dover essere riscritto.

Osservazione 4.4. Il polimorfismo è la capacità di una variabile di riferirsi a tipi diversi. Il dynamic binding è la capacità della JVM di scegliere il metodo corretto per quel tipo specifico. Sono due facce della stessa medaglia.

Osservazione 4.5 (Hiding dei Metodi Statici). Cosa succede se definiamo in una sottoclasse un metodo `static` con la stessa firma di un metodo `static` della superclasse?

A differenza dei metodi d'istanza, i metodi statici **non partecipano al polimorfismo**. In questo caso non si parla di *overriding* ma di **hiding** (nascondimento).

- **Binding Statico:** Il metodo da eseguire viene deciso dal compilatore in base al *tipo dichiarato* della variabile, non al *tipo effettivo* dell'oggetto.
- **Mancanza di Polimorfismo:** Se una variabile `Employee` `e` punta a un oggetto `Manager`, l'invocazione di un metodo statico tramite `e` chiamerà sempre la versione di `Employee`.

Listing 4.7: Differenza tra Overriding e Hiding

```
1 Employee e = new Manager();
2
3 e.getSalary();           // Overriding: esegue il metodo di
                           Manager (Runtime)
4 e.staticMethod();       // Hiding: esegue il metodo di Employee (
                           Compile-time)
```



Best Practice

Proprio perché il comportamento dei metodi statici può trarre in inganno quando usati con variabili polimorfiche, in Java è considerata cattiva pratica invocare metodi statici tramite un'istanza (es. `e.staticMethod()`). Si dovrebbe sempre usare il nome della classe (es. `Employee.staticMethod()`).

4.4.2 Sintesi Mnemonica: Il Senso di Marcia

Per evitare confusione nel gestire le gerarchie, possiamo definire una regola basata sulla "direzione" in cui Java percorre l'albero delle classi:

I Costruttori partono dall'alto (Top-Down): Quando crei un oggetto, Java garantisce che le fondamenta siano solide prima di costruire i piani superiori. L'esecuzione risale fino a `Object` e poi scende: *Padre* → *Figlio*. Non puoi esistere come "Manager" se prima non sei stato inizializzato come "Employee".

I Metodi partono dal basso (Bottom-Up): Quando invochi un metodo su un oggetto polimorfico, Java cerca l'implementazione più specifica possibile. Parte dalla classe effettiva dell'oggetto e risale solo se non trova una sovrascrittura: *Figlio* → *Padre*. Si predilige sempre la "personalità" della sottoclasse rispetto alla generalità della superclasse.



Perché questo dualismo?

Questa differenza esiste per un motivo preciso:

- I **costruttori** servono alla sicurezza: garantiscono che ogni parte dell'oggetto sia configurata correttamente.
- I **metodi** servono alla flessibilità: permettono a una sottoclasse di cambiare comportamento senza che chi usa l'oggetto debba accorgersene.

4.4.3 Covarianza dei Tipi di Ritorno

Una regola particolare dell'overriding riguarda il tipo di ritorno del metodo. La firma del metodo sovrascritto deve in ogni caso rimanere identica, ed in teoria anche il tipo di ritorno dovrebbe essere lo stesso. Java però permette alla sottoclasse di restituire un **sottotipo** rispetto a quello dichiarato nella superclasse.

Osservazione 4.6 (Definizione di Covarianza). Si parla di *tipi di ritorno varianti* (o covarianza) quando una sottoclasse sovrascrive un metodo cambiando il tipo di ritorno con uno più specifico (una sua sottoclasse).

Esempio 4.1 (Esempio `getBuddy`). Consideriamo i seguenti metodi:

- Superclasse: `public Employee getBuddy() { ... }`
- Sottoclasse: `public Manager getBuddy() { ... }`

Questo codice è perfettamente legale perché `Manager` è un sottotipo di `Employee`. L'overriding è valido e il tipo di ritorno è considerato "compatibile".

⚠ Vincolo di direzione

La covarianza funziona solo in una direzione: si può restituire qualcosa di **più specifico**, mai qualcosa di più generico.

- Se il padre restituisce `Manager`, il figlio non può restituire `Employee`.

4.5 Prevenire l'Ereditarietà: Il modificatore `final`

Il modificatore `final` può essere applicato a classi e metodi per impedirne, rispettivamente, l'estensione o la sovrascrittura.

Listing 4.8: Sintassi di `final`

```

1 public final class Executive extends Manager { ... } // Classe
   non estensibile
2
3 public class Employee {
4     public final String getName() { return name; } // Metodo non
       sovrascrivibile
5 }

```

⚠ Distinzione tra Campi e Classi

Se una classe è `final`, i suoi **metodi** sono automaticamente `final`, ma i suoi **campi** NO. La mutabilità dei dati rimane regolata dai singoli modificatori sui campi.

Dunque l'effetto di dichiarare `final` classi o metodi è piuttosto semplice da capire; meno ovvio è capire quando e perché farlo. Esistono due motivazioni principali, una di **design** e una di **sicurezza**.

1. Integrità della Semantica (Design)

Si usa `final` quando il progettista vuole garantire che il comportamento di una classe non venga alterato o "corrotto" da sottoclassi.

- **Esempio di String:** Se `String` non fosse `final`, qualcuno potrebbe creare una sottoclasse `EvilString` che sembra una stringa normale ma invia i dati a un server remoto. Raggiungere l'immutabilità e la sicurezza richiede che nessuno possa cambiare come funziona una stringa.

2. Sicurezza nei Costruttori: Il problema della "Fuga di `this`"

Chiamare un metodo sovrascrivibile all'interno di un costruttore è estremamente pericoloso a causa dello scontro tra la costruzione *Top-Down* e il binding *Bottom-Up* (vedi paragrafo 4.4.2).

Listing 4.9: Esempio del fallimento nel costruttore

```

1 public class Employee {
2     public Employee(...) {
3         // ... inizializzazione campi base ...
4         System.out.println("Costruito: " + description());
5     }
6
7     public String description() {
8         return "Dipendente con stipendio " + salary;
9     }
10 }
11
12 public class Executive extends Employee {
13     private String title; // Inizialmente null
14
15     public Executive(String name, String title, ...) {
16         super(name, ...); // Chiama il costruttore di Employee
17         this.title = title;
18     }
19
20     @Override
21     public String description() {
22         // Se title e' null, length() lancia NullPointerException
23         return "Executive con titolo: " + title + " (lunghezza: "
24             + title.length() + ")";
25     }
26 }

```

Analisi del crash: Quando esegui `new Executive(...)`:

1. Il costruttore di `Executive` invoca quello di `Employee` (**Top-Down**).
2. Il costruttore di `Employee` chiama `description()`.
3. Per il **Dynamic Binding**, la JVM cerca il metodo partendo dal basso e trova `Executive.description()`.
4. Il metodo del figlio prova a leggere `title.length()`, ma il campo `title` non è ancora stato inizializzato (l'istruzione `this.title = title` verrà eseguita solo **dopo** che il costruttore del padre avrà finito).
5. **Risultato:** Il programma crasha con un `NullPointerException`.

Soluzione

Per evitare questo comportamento imprevedibile su oggetti "parzialmente costruiti", i costruttori dovrebbero invocare solo metodi dichiarati come **final** (che non possono essere sovrascritti) o **private** (che sono implicitamente final).

Osservazione 4.7 (Metodi statici e Binding Statico). Sebbene vengano usati meno spesso per questo scopo, oltre ai metodi **private** e **final**, anche i **metodi statici** sono sicuri da invocare all'interno di un costruttore.

Questo accade per effetto del *hiding* descritto nell'osservazione 4.5: il metodo da eseguire viene determinato dal compilatore in base al tipo della classe e non dall'oggetto reale a runtime. Di conseguenza, non vi è il rischio di invocare accidentalmente logica

della sottoclasse prima che questa sia stata inizializzata.

✓ Non abusare dei final!

In passato molti programmatori tendevano ad usare spesso `final` anche in contesti in cui non era necessario, per ragioni di ottimizzazione (che qui omettiamo). Oggi non è più necessario, perché la JVM moderna è già abbastanza intelligente da gestire l'ottimizzazione da sola. Dovresti usare `final` solo per ragioni di **chiarezza di design** e **sicurezza**.

4.6 Il Casting degli Oggetti

Il casting consiste nel forzare il compilatore a trattare un riferimento di una superclasse come se fosse di una sottoclasse. Questa operazione è necessaria per accedere ai membri (campi o metodi) che sono presenti solo nella sottoclasse.

Listing 4.10: Esempio di Downcasting

```
1 Employee e = new Manager(...); // Polimorfismo
2 // e.setBonus(5000); // ERROR: Employee non ha setBonus
3
4 Manager m = (Manager) e; // Downcasting
5 m.setBonus(5000); // OK
```

Mentre l'*upcasting* (da sottoclasse a superclasse) è sempre sicuro e automatico, il **downcasting** è una scommessa: se l'oggetto `e` fosse davvero un `Employee` "semplice" e non un `Manager`, l'istruzione `(Manager) e` causerebbe il crash immediato del programma con una **ClassCastException**.

Il casting non cambia l'oggetto in memoria. L'oggetto è sempre lo stesso. Quello che cambia sono gli "occhiali" con cui Java lo guarda:

- Con gli occhiali `Employee`, vedi solo i metodi e campi dichiarati in `Employee`.
- Con gli occhiali `Manager`, vedi anche i metodi e campi dichiarati in `Manager`.

Il casting dice a Java: "Togli gli occhiali da `Employee` e metti quelli da `Manager`. Mi assumo io la responsabilità che dietro quel vetro ci sia effettivamente un `Manager`". Se ti sbagli, Java "ti punisce" interrompendo il programma.

💡 Design Rule

Se ti ritrovi a voler chiamare un metodo specifico (es. `setBonus`) su una lista generica di oggetti (`Employee`), chiediti:

1. Quel metodo dovrebbe forse far parte della superclasse?
2. Sto usando la struttura dati corretta o dovrei avere una lista separata di soli `Manager`?

Il casting dovrebbe essere l'eccezione, non la regola: l'abuso del casting spesso è il segnale di un'architettura debole

4.7 Evoluzione di `instanceof`: Pattern Matching

A partire da Java 16, l'operatore `instanceof` è stato potenziato per ridurre la verbosità del *downcasting* e migliorare la leggibilità del codice.

Invece di controllare il tipo e poi eseguire un cast manuale, è possibile dichiarare una **pattern variable** direttamente nel test.

Listing 4.11: Confronto tra vecchio e nuovo approccio

```

1 // Prima di Java 16 (Verboso)
2 if (staff[i] instanceof Manager) {
3     Manager boss = (Manager) staff[i]; // Cast manuale ripetitivo
4     boss.setBonus(5000);
5 }
6
7 // Da Java 16 in poi (Pattern Matching)
8 if (staff[i] instanceof Manager boss) {
9     boss.setBonus(5000); // boss e' già di tipo Manager
10 }

```

4.7.1 Scoping e Operatori Logici

La variabile introdotta dal pattern (es. `boss`) ha uno **scope limitato** basato sul flusso logico (*flow-sensitive scoping*):

- **AND (&&)**: È permesso usare la variabile nella parte destra dell'espressione, poiché viene valutata solo se il test `instanceof` è vero.
- **OR (||)**: È proibito, poiché la parte destra viene eseguita se il test è falso, rendendo la variabile non definita.

```

1 if (e instanceof Executive exec && exec.getTitle().length() > 10)
    { ... } // OK

```

⚠ Attenzione: Shadowing dei Campi

La variabile introdotta da un pattern (es. `exec`, `boss`) è una variabile locale. Come tale, può **oscurare** (shadow) un campo della classe che ha lo stesso nome. Questo può portare a errori logici difficili da individuare.

Listing 4.12: Esempio di Shadowing con instanceof

```

1 class Value {
2     private double v; // Campo della classe
3
4     public boolean equals(Object other) {
5         // Se il test e vero, 'v' all'interno del blocco
6         // si riferisce a LabeledValue, NON al campo
7         // della classe.
8         if (other instanceof LabeledValue v) {
9             return this.v == v.getValue();
10        }
11        return false;
12    }
13 }

```

Best Practice: Per evitare confusione, è sempre consigliabile dare alla variabile del pattern un nome univoco o diverso dai campi della classe (es. `otherValue` invece di `v`).

4.7.2 Il Record Pattern e la Decostruzione (Java 21+)

L'evoluzione di Java ha cercato di rendere sempre più immediata la decostruzione dei records. Supponiamo, ad esempio, di avere il record: `record Point(double x, double y) {}`. Ecco come è cambiato il modo di estrarre le coordinate `x` e `y` da un oggetto `obj`.

1. Prima di Java 16

Lavoro manuale in tre passaggi: controllo, cast manuale ed estrazione tramite metodi accessor.

```

1 if (obj instanceof Point) {
2     Point p = (Point) obj; // Cast fastidioso e ripetitivo
3     double x = p.x();      // Estrazione manuale 1
4     double y = p.y();      // Estrazione manuale 2
5 }

```

2. Java 16 - Type Pattern: Il primo passo

Il cast scompare grazie alla *pattern variable*, ma l'estrazione dei componenti è ancora manuale.

```

1 if (obj instanceof Point p) {
2     double x = p.x(); // Devo ancora chiamare esplicitamente i
        metodi
3     double y = p.y();
4 }

```

3. Java 21 - Record Pattern: La Rivoluzione

L'oggetto viene **decostruito** sul posto. Non serve più una variabile d'appoggio per l'intero oggetto se ci servono solo i suoi componenti.

```

1 if (obj instanceof Point(double x, double y)) {
2     // x e y sono variabili locali già estratte e pronte all'uso
        !
3     System.out.println("Coordinate: " + x + ", " + y);
4 }

```

Il "Superpotere": Il Nesting (Annidamento)

Il vero vantaggio emerge con strutture dati complesse. Il Record Pattern permette di scendere in profondità in un'unica riga di codice.

Listing 4.13: Decostruzione nidificata

```

1 record Point(double x, double y) {}
2 record Circle(Point center, double radius) {}
3
4 // Recupero la coordinata X del centro di un cerchio obj in un
    colpo solo:
5 if (obj instanceof Circle(Point(double x, double y), double r)) {

```

```

6      System.out.println("La X del centro e': " + x);
7  }

```

Osservazione 4.8 (Utilità nel Backend). Questa sintassi è estremamente potente quando si maneggiano strutture dati nidificate (come quelle provenienti da JSON o database). Permette di "interrogare" la forma dell'oggetto e contemporaneamente estrarre chirurgicamente solo i dati necessari, riducendo drasticamente il *boilerplate code* e il rischio di errori.

Osservazione 4.9 (Variabili non usate). In Java 21, se un componente del record non è necessario, si può usare il carattere *underscore* (`_`) per ignorarlo (JEP 443): `if (p instanceof Point(var a, _)) { ... }`

4.8 Accesso Protetto: Il modificatore `protected`

Il modificatore `protected` offre una via di mezzo tra il rigore del `private` e la totale apertura del `public`. Permette l'accesso ai membri di una classe alle sue sottoclassi e a tutte le classi all'interno dello stesso package.

Ecco un confronto dei quattro livelli di accesso in Java:

Modificatore	Classe	Package	Sottoclasse	Mondo
<code>public</code>	Sì	Sì	Sì	Sì
<code>protected</code>	Sì	Sì	Sì*	No
<i>Default</i>	Sì	Sì	No	No
<code>private</code>	Sì	No	No	No

Tabella 4.1: Tabella dei modificatori di accesso.

Spieghiamo il significato dell'asterisco (*) in tabella. Esiste una regola sottile ma fondamentale: se una sottoclasse si trova in un **package diverso** dalla superclasse, essa può accedere ai membri **`protected` solo attraverso riferimenti del proprio tipo** (o dei propri sottotipi).

Listing 4.14: Restrizione di accesso `protected`

```

1  // Package A
2  public class Employee {
3      protected LocalDate hireDay;
4  }
5
6  // Package B
7  public class Administrator extends Employee {
8      public void check(Employee e, Administrator a) {
9          this.hireDay = ...; // OK: accesso al proprio campo
           ereditato
10         a.hireDay = ...;    // OK: a e' un Administrator
11         // e.hireDay = ...; // ERRORE: non puo spiare dentro un
           Employee generico
12     }
13 }

```


**Considerazioni di Design: Campi vs Metodi**

- **Campi protected:** Generalmente sconsigliati. Violano l'incapsulamento perché una volta dichiarati, non puoi più cambiare l'implementazione interna senza rompere il codice di tutte le sottoclassi scritte da altri.
- **Metodi protected:** Molto più utili. Indicano un metodo "delicato" che solo le sottoclassi (che conoscono bene il funzionamento interno del padre) sono autorizzate a invocare.

4.9 La Classe Object

In Java, la classe `java.lang.Object` è l'antenata comune di tutte le classi. Se una classe non specifica esplicitamente una superclasse con `extends`, essa eredita automaticamente da `Object`. Dunque variabile di tipo `Object` funge da "contenitore universale" per qualsiasi riferimento a oggetto.

Ogni classe in Java eredita quindi tutti i metodi di questa "superclasse cosmica" `Object`. Tuttavia, poiché `Object` è un'astrazione totale, le sue implementazioni di default sono necessariamente generiche e basate sull'identità fisica dell'oggetto piuttosto che sul suo stato logico.

Affinché tali metodi possano eseguire azioni utili per le specifiche classi, è quasi sempre cruciale effettuare l'override di questi metodi. Questa operazione è spesso delicata e bisogna sapere come eseguirla correttamente, altrimenti si rischia di introdurre bug difficili da diagnosticare.

Vedremo quindi come implementare correttamente i tre metodi principali: `equals()`, `hashCode()` e `toString()`.

4.9.1 Il Metodo `equals()`

Il metodo `equals()` della classe `Object` implementa di default il test di **identità**: due riferimenti sono uguali solo se puntano allo stesso identico indirizzo di memoria (`this == otherObject`).

Tuttavia, per molte classi è necessario un test basato sullo **stato**: due oggetti sono uguali se contengono gli stessi dati. Ad esempio, due stringhe con lo stesso testo dovrebbero essere considerate uguali anche se sono due oggetti distinti in memoria, oppure due array con gli stessi elementi dovrebbero essere considerati uguali.

Di seguito viene riportata l'implementazione del metodo `equals()` per `Employee`. Questo esempio rappresenta la struttura "tipo" che ogni sviluppatore dovrebbe seguire per garantire un confronto robusto.

Listing 4.15: Implementazione standard di `equals` nella classe `Employee`

```
1 public class Employee
2 {
3     /* ... campi della classe ... */
4     @Override
5     public boolean equals(Object otherObject)
6     {
7         // 1. Test rapido di identità:
8         // Se i riferimenti sono identici, gli oggetti sono
9         // certamente uguali.
10        if (this == otherObject) return true;
```

```

10
11 // 2. Controllo del valore null:
12 // Per contratto, equals(null) deve sempre restituire false
13
14 if (otherObject == null) return false;
15
16 // 3. Confronto delle classi:
17 // Se le classi non coincidono esattamente, gli oggetti non
18 // sono uguali.
19 // Si usa getClass() per garantire la simmetria nelle
20 // gerarchie:
21 // se a.equals(b) e' true, allora b.equals(a) deve essere
22 // true.
23 if (getClass() != otherObject.getClass())
24     return false;
25
26 // 4. Casting:
27 // Arrivati a questo punto, sappiamo che otherObject e' un
28 // Employee non nullo. Possiamo procedere al cast in
29 // sicurezza.
30 Employee other = (Employee) otherObject;
31
32 // 5. Confronto dei campi (Uguaglianza di stato):
33 // Si confrontano i singoli campi significativi per
34 // determinare
35 // se il contenuto degli oggetti e lo stesso.
36 return name.equals(other.name)
37     && salary == other.salary
38     && hireDay.equals(other.hireDay);
39 }
40 }

```

Osservazione 4.10. Da notare come il test di identità (`this == otherObject`) sia posto all'inizio come ottimizzazione: se stiamo confrontando un oggetto con se stesso, risparmiamo il tempo di eseguire tutti i controlli successivi e i confronti tra i campi.

Listing 4.16: Implementazione robusta di equals

```

1 public boolean equals(Object otherObject) {
2     if (this == otherObject) return true; // 1
3     if (otherObject == null) return false; // 2
4     if (getClass() != otherObject.getClass()) return false; // 3
5
6     Employee other = (Employee) otherObject; // 4
7
8     // 5. Utilizzo di Objects.equals per gestire i campi null
9     return Objects.equals(name, other.name)
10         && salary == other.salary
11         && Objects.equals(hireDay, other.hireDay);
12 }

```

Ereditarietà e equals()

Quando si estende una classe che ha già un proprio `equals()`, la sottoclasse deve innanzitutto invocare il metodo della superclasse.

```

1 public class Manager extends Employee {
2     @Override
3     public boolean equals(Object otherObject) {
4         if (!super.equals(otherObject)) return false;
5         Manager other = (Manager) otherObject;
6         return bonus == other.bonus;
7     }
8 }

```

Si procede in questo modo non solo per una questione di efficienza (richiede meno codice), ma soprattutto perché la superclasse è l'unica che conosce i propri campi (privati), quindi è l'unica che può confrontarli correttamente. La sottoclasse si occupa solo dei campi aggiuntivi di cui è responsabile.

Uguaglianza ed Ereditarietà: Il Problema della Simmetria

Il design del metodo `equals()` deve rispettare i cinque principi stabiliti dalla *Java Language Specification*. La violazione di uno di questi può causare bug imprevedibili quando gli oggetti vengono inseriti in collezioni (come `HashSet` o `ArrayList`): per ogni riferimento non nullo x, y, z :

- **Riflessività:** `x.equals(x)` deve essere `true`.
- **Simmetria:** `x.equals(y)  $\iff$  y.equals(x)`.
- **Transitività:** `(x.equals(y)  $\wedge$  y.equals(z))  $\implies$  x.equals(z)`.
- **Consistenza:** Il risultato non deve cambiare a meno che non cambi lo stato degli oggetti.
- **Test del Null:** `x.equals(null)` deve sempre essere `false`.

Vale la pena sapere che esistono due scuole di pensiero riguardo l'implementazione del metodo `equals()` quando si deve gestire il confronto tra una superclasse (`Employee`) e una sottoclasse (`Manager`).

L'approccio `instanceof` Molti programmatori usano `if (!(otherObject instanceof Employee))`.

- **Pro:** Rispetta il *Principio di Sostituzione di Liskov* (in pratica: posso usare un `Manager` ovunque sia richiesto un `Employee`).
- **Contro: viola la simmetria.** Se `Employee.equals` usa `instanceof`, allora `e.equals(m)` sarà `true`. Per la regola di simmetria, anche `m.equals(e)` deve essere `true`. Tuttavia, un `Manager` ha campi extra (il `bonus`): se per essere simmetrico deve ignorare il `bonus` quando parla con un `Employee`, l'uguaglianza diventa logicamente inconsistente per la sottoclasse.

L'approccio `getClass()` È quello usato nell'esempio di `Employee`.

- **Pro:** Garantisce la simmetria assoluta. Se le classi non sono identiche, il test fallisce subito.
- **Contro:** Impedisce l'uguaglianza tra tipi diversi anche se condividono la stessa logica di identità.

La Soluzione di Horstmann: Due Scenari Non esiste una regola unica, ma due scenari di design:

1. **Scenario 1: Le sottoclassi aggiungono stato**

Se un `Manager` ha campi che lo rendono "diverso" da un `Employee` (come il `bonus`), l'uguaglianza deve essere limitata alla classe esatta.

→ Usa `getClass()`.

2. **Scenario 2: L'uguaglianza è fissata dalla superclasse**

Se l'uguaglianza è determinata solo da un ID univoco definito nella superclasse e vale per tutti i figli.

→ Usa `instanceof` e dichiara il metodo `equals` come `final` nella superclasse (per impedire ai figli di romperne la simmetria).

Osservazione 4.11 (Pasticci nella libreria standard). La classe `java.sql.Timestamp` estende `java.util.Date`. Poiché `Date` usa `instanceof`, gli sviluppatori Java hanno ammesso che è impossibile per `Timestamp` essere simmetrico e accurato allo stesso tempo. È un classico esempio di cattivo design dell'ereditarietà.

Osservazione 4.12. Nel mondo professionale (specialmente con framework come Hibernate/Spring Data), l'uguaglianza basata sull'ID è lo standard quasi assoluto perché rispecchia la struttura dei database.

Riepilogo: La Ricetta Definitiva per un metodo `equals()` perfetto

Possiamo infine sintetizzare la creazione di un metodo `equals()` robusto in cinque passaggi fondamentali.

1. **Assegnazione dei nomi:** Chiamare il parametro esplicito `otherObject`. In seguito, verrà convertito in una variabile locale chiamata `other`.

2. **Test di Identità:**

```
1 if (this == otherObject) return true;
```

3. **Test del null:**

```
1 if (otherObject == null) return false;
```

Questo controllo è obbligatorio per contratto.

4. **Il bivio del Test dei Tipi:**

- Se la semantica dell'uguaglianza **può cambiare** nelle sottoclassi:

```
1 if (getClass() != otherObject.getClass()) return false;
2 ClassName other = (ClassName) otherObject;
```

- Se la semantica è **fissata nella superclasse** (uguaglianza per ID):

```
1 if (!(otherObject instanceof ClassName other)) return
  false;
```

Nota: ricordando quanto detto nel paragrafo 4.7, la variabile `other` viene creata e castata automaticamente.

5. **Confronto dei campi:**

- Usa `==` per i tipi primitivi.
- Usa `Objects.equals(a, b)` per i campi oggetto (per evitare `NullPointerException`).

```
1 return field1 == other.field1
2    && Objects.equals(field2, other.field2)
3    && ...;
```

⚠ Sottoclassi

Se stai ridefinendo `equals()` in una sottoclasse, la prima operazione dopo i controlli di base deve essere l'invocazione di del metodo della superclasse **`super.equals(otherObject)`**. Va anche ricordato che il parametro del metodo `equals()` è sempre di tipo `Object` (ma ci si protegge da questo tipo di errore usando l'annotazione `@Override`).

4.9.2 Il Metodo `hashCode()`

Come approfondito nella sezione teorica in appendice (cfr. A.1), l'hashing è un processo fondamentale per la sintesi dei dati. In Java, il metodo `hashCode()` della classe `Object` restituisce un valore intero (`int`) che funge da "impronta digitale" dell'oggetto.

L'intero restituito da `hashCode()` è calcolato in modo da minimizzare la probabilità che due oggetti diversi abbiano lo stesso valore. Mentre l'implementazione predefinita di `Object` deriva questo numero dall'indirizzo di memoria dell'istanza (identità), nelle classi che rappresentano dati esso viene sovrascritto per riflettere lo stato dell'oggetto.

In Java, l'utilizzo principale di questo metodo avviene all'interno delle **Collezioni** (come `HashSet` e `HashMap`), dove l'hash viene usato come indice per distribuire gli oggetti in diversi "compartimenti" (*buckets*), rendendo le operazioni di ricerca quasi istantanee. Riprenderemo questo concetto nel capitolo sulle collections, ma è importante capire fin da ora che un'implementazione errata di `hashCode()` può portare a prestazioni disastrose e a comportamenti imprevedibili quando gli oggetti vengono inseriti in queste strutture dati.

Esiste una regola fondamentale che ogni sviluppatore Java deve rispettare: **se due oggetti sono uguali secondo il metodo `equals()`, devono necessariamente avere lo stesso `hashCode()`.**

⚠ Attenzione

Se si sovrascrive `equals()`, si deve sovrascrivere anche `hashCode()`. Se non lo si fa, gli oggetti della classe non funzioneranno correttamente nelle strutture dati basate su hash, poiché oggetti logicamente identici verrebbero cercati in posizioni diverse della memoria.

Dalla versione 7 di Java, il modo più semplice e sicuro per implementare questo metodo è l'utilizzo della funzione statica `Objects.hash()`. Questa funzione accetta un numero variabile di argomenti e gestisce automaticamente i valori `null`.

Listing 4.17: Esempio di implementazione coordinata in `Employee`

```
1 public class Employee {
2     private String name;
3     private double salary;
4     private LocalDate hireDay;
5 }
```

```

6      @Override
7      public boolean equals(Object otherObject) {
8          // ... (implementazione vista in precedenza) ...
9      }
10
11     @Override
12     public int hashCode() {
13         // Genera l'hash combinando i campi significativi
14         return Objects.hash(name, salary, hireDay);
15     }
16 }

```

⚠ Attenzione

Se la classe contiene dei campi che sono **array**, `Objects.hash()` non ne verificherà il contenuto (userebbe l'hash dell'indirizzo di memoria dell'array). In questo caso specifico, è necessario utilizzare il metodo `Arrays.hashCode()` per calcolare un hash basato sui singoli elementi dell'array. Per lo stesso motivo, usare `Arrays.deepHashCode` per array multidimensionali.

Approfondimento: Calcolo manuale e Performance Sebbene `Objects.hash()` sia la scelta standard, in scenari di **alta performance** o in sistemi legacy, si preferisce il calcolo manuale. Il metodo `Objects.hash`, infatti, essendo un metodo *varargs*, crea un array temporaneo a ogni invocazione, introducendo un overhead che può diventare significativo in cicli critici.

Listing 4.18: Calcolo manuale ottimizzato

```

1  @Override
2  public int hashCode() {
3      // Si usano numeri primi per distribuire meglio i valori
4      return 31 * Objects.hashCode(name)
5             + 11 * Double.hashCode(salary)
6             + 13 * Objects.hashCode(hireDay);
7  }

```

Se il range dei valori dei campi è limitato e noto, è possibile progettare una funzione di hash **perfetta** (priva di collisioni). Ad esempio, per una data composta da giorno, mese e anno, una formula come:

$$\text{hash} = 31 * 12 * \text{anno} + 31 * \text{mese} + \text{giorno}$$

garantisce che ogni data possibile produca un numero univoco (entro un range ragionevole di anni), poiché i "pesi" assegnati ai campi impediscono sovrapposizioni.

⚠ Attenzione

Nel calcolo manuale, per i tipi primitivi (`double`, `long`), è fondamentale usare i metodi statici delle classi wrapper (es. `Double.hashCode(salary)`) invece di castare a `int`, per non perdere precisione e mantenere una buona distribuzione.

4.9.3 Il Metodo toString()

Il metodo `toString()` restituisce una stringa che rappresenta il valore dell'oggetto. La sua implementazione predefinita nella classe `Object` fornisce solo il nome della classe seguito dall'hash code (es. `Employee@15db9742`), un'informazione raramente utile.

Sovrascrivere questo metodo è considerata una *best practice* fondamentale per facilitare il debugging e il logging, poiché permette di ispezionare lo stato dell'oggetto in modo immediato.

Convenzione di Formattazione Horstmann raccomanda un formato standard per la stringa restituita:

NomeClasse[campo1=valore1, campo2=valore2, ...]

Listing 4.19: Implementazione di `toString` in `Employee` e `Manager`

```

1 // In Employee
2 public String toString() {
3     return getClass().getName() + "[name=" + name + ",salary=" +
4         salary + ",hireDay=" + hireDay + "]";
5 }
6 // In Manager (Sottoclasse)
7 public String toString() {
8     return super.toString() + "[bonus=" + bonus + "]";
9 }

```

Invocazione Implicita Un aspetto potente di `toString()` è che viene invocato **automaticamente** dal compilatore in due casi comuni:

1. Quando un oggetto viene concatenato a una stringa: `"Dati: " + myObject`.
2. Quando un oggetto viene passato a `System.out.println(myObject)`.

Osservazione 4.13 (Array e `toString`). Gli array ereditano il `toString` standard di `Object`, rendendo la loro stampa diretta illeggibile. Per ottenere il contenuto di un array, è necessario utilizzare i metodi statici:

- `Arrays.toString(a)`: per array monodimensionali.
- `Arrays.deepToString(a)`: per array multidimensionali.

4.9.4 Il caso particolare dei Records

Nonostante la loro sintassi semplificata, i Record rimangono pienamente inseriti nella gerarchia degli oggetti Java.

- **Ereditarietà da `Object`**: Ogni record è, a tutti gli effetti, un `Object`. Può essere assegnato a una variabile di tipo `Object` e ne possiede tutti i metodi fondamentali.
- **La classe `java.lang.Record`**: Ogni record estende implicitamente la classe astratta `java.lang.Record`. Questa classe funge da "ponte" tra il tipo specifico e la superclasse cosmica `Object`.

Proprio perché estendono implicitamente `java.lang.Record`:

1. Un record **non può estendere** altre classi.

2. Un record è **sempre final**, il che significa che nessuna classe può estendere un record.

Essendo progettati per essere "vettori di dati immutabili", per i Records Java rimuove tutto il *boilerplate code* relativo ai metodi della classe `Object`: per ogni Record, il compilatore genera automaticamente le seguenti implementazioni:

- **`equals()`**: Confronta due record componente per componente (state-based equality). Due record sono uguali se hanno lo stesso tipo e i valori di tutti i loro campi coincidono.
- **`hashCode()`**: Generato in modo coerente con `equals()`.
- **`toString()`**: Restituisce una stringa leggibile che elenca tutti i componenti e i loro valori (es. `Point[x=10.0, y=20.0]`).

Osservazione 4.14. Sebbene sia possibile effettuare l'override di `equals()` anche in un record, ciò ne violerebbe quasi sempre la semantica. I record dovrebbero rappresentare **solo** i dati che contengono; pertanto, l'uguaglianza basata sullo stato generata automaticamente è quasi sempre quella corretta.

4.10 Classi Astratte

Salendo nella gerarchia dell'ereditarietà, le classi diventano sempre più generiche e astratte. A un certo punto, una classe antenata può diventare così generica che non ha più senso considerarla come un'entità a sé stante, ma piuttosto come una base comune per altre classi.

Una classe dichiarata con la parola chiave **`abstract`** non può essere istanziata (non è possibile creare oggetti con `new NomeClasse`). Essa funge da "modello" per le sue sottoclassi concrete.

Le classi astratte possono opzionalmente contenere metodi astratti, cioè dichiarati senza un'implementazione (senza corpo: il blocco di codice è assente). Una sottoclasse di una classe astratta può essere a sua volta astratta (se non implementa tutti i metodi astratti) o concreta (e in questo caso deve implementare tutti i metodi astratti).

Listing 4.20: Esempio di classe astratta `Person`

```

1 public abstract class Person {
2     private String name;
3
4     public Person(String name) { this.name = name; }
5
6     public String getName() { return name; }
7
8     // Metodo astratto: ogni sottoclasse deve decidere come
9     //    descriversi
10    public abstract String getDescription();
11 }
```

Caratteristiche principali delle classi astratte:

- **Stato e Comportamento:** Anche se astratta, una classe può avere campi (variabili di istanza) e metodi concreti. È buona norma spostare campi e metodi comuni il più in alto possibile nella gerarchia.
- **Istanziamento:** È vietato fare `new Person(...)`. Tuttavia, è perfettamente legale (e fondamentale per il polimorfismo) creare variabili di tipo astratto che puntano a oggetti concreti:


```
Person p = new Student("Emanuele", "Matematica");
```

Osservazione 4.15 (Polimorfismo e Classi Astratte). In un array di tipo `Person[]`, possiamo inserire sia `Student` che `Employee`. Quando invochiamo `p.getDescription()` su un elemento dell'array, la JVM chiamerà il metodo corretto in base all'oggetto reale (*dynamic binding*), anche se il metodo era dichiarato come astratto nella classe base. Senza la dichiarazione **abstract** nel padre, il compilatore ci impedirebbe di chiamare il metodo sulla variabile di tipo `Person`.

4.11 Classi Sigillate (*Sealed Classes*)

Introdotte stabilmente in Java 17, le **sealed classes** permettono allo sviluppatore di restringere la gerarchia di ereditarietà, specificando esattamente quali sottoclassi possono estendere una determinata classe o interfaccia.

Una classe dichiarata con il modificatore **sealed** deve elencare esplicitamente le sue sottoclassi autorizzate tramite la clausola **permits**.

Listing 4.21: Esempio di Sealed Class

```
1 public sealed class Shape permits Circle, Square, Triangle {
2     // ...
3 }
```

I motivi principali per utilizzare le classi sigillate sono:

- **Gerarchie Controllate:** Evitare che chiunque possa estendere una classe e potenzialmente rompere l'integrità del design.
- **Controllo del Dominio:** Utile quando si modella una struttura dati "finita" (come i tipi di un formato dati, es. JSON).
- **Analisi Esaustiva:** Il compilatore può verificare se in uno **switch** abbiamo coperto tutti i casi possibili della gerarchia, rendendo superfluo il ramo **default**.

I Vincoli delle Sottoclassi Il meccanismo di sigillatura impone regole ferree sulle classi figlie. Ogni classe elencata nella clausola **permits** deve estendere la classe sigillata e **deve obbligatoriamente** dichiarare uno dei seguenti tre modificatori per esplicitare come prosegue (o termina) la gerarchia:

1. **final:** Impedisce ulteriori estensioni. La gerarchia si chiude definitivamente su questo ramo. È la scelta più comune (ed è il comportamento intrinseco se la sottoclasse è un **record**).
2. **sealed:** Permette a questa sottoclasse di essere a sua volta estesa, ma solo da un nuovo set ristretto e definito di classi (richiede una propria clausola **permits**).
3. **non-sealed:** Interrompe la sigillatura per questo specifico ramo. Da questo punto in giù, la classe torna a essere "aperta" e chiunque potrà estenderla. *Nota: Java ha introdotto questo modificatore con il trattino, un'eccezione rispetto alla sintassi tradizionale.*

Listing 4.22: Esempio dei modificatori obbligatori

```
1 // 1. Ramo chiuso: non estensibile
2 public final class Circle extends Shape { ... }
3
4 // 2. Ramo sigillato: controllato a sua volta
5 public sealed class Square extends Shape permits ColoredSquare {
6     ... }
```

```
6  
7 // 3. Ramo aperto: estensibile liberamente  
8 public non-sealed class Triangle extends Shape { ... }
```

Regole di Compilazione e Package Per garantire che il compilatore possa verificare la validità della gerarchia, tutte le sottoclassi permesse devono trovarsi nello **stesso modulo** della classe sigillata (o nello **stesso package** se si opera in moduli senza nome, come avviene nei progetti base).

Osservazione 4.16 (Omissione di `permits`). Se la classe sigillata e le sue sottoclassi sono definite tutte all'interno dello **stesso file sorgente** (ad esempio definendo più classi top-level in un solo file `.java` o usando classi annidate), è possibile omettere la clausola `permits`. Il compilatore inferirà automaticamente l'elenco delle classi permesse analizzando il file.

Interfacce, Lambdas e Inner Classes

5.1 Interfacce in Java

Un'interfaccia è un costrutto della programmazione ad oggetti che definisce un **contratto** di comportamento. Mentre una classe descrive *cosa un oggetto è* (stato e comportamento), un'interfaccia descrive *cosa un oggetto sa fare*.

Un'interfaccia viene dichiarata con la parola chiave **interface**. Per sua natura, è una struttura puramente astratta:

- **Metodi:** Tutti i metodi dichiarati in un'interfaccia sono implicitamente **public** e **abstract** (fino a Java 7). Non è necessario specificare questi modificatori.
- **Campi:** Qualsiasi variabile dichiarata in un'interfaccia è implicitamente **public**, **static** e **final** (costante).
- **Istanziamento:** Non è possibile creare un'istanza di un'interfaccia tramite **new**. Tuttavia, un'interfaccia può essere utilizzata come **tipo di dato** per riferimenti a oggetti di classi che la implementano. In tale contesto, il polimorfismo ed il pattern matching funzionano in modo del tutto analogo a quanto visto nel capitolo precedente.

Una classe "aderisce" al contratto dell'interfaccia usando la parola chiave **implements**.

- La classe deve fornire un'implementazione concreta (*override*) per **tutti** i metodi astratti dell'interfaccia, a meno che la classe stessa non sia dichiarata **abstract**. (N.B.: i metodi di un'interfaccia devono essere implementati con visibilità non inferiore; essendo implicitamente **public**, essi devono essere implementati con la stessa visibilità).
- **Ereditarietà Multipla:** A differenza delle classi, Java permette a una classe di implementare un numero arbitrario di interfacce, risolvendo il problema dell'ereditarietà multipla.

Oltre alle proprietà standard appena citate, le interfacce moderne presentano caratteristiche avanzate che ne estendono la potenza:

- **Metodi static o private** (Java 8+): Un'interfaccia può fornire simili metodi di utilità (ovviamente non astratti), ma non possono essere invocati su istanze di classi che implementano l'interfaccia. Devono essere chiamati direttamente sull'interfaccia stessa.
- **Estensione tra interfacce:** Un'interfaccia può estendere una o più altre interfacce utilizzando la parola chiave **extends**. A differenza delle classi, qui l'ereditarietà multipla è permessa: un'interfaccia "figlia" eredita tutti i contratti delle "madri".
- **Implementazione in Records ed Enum:** Anche i **records** (introdotti stabilmente in Java 16) e le **enum** possono implementare interfacce. Questo permette, ad esempio, di definire un comportamento comune per un insieme di costanti o per oggetti che sono puri contenitori di dati.
- **Sealed Interfaces** (Java 17+): È possibile dichiarare un'interfaccia come **sealed**. In questo caso, lo sviluppatore può specificare esattamente quali classi o interfacce sono

autorizzate a implementarla tramite la clausola `permits`, garantendo un controllo totale sulla gerarchia.

5.1.1 L'interfaccia Comparable

L'interfaccia `Comparable` è uno degli esempi più chiari di come Java utilizzi le interfacce per definire protocolli di interazione tra oggetti.

```
1 public interface Comparable {  
2     int compareTo(Object other);  
3 }
```

Il "contratto" di questa interfaccia si sviluppa su tre livelli distinti:

- **Obbligo Sintattico:** A livello puramente formale, il compilatore richiede solo che la classe concreta implementi il metodo `compareTo` con la firma corretta e che restituisca un valore di tipo `int`. Il compilatore **non entra nel merito** della logica interna: se il metodo restituisse sempre 42, il codice sarebbe sintatticamente corretto e verrebbe compilato senza errori, pur essendo logicamente inutile.
- **Obbligo Semantico:** Molti algoritmi (come `Arrays.sort`) sono programmati assumendo che lo sviluppatore abbia rispettato la logica del confronto. Se la logica di `compareTo` è incoerente (es. $x < y$ ma il metodo dice il contrario), gli algoritmi di ordinamento produrranno risultati errati, loop infiniti o eccezioni a runtime. Si parla in tal caso di **contratto semantico**: l'interfaccia garantisce la forma, lo sviluppatore garantisce il significato.
- **La Logica del Valore di Ritorno:** La convenzione sui valori di ritorno è basata sul segno numerico, il che permette una gestione matematica molto efficiente del confronto:
 - **Valore Negativo** (< 0): Indica che x (l'oggetto corrente) è più piccolo di y (`other`). L'oggetto x deve apparire *prima* di y .
 - **Zero** ($= 0$): Indica che gli oggetti sono equivalenti ai fini dell'ordinamento.
 - **Valore Positivo** (> 0): Indica che x è più grande di y . L'oggetto x deve apparire *dopo* di y .



Il Contratto Semantico

Il **contratto semantico** rappresenta l'insieme di vincoli e assunzioni sul comportamento di un modulo che non possono essere espressi tramite la sintassi del linguaggio (il "contratto sintattico").

In Java, molte librerie sono scritte assumendo che lo sviluppatore conosca e rispetti queste convenzioni. Un esempio classico è il legame tra `equals()` e `hashCode()`:

- Se due oggetti sono uguali secondo `equals()`, **devono** restituire lo stesso `hashCode()`.

La violazione di questo contratto non genera errori di compilazione, ma porta a comportamenti imprevedibili e bug logici difficili da individuare (es. oggetti "scomparsi" da una `HashSet`).

Per quanto riguarda le interfacce, in Java sono lo strumento principale per definire il contratto sintattico, ma la documentazione Javadoc è il luogo dove viene sancito il contratto semantico.

Esempio Pratico: La Classe `Employee`

Supponiamo di voler rendere una classe "ordinabile" tramite i metodi di sistema (come `Arrays.sort`); essa deve innanzitutto implementare l'interfaccia `Comparable`: si utilizza la parola chiave `implements` nella firma della classe per comunicare al compilatore che la classe si impegna a rispettare il contratto di `Comparable`:

```
1 class Employee implements Comparable {  
2     // ...  
3 }
```

La classe deve poi fornire il corpo del metodo `compareTo`. Supponendo di voler ordinare i dipendenti in base al salario (`salary`), l'implementazione sarà la seguente:

```
1 public int compareTo(Object otherObject) {  
2     // Cast obbligatorio: l'interfaccia accetta Object,  
3     // noi dobbiamo trattarlo come Employee  
4     Employee other = (Employee) otherObject;  
5  
6     // Utilizzo del metodo statico di utilità  
7     return Double.compare(this.salary, other.salary);  
8 }
```

Da notare come, invece di sottrarre manualmente i salari (operazione rischiosa con i numeri a virgola mobile a causa della precisione), abbiamo usato il metodo statico `Double.compare(x, y)`. Esso restituisce automaticamente $-1, 0, 1$ rispettando perfettamente il contratto semantico dell'interfaccia.

Grazie a questa implementazione, se avessimo un array di dipendenti chiamato `staff`, potremmo semplicemente scrivere `Arrays.sort(staff)`. L'algoritmo di ordinamento chiamerà internamente il metodo `compareTo` su ogni coppia di dipendenti per decidere come spostarli.

Evoluzione: l'interfaccia `Comparable` generica

Sebbene i generics saranno trattati in un capitolo successivo, vale la pena accennarne l'uso nel contesto specifico che stiamo trattando.

Nell'esempio precedente, l'interfaccia `Comparable` è stata utilizzata in una forma "grezza" (*raw type*), il che comporta alcuni svantaggi significativi in termini di sicurezza dei tipi e leggibilità del codice; nello specifico, poiché il metodo dell'interfaccia accetta un `Object`, abbiamo dovuto eseguire un cast esplicito a `Employee` per poter accedere al campo `salary`. Se avessimo passato un oggetto di tipo diverso (es. una `String`), il codice avrebbe lanciato una `ClassCastException`.

A partire da Java 5, l'interfaccia `Comparable` è stata trasformata in un **tipo generico**. Questo aggiornamento permette di specificare immediatamente con quale tipo di oggetti la classe è in grado di confrontarsi:

```
1 public interface Comparable<T> {  
2     int compareTo(T other);  
3 }
```

Possiamo quindi migliorare la nostra classe `Employee` implementando la versione generica di `Comparable`, evitando così il fastidioso downcasting:

```

1 class Employee implements Comparable<Employee>
2 {
3     public int compareTo(Employee other)
4     {
5         return Double.compare(salary, other.salary);
6     }
7     . . .
8 }

```

Il vantaggio principale di questa evoluzione è che il compilatore ora può garantire la coerenza dei tipi durante la compilazione, eliminando la possibilità di errori di tipo a runtime e migliorando la leggibilità del codice.

⚠ Attenzione

L'uso di `Comparable<T>` è oggi lo standard assoluto. La versione non generica (*raw type*) viene mantenuta solo per compatibilità con il codice scritto prima del 2004, ma il suo utilizzo è fortemente sconsigliato (*deprecated* nella logica di progettazione).

Comparable ed Ereditarietà: Il problema dell'Antisimmetria

Il contratto di `Comparable` impone una regola matematica ferrea definita **antisimmetria**:

$$\text{sgn}(x.\text{compareTo}(y)) = -\text{sgn}(y.\text{compareTo}(x)) \quad (5.1)$$

In termini semplici: se invertiamo gli addendi del confronto, il segno del risultato deve invertirsi. Se $x < y$, allora deve essere vero che $y > x$.

I problemi insorgono quando una sottoclasse (es. `Manager`) estende una superclasse che implementa `Comparable` (es. `Employee`).

Se `Manager` esegue l'override di `compareTo` tentando di confrontare campi specifici dei manager, si rischia di rompere l'antisimmetria:

- `impiegato.compareTo(manager)`: Funziona, perché il manager viene trattato come un semplice impiegato (upcasting).
- `manager.compareTo(impiegato)`: Lancia una `ClassCastException` se il manager tenta di forzare l'impiegato a essere un manager per leggerne i campi extra.

Esistono due strategie per gestire il confronto in una gerarchia di classi, a seconda del requisito di business:

Scenario A: Confronto vietato tra classi diverse Se classi diverse non devono mai essere confrontate tra loro (anche se una è sottoclasse dell'altra), il metodo `compareTo` deve iniziare con un controllo rigoroso sulla classe di appartenenza:

```

1 if (getClass() != other.getClass())
2     throw new ClassCastException();

```

Questo garantisce che il confronto avvenga solo tra "pari", evitando asimmetrie nel lancio di eccezioni.

Scenario B: Algoritmo di confronto comune Se esiste un modo logico per confrontare oggetti di diverse sottoclassi (es. una gerarchia aziendale), si deve adottare una strategia centralizzata:

1. Definire il metodo `compareTo` come **final** nella superclasse.
2. Utilizzare un metodo di supporto (es. `rank()`) che ogni sottoclasse può sovrascrivere per definire il proprio "peso" o posizione nell'ordinamento.

Esempio 5.1. In una gerarchia `Employee`, `Manager`, `Executive`, si può definire un metodo `rank()` che restituisce un intero (es. 1 per segretari, 2 per manager, 3 per executive). Il `compareTo` nella superclasse userà questo valore per stabilire chi "viene prima", garantendo che la regola dei segni sia sempre rispettata.

5.1.2 Interfacce vs Classi Astratte

Una domanda sorge spontanea: perché i progettisti di Java hanno introdotto le interfacce se esistevano già le classi astratte? Non si poteva definire `Comparable` come una classe astratta?

Listing 5.1: Perché questo approccio fallisce

```

1 // Se Comparable fosse una classe astratta...
2 abstract class Comparable {
3     public abstract int compareTo(Object other);
4 }
5
6 // ...Employee non potrebbe estenderla se fosse già figlio di
   Person
7 class Employee extends Person, Comparable // ERRORE DI
   COMPILAZIONE

```

Il motivo fondamentale è che Java permette a una classe di estendere **una sola** superclasse (ereditarietà singola). Se `Employee` estende già `Person`, non può estendere anche `Comparable`.

Tuttavia, una classe può implementare un numero arbitrario di interfacce:

```

1 class Employee extends Person implements Comparable, Serializable
   , Cloneable // OK

```

I progettisti di Java (ispirandosi a C++) hanno scelto di non supportare l'ereditarietà multipla tra classi perché rende il linguaggio:

- **Complesso:** si pensi al "diamante della morte" in C++ dove non si sa quale metodo della superclasse invocare.
- **Meno efficiente:** la gestione della memoria e dei puntatori diventa più onerosa.

Le interfacce offrono quasi tutti i benefici dell'ereditarietà multipla evitando però queste complicazioni, poiché non trasportano **stato** (variabili di istanza) ma solo **comportamento**.

Guida alla scelta: Classe Astratta o Interfaccia? La scelta tra una classe astratta e un'interfaccia non è solo tecnica, ma concettuale. Possiamo riassumere i criteri di scelta in questo modo:

- **Identità vs. Comportamento (Is-a vs. Can-do):**

- **Classe Astratta (Is-a)**: Si usa quando si vuole definire l'identità core di un oggetto. Una classe astratta risponde alla domanda: *"Che cos'è questo oggetto?"*. Esempio: **Aereo** è un **Veicolo**.
- **Interfaccia (Can-do / Role)**: Si usa per definire un'abilità o un ruolo che un oggetto può ricoprire. Risponde alla domanda: *"Cosa sa fare questo oggetto?"*. Esempio: **Aereo** è **Volante**, ma lo è anche un **Uccello** o un **Drone**.
- **Stato vs. Puro Contratto**:
 - Se hai bisogno di condividere dello **stato** (variabili di istanza non costanti) o costruttori comuni tra le sottoclassi, la **classe astratta** è l'unica scelta.
 - Se vuoi definire solo un **contratto** comportamentale senza preoccuparti di come i dati verranno memorizzati, l' **interfaccia** è la scelta più pulita.

5.1.3 Metodi Default: Evoluzione e Compatibilità

Apartire da Java 8, utilizzando il modificatore `default` è possibile definire un metodo di istanza all'interno di un'interfaccia che include un'implementazione concreta: le classi che implementano l'interfaccia ereditano automaticamente il metodo senza l'obbligo di sovrascriverlo.

Ciò permette l'evoluzione delle interfacce (aggiunta di nuovi metodi) senza rompere il codice esistente che le implementa (*Binary Compatibility*). Tuttavia, ciò pone un problema: se una classe implementa due interfacce con metodi `default` identici (stessa firma), si verifica un conflitto di nomi. Java impone le seguenti regole per risolvere questo scenario:

1. **Le classi vincono sulle interfacce**: Se una superclasse fornisce un'implementazione, il metodo `default` dell'interfaccia viene ignorato.
2. **Override obbligatorio**: Se non c'è una superclasse a risolvere il conflitto, la classe deve obbligatoriamente implementare il metodo, decidendo quale comportamento adottare o fornendone uno nuovo.

5.2 Il Pattern Callback: Passare Comportamento

Introduciamo ora un concetto che ci permetterà di comprendere appieno le Lambda Expressions: il pattern **callback**.

Nello studio della programmazione abbiamo imparato a passare dati (primitivi o oggetti) come parametri. Tuttavia, spesso nasce l'esigenza di passare non un dato, ma un **comportamento** (una funzione). In linguaggi come C o C++ è possibile passare direttamente un *puntatore a funzione*. Si dice al programma: "Esegui il codice che si trova a questo indirizzo di memoria". In Java, invece, le funzioni non esistono indipendentemente dalle classi. Per passare un'azione, dobbiamo "impacchettarla" dentro un riferimento a un oggetto.

Le **Lambdas** (che vedremo a breve) sono semplicemente una "scorciatoia sintattica" per questo pattern. Senza aver compreso che in Java stiamo passando un *riferimento a un oggetto che implementa un contratto*, la sintassi delle Lambda sembrerebbe magica. Capire la callback significa capire *cosa succede dietro le quinte* di ogni Lambda Expression.

In sintesi: una callback è il modo in cui Java permette di trattare un blocco di codice come se fosse un oggetto, permettendone il passaggio tra diversi moduli del programma.

Il pattern **callback** (richiamata) si realizza come segue:

1. Si definisce un'interfaccia con un singolo metodo (il "contratto").
2. Si crea un oggetto che implementa quel metodo.

3. Si passa il **riferimento** a tale oggetto.

L'oggetto funge quindi da *contenitore* (wrapper) per la funzione che vogliamo eseguire in differita.

5.2.1 Un esempio di Callback: l'interfaccia Comparator

Il pattern callback trova una sua tipica applicazione nella personalizzazione di algoritmi esistenti, come l'ordinamento.

Consideriamo ad esempio la classe `String` che ha già un ordine naturale (lessicografico) definito tramite `Comparable`. Se vogliamo ordinare per lunghezza, non possiamo modificare la classe `String`.

Seguendo il pattern callback, possiamo però definire un'interfaccia con un singolo metodo di confronto (in questo caso, `Comparator`) e creiamo una classe che implementa questo metodo per confrontare le stringhe in base alla loro lunghezza.

L'interfaccia `Comparator` contiene il metodo astratto `compare` che accetta due oggetti dello stesso tipo e restituisce un intero che indica l'ordine relativo tra di essi:

Listing 5.2: il metodo compare di Comparator

```

1 public interface Comparator<T> {
2     int compare(T first, T second);
3 }

```

Quando implementiamo il metodo `int compare(T o1, T o2)`, stiamo definendo una relazione di **ordine totale**. La documentazione ufficiale (JDK 21) impone tre vincoli logici (il **contratto semantico** di cui abbiamo già parlato) che garantiscono la stabilità degli algoritmi di ordinamento:

1. Antisimmetria:

$$\text{sgn}(\text{compare}(x, y)) == -\text{sgn}(\text{compare}(y, x))$$

Se $x > y$, allora necessariamente $y < x$. Se il primo confronto lancia un'eccezione, deve farlo anche il secondo.

2. Transitività:

$$(\text{compare}(x, y) > 0 \wedge \text{compare}(y, z) > 0) \implies \text{compare}(x, z) > 0$$

Se x è maggiore di y e y è maggiore di z , allora x deve essere maggiore di z . È il pilastro che permette agli algoritmi di evitare cicli infiniti.

3. Coerenza dell'Uguaglianza:

$$\text{compare}(x, y) == 0 \implies \text{sgn}(\text{compare}(x, z)) == \text{sgn}(\text{compare}(y, z))$$

Se due oggetti sono considerati "uguali" dal comparatore, devono comportarsi allo stesso modo rispetto a un terzo oggetto z .

Osservazione 5.1 (Consistenza con Equals). Il punto 3. non è **strettamente richiesto**, ma è caldamente raccomandato.

- Un comparatore che viola questa condizione è detto *"inconsistent with equals"*.

- **Il rischio:** Collezioni ordinate come `TreeSet` o `TreeMap` usano `compare` (e non `equals`) per gestire i duplicati. Se `compare` dice che due oggetti sono uguali ma `equals` dice di no, la collezione violerà il contratto generale dei `Set`, rifiutando l'inserimento del secondo oggetto. Quando un programmatore non rispetta questa coerenza, dovrebbe indicarlo chiaramente nella documentazione del comparatore per evitare confusione agli utenti.

Seguendo il pattern callback, creiamo una classe che implementa `Comparator<String>` per confrontare le stringhe in base alla loro lunghezza:

Listing 5.3: Comparator come "pacchetto" di una funzione di confronto

```

1 // 1. Definiamo il comportamento (la funzione di callback)
2 class LengthComparator implements Comparator<String> {
3     public int compare(String first, String second) {
4         return first.length() - second.length();
5     }
6 }
7
8 // 2. Passiamo l'oggetto/funzione all'algoritmo di sorting
9 Arrays.sort(friends, new LengthComparator());

```

Per utilizzare il comparatore personalizzato, la classe `Arrays` fornisce un metodo specifico che sfrutta il polimorfismo delle interfacce:

```
public static <T> void sort(T[] a, Comparator<? super T> c)
```

- **Parametro a:** L'array di oggetti da ordinare.
- **Parametro c:** Il riferimento all'oggetto che implementa la callback di confronto. La sintassi `? super T` indica che il comparatore può essere di tipo `T` oppure un supertipo di `T`, garantendo così una maggiore flessibilità.

Conclusioni

Una volta compreso che un'interfaccia può essere usata per passare un comportamento (callback), si apre una vasta gamma di possibilità, ad esempio:

- **Ordinamenti multipli:** Fornire criteri diversi per la stessa classe senza modificarne l'ordine naturale.
- **Filtraggio dinamico:** Passare logiche di selezione diverse a un unico metodo di ricerca.

Tuttavia, implementare manualmente ogni singola logica tramite classi dedicate risulterebbe eccessivamente verboso. In Java moderno, questa potenza viene sfruttata appieno tramite le **Lambda Expressions** spiegate a seguire, che permettono di definire queste callback "al volo", mantenendo il codice conciso, leggibile ed elegante.

5.3 Lambda Expressions

5.3.1 Introduzione e Motivazioni

Le *Lambda Expressions* nascono dalla necessità di passare blocchi di codice in modo conciso. In precedenza, Java imponeva un approccio puramente *Object-Oriented*, richiedendo la creazione di classi e oggetti anche per frammenti di logica minimi.

- **Codice per dopo:** Il cuore delle Lambda è la possibilità di definire un'azione ora per eseguirla in futuro (callback).
- **Evoluzione del linguaggio:** L'introduzione delle Lambda segna l'ingresso ufficiale di Java nel mondo della *Programmazione Funzionale*, permettendo di scrivere API molto più espressive e consistenti.

Per anni, i progettisti di Java hanno evitato questa funzione per mantenere il linguaggio semplice. Tuttavia, il confronto con altri linguaggi ha dimostrato che la possibilità di manipolare blocchi di codice rende lo sviluppo molto più efficiente e meno prone a errori di "boilerplate".

5.3.2 Sintassi delle Lambda Expressions

La sintassi delle *lambdas* ricorda molto da vicino il simbolo matematico utilizzato per definire una funzione di più variabili:

$$(x_1, \dots, x_n) \rightarrow f(x_1, \dots, x_n)$$

In Java, la sintassi standard nel caso più generale ed esplicito rispecchia fedelmente questa struttura ed è la seguente:

```
(Tipo1 p1, Tipo2 p2, ..., TipoN pN) -> { // corpo della lambda }
```

Il blocco di codice racchiuso tra parentesi graffe {} può contenere più istruzioni. Se il corpo della lambda è costituito da una singola istruzione, è possibile omettere le parentesi graffe e la parola chiave **return** (se presente), rendendo la sintassi ancora più compatta.

Listing 5.4: Esempi di sintassi delle Lambda Expressions

```

1 (String first, String second) ->
2   first.length() - second.length()
3
4 (String first, String second) ->
5 {
6     if (first.length() < second.length()) return -1;
7     else if (first.length() > second.length()) return 1;
8     else return 0;
9 }
```

Se non ci sono parametri, si usano le parentesi vuote (proprio come nei metodi):

```

1 () -> { for (int i = 100; i >= 0; i--) System.out.println(i); }
```

Se i tipi dei parametri possono essere dedotti dal contesto (*inferenza dei tipi*), è possibile ometterli (vedremo a breve questo tipo di espressione):

```

1 Comparator<String> comp =
2   (first, second) -> first.length() - second.length();
```

Se e solo se è presente un unico parametro e il suo tipo è omesso per inferenza, le parentesi diventano facoltative (N.B.: se si decide di dichiarare esplicitamente il tipo, le parentesi tornano obbligatorie):

```

1 // Forma completa:
2 (Double x) -> x * x
3
4 // Forma minimalista:
5 x -> x * x

```

Osservazione 5.2. Al contrario dei metodi, nelle Lambda il tipo di ritorno è sempre dedotto dal contesto e non può essere dichiarato esplicitamente.

⚠ Attenzione

Se il corpo della Lambda ha dei rami condizionali (`if/else`), è **illegale** che alcuni rami restituiscano un valore e altri no. Esempio errato: `(int x) -> { if (x >= 0) return 1; }` Il compilatore darà errore perché manca il valore di ritorno per il caso `x < 0`.

A partire da Java 21, è stata introdotta la possibilità di utilizzare il carattere **underscore** (`_`) per denotare parametri di una Lambda expression che non vengono mai utilizzati nel corpo della funzione (abbiamo già incontrato questo tipo di sintassi nel pattern matching con `switch` o `instanceof`).

Listing 5.5: Esempi di Unnamed Parameters

```

1 // Caso 1: ActionListener (ignora l'oggetto Event)
2 ActionListener listener = _ -> System.out.println("Evento
   rilevato alle " + Instant.now());
3
4 // Caso 2: Comparator (ignora entrambi i parametri, ad esempio
   per un ordinamento neutro)
5 Comparator<String> comp = (_, _) -> 0;

```

Osservazione 5.3. Sebbene nel testo venga citata come *preview feature* di Java 21, l'uso dell'underscore per variabili e parametri non utilizzati è diventato uno standard nelle versioni successive, riducendo i warning del compilatore relativi a variabili dichiarate ma mai lette.

5.3.3 Uso corretto delle Lambda Expressions

A questo punto abbiamo capito il concetto di Lambda come "funzione" e ne abbiamo visto la sintassi; ora vedremo cosa rappresenta esattamente e come si usa.

In Java, una lambda expression non può esistere in isolamento; essa deve essere associata a una **Interfaccia Funzionale**.

💡 Definizione: interfaccia funzionale

Un'interfaccia si dice **funzionale** se dichiara esattamente **un solo metodo astratto** che non abbia la stessa firma di un metodo di `java.lang.Object`. Esempi classici:

- `Comparator` (metodo `compare`)
- `ActionListener` (metodo `actionPerformed`)



- `Runnable` (metodo `run`)

Osservazione 5.4. L'interfaccia `Comparable` contiene i due metodi astratti `compareTo` e `equals`, ma il secondo è un metodo di `Object`, dunque è un'interfaccia funzionale.

**Tip**

Possiamo usare una lambda ogni volta che ci si aspetta un riferimento ad un oggetto di una classe che implementa un'interfaccia funzionale (e questo è l'unico caso in cui possiamo usare una lambda).

Il seguente esempio mostra come l'uso della lambda eviti la verbosità di dover creare una classe dedicata (come abbiamo fatto con `LengthComparator` nel paragrafo del callback); dall'esempio è anche chiaro cosa succeda quando si usa una lambda e perché il metodo dell'interfaccia debba essere uno solo:

Listing 5.6: La Lambda come "implementazione al volo"

```

1 // Il compilatore vede che Arrays.sort vuole un Comparator.
2 // Comparator ha un solo metodo astratto (che non sia di Object):
3 // int compare(T o1, T o2).
4 // Quindi "mappa" la lambda su quel metodo:
5 // tratta i parametri come se fossero quelli di compare
6 // e il corpo come se fosse una sua implementazione.
7 Arrays.sort(words, (first, second) -> first.length() - second.
    length());

```

Quello che succede "sotto il cofano" è che il compilatore genera una classe sintetica che implementa l'interfaccia e ne crea un'istanza. Per il programmatore, però, è come se passasse direttamente la funzione.

A fronte di questa comodità, però, chi usa una lambda deve *conoscere bene l'interfaccia funzionale ed il metodo astratto che sta implicitamente implementando*. Infatti, affinché una *lambda expression* sia valida, deve essere **assegnabile** al tipo dell'interfaccia funzionale richiesta. Questo implica una corrispondenza esatta su tre livelli:

Parametri : Il numero e l'ordine dei parametri della lambda devono coincidere con quelli del metodo astratto. I tipi possono essere omessi (inferenza), ma devono essere logicamente corretti.

Tipo di Ritorno : Se il metodo dell'interfaccia non è `void`, il corpo della lambda deve produrre un valore di tipo compatibile.

Eccezioni : La lambda non può lanciare eccezioni *checked* che non siano già dichiarate nella clausola `throws` del metodo dell'interfaccia.

Oltre alla compatibilità sintattica (che controlla il compilatore), resta a carico del programmatore il rispetto del **contratto semantico**. Ad esempio, scrivendo una lambda per un `Comparator`, bisogna assicurarsi che la logica implementata sia transitiva e antisimmetrica, anche se la sintassi è corretta.

Infine, bisogna essere consapevoli di dove si sta passando la lambda: se la si passa a un metodo che si aspetta un certo tipo di interfaccia funzionale, non se ne potrà usare un'altra. Ad esempio, non si può passare una lambda che implementa `Runnable` a un

metodo che si aspetta un `Comparator` (come `Arrays.sort` di cui abbiamo visto la firma qui 5.2.1), anche se la sintassi della lambda è corretta per entrambi i casi.

L'annotazione `@FunctionalInterface`

Per evitare errori, si usa spesso l'annotazione `@FunctionalInterface` sopra la definizione dell'interfaccia funzionale. Non è obbligatoria, ma serve a dire al compilatore: "Avvisami se per sbaglio aggiungo un secondo metodo astratto a questa interfaccia, perché romperei tutte le lambda che la usano".

5.3.4 Interfacce Funzionali Standard

Per promuovere il riutilizzo del codice e non costringere il programmatore a definire nuove interfacce per ogni necessità, Java fornisce il pacchetto `java.util.function`. Esso contiene una serie di interfacce funzionali predefinite che coprono il 99% dei casi d'uso comuni, soprattutto quelli legati alla manipolazione di collezioni e flussi di dati. Possono essere ricondotte a uno di questi quattro modelli:

`Predicate<T>` (Il Filtro): Accetta un argomento e restituisce un `boolean`. Viene usato per testare condizioni.

`Consumer<T>` (L'Esecutore): Accetta un argomento e non restituisce nulla (`void`). Viene usato per operazioni con "side-effect" (es. stampa a video).

`Function<T, R>` (Il Trasformatore): Accetta un argomento di tipo T e restituisce un risultato di tipo R .

`Supplier<T>` (Il Generatore): Non accetta argomenti e restituisce un oggetto di tipo T .

Interfaccia	Metodo	Ritorno	Scopo Tipico
<code>Predicate<T></code>	<code>test(T)</code>	<code>boolean</code>	Filtrare o validare dati.
<code>Consumer<T></code>	<code>accept(T)</code>	<code>void</code>	Eeguire un'azione sull'input.
<code>Function<T, R></code>	<code>apply(T)</code>	R	Trasformare T in R .
<code>Supplier<T></code>	<code>get()</code>	T	Creare o fornire un valore.

Tabella 5.1: Sintesi delle interfacce funzionali standard.

Ognuna di queste è di per sé un'interfaccia funzionale, ma anche "capostipite" (in senso concettuale: non c'è ereditarietà) di una famiglia di varianti ottimizzate per tipi primitivi (evitando il costo computazionale dei wrapper) e multi-argomento.

Capostipite	Variante Primitiva	Variante Multi-argomento
<code>Predicate<T></code>	<code>IntPredicate</code> , <code>LongPredicate</code>	<code>BiPredicate<T, U></code>
<code>Consumer<T></code>	<code>DoubleConsumer</code> , <code>IntConsumer</code>	<code>BiConsumer<T, U></code>
<code>Function<T, R></code>	<code>IntToDoubleFunction</code> , ...	<code>BiFunction<T, U, R></code>
<code>Supplier<T></code>	<code>BooleanSupplier</code> , <code>IntSupplier</code>	//

Osservazione 5.5. (Inferenza dell'Overload) Quando un metodo ha più overload che accettano interfacce funzionali diverse, il compilatore sceglie automaticamente quale lambda è compatibile con quale interfaccia, basandosi sulla sintassi e sul contesto in cui la lambda viene usata. Ad esempio:

```

1      var values = new int[100];
2      Arrays.setAll(values, i -> i * i);

```

Qui l'overload di `setAll` che accetta un `int[]` usa `IntUnaryOperator` come secondo parametro, che lavora con gli interi evitando l'autoboxing. Spesso si usano le lambda senza nemmeno pensare a questi dettagli. Il compilatore si occupa di mappare la lambda sull'interfaccia funzionale corretta, garantendo così prestazioni ottimali.

Esempio: Valutazione Eager vs. Lazy Le interfacce `Supplier<T>` sono spesso usate per permettere di passare da una valutazione immediata (**Eager Evaluation**) a una valutazione differita o "pigra" (**Lazy Evaluation**), come mostriamo nel seguente esempio.

Nel primo caso, il valore di default viene calcolato **sempre**, prima ancora che il metodo venga eseguito:

```

1  // EAGER: La data viene istanziata anche se 'day' non e' null
2  LocalDate hireDay = Objects.requireNonNullElse(day, LocalDate.of
    (1970, 1, 1));

```

Nel secondo caso, passiamo un'interfaccia `Supplier<T>`. Il codice della lambda verrà eseguito solo se necessario:

```

1  // LAZY: La lambda viene eseguita SOLO se 'day == null'
2  LocalDate hireDay = Objects.requireNonNullElseGet(day, () ->
    LocalDate.of(1970, 1, 1));

```

5.3.5 Method e Constructor References

I method references forniscono una sintassi ancora più concisa delle lambda expression quando queste si limitano a invocare un singolo metodo già esistente, senza effettuare operazioni aggiuntive. L'operatore principale è il doppio due punti (`::`). Si può usare un *method reference* solo se la firma del metodo esistente è **perfettamente compatibile** con i parametri dell'interfaccia funzionale attesa (Regola di Sostituzione); perciò non è nemmeno necessario specificare i parametri: vengono mappati automaticamente.

Questo tipo di sintassi ha tre forme:

Class::staticMethod Corrisponde a una lambda che passa i suoi argomenti a un metodo statico. Esempio:

```

1      //senza method reference:
2      x -> Math.sqrt(x);
3      //con method reference:
4      Math::sqrt;

```

object::instanceMethod Corrisponde a una lambda che invoca un metodo su un oggetto esistente. Esempio:

```

1      //senza method reference:
2      x -> System.out.println(x);
3      //con method reference:
4      System.out::println;

```


Class::instanceMethod Il primo parametro della lambda diventa il *target* del metodo e gli altri parametri vengono passati come argomenti. Esempio:

```

1      //senza method reference:
2      (s1, s2) -> s1.compareToIgnoreCase(s2);
3      //con method reference:
4      String::compareToIgnoreCase;
```

Esiste una variante speciale dei *method references* dedicata alla creazione di nuovi oggetti. La sintassi utilizza la parola chiave **new**.

- **Sintassi:** NomeClasse::new
- **Esempio:** ArrayList::new equivale alla lambda () -> new ArrayList<>().

Listing 5.7: Uso dei Constructor References

```

1  // Creazione di una lista di stringhe partendo da una lista di
   oggetti
2  List<String> nomi = stream.map(String::new).toList();
3
4  // Creazione di array (sintassi particolare)
5  int[] numeri = IntFunction<int[]>.apply(10); // lambda: n -> new
   int[n]
6  int[] numeriRef = int[]::new;
```

5.3.6 Accesso alle variabili (Variable Capture)

Le lambda expression possono accedere alle variabili locali del blocco di codice in cui sono definite. Tuttavia, tali variabili devono essere **effectively final**.

La regola dell'Effectively Final

Una variabile si dice *effectively final* se non viene mai modificata dopo la sua inizializzazione. Se provi a modificare una variabile locale all'interno di una lambda, o se la modifichi fuori dopo che la lambda l'ha "catturata", il codice non compilerà.

```

1  int counter = 0;
2  // ERRORE: non puoi modificare counter dentro la lambda
3  Runnable r = () -> { counter++; };
4
5  String prefix = "Log: ";
6  // OK: prefix non cambia mai, quindi e' effectively final
7  Consumer<String> printer = s -> System.out.println(prefix + s);
```

Perché questa restrizione? Il motivo principale è il meccanismo di funzionamento della variable capture, che sarà spiegato nel paragrafo 5.4.3 (ad essere precisi le lambdas funzionano in modo leggermente diverso da quel caso, ma la logica è identica).

C'è anche un altro motivo; sebbene i dettagli verranno approfonditi con lo studio della **concorrenza**, è bene anticipare che le lambda vengono spesso eseguite in momenti diversi o su **thread** diversi rispetto a dove sono state create.

Le variabili locali vivono nello **stack**; se la lambda potesse modificarle liberamente, si creerebbero gravi problemi di sincronizzazione e instabilità dei dati. Imponendo che

la variabile sia una "costante nel tempo", Java garantisce la sicurezza del thread (*thread-safety*).

Se è assolutamente necessario modificare un valore all'interno di una lambda (es. un contatore), non potendo usare un tipo primitivo locale, si deve spostare il dato dallo **stack** allo **heap**.

Listing 5.8: Aggirare il vincolo con un array

```
1 // counter[0] e' memorizzato nello heap, non nello stack locale
2 int[] counter = { 0 };
3 button.addActionListener(e -> counter[0]++);
```



Attenzione

Nota di cautela: L'uso di array di un solo elemento o di "wrapper" per aggirare l'impossibilità di modificare variabili locali è un *workaround* che va usato con **estrema cautela**. In ambienti multi-thread, questa pratica espone il codice a *race conditions* se non opportunamente sincronizzata.

5.4 Inner Classes

Una **Inner Class** è una classe definita all'interno di un'altra. La sua caratteristica distintiva è il legame con l'istanza della classe esterna (*Outer Class*).

Caratteristiche principali:

- **Accesso Privilegiato:** I metodi della classe interna possono accedere a tutti i campi della classe esterna, inclusi quelli dichiarati **private**.
- **Visibilità Controllata:** Può essere dichiarata **private**, diventando invisibile a tutte le altre classi del pacchetto (impossibile per le classi top-level).
- **Riferimento Implicito:** Ogni istanza di una classe interna mantiene un riferimento invisibile all'oggetto della classe esterna che l'ha generata.

Le Inner Classes esistono dalle prime versioni di Java e sono state ampiamente utilizzate per implementare il pattern callback prima dell'introduzione delle Lambda. Prima di Java 8, quando si voleva gestire un click di un bottone, si doveva scrivere un'intera Inner Class. Oggi basta una riga di Lambda.

Tuttavia, le Inner Classes restano superiori quando:

- si deve implementare un'interfaccia con molti metodi (le Lambda funzionano solo con le interfacce funzionali, che hanno un solo metodo astratto e *proprio*).
- la logica da implementare è complessa e richiede più di poche righe di codice.
- si ha bisogno di mantenere uno stato interno complesso (variabili di istanza) che non può essere gestito efficacemente con variabili locali "catturate" da una lambda.

5.4.1 Struttura e Accesso allo Stato

Spiegheremo il concetto con un esempio artificiale, per puri scopi didattici. Consideriamo un **BankAccount** (classe esterna) che affida il calcolo degli interessi a un **InterestProcessor** (classe interna). Quest'ultimo ha bisogno di modificare il saldo, che però deve rimanere segreto al resto del mondo.

Listing 5.9: Esempio di Inner Class per l'incapsulamento

```

1 public class BankAccount {
2     private double balance; // Campo privato della classe esterna
3     private String owner;
4
5     public BankAccount(String owner, double initialBalance) {
6         this.owner = owner;
7         this.balance = initialBalance;
8     }
9
10    // CLASSE INTERNA: Definita dentro BankAccount
11    private class InterestProcessor {
12        public void updateBalance(double rate) {
13            // NOTA: 'balance' non e' definito qui, ma appartiene
14                a BankAccount!
15            double interest = balance * (rate / 100);
16            balance += interest;
17            System.out.println("Interessi calcolati per " + owner
18                );
19        }
20    }
21
22    public void startInterestProcess(double rate) {
23        // La classe esterna crea un oggetto della interna
24        InterestProcessor processor = new InterestProcessor();
25        processor.updateBalance(rate);
26    }
27 }

```

`InterestProcessor` non ha un campo `balance`, eppure lo modifica. Come è possibile?

Il compilatore modifica implicitamente tutti i costruttori della classe interna per accettare un riferimento alla classe esterna, aggiungendo un campo nascosto che memorizza questo riferimento. Nel nostro esempio, il costruttore di `InterestProcessor` è un no-args-constructor di default che implicitamente diventa qualcosa del genere:

Listing 5.10: Costruttore implicito della classe interna

```

1 public InterestProcessor(BankAccount bankAccount) {
2     outer = bankAccount; /* N.B.: 'outer' non e' una parola
3     chiave di Java, ma il nome che abbiamo dato al campo
4     nascosto creato dal compilatore */
5 }

```

In questo modo, ogni istanza di `InterestProcessor` è automaticamente collegata a un'istanza specifica di `BankAccount`, permettendo l'accesso diretto ai suoi campi e metodi, anche quelli privati. Quando viene istanziato un oggetto della classe interna, il compilatore si assicura che venga passato un riferimento all'istanza della classe esterna che lo contiene, cioè l'oggetto "this" della classe esterna, quindi implicitamente avviene questo passaggio:

Listing 5.11: Creazione di un'istanza della classe interna

```

1 InterestProcessor processor = new InterestProcessor(this);
2 // 'this' si riferisce all'istanza di BankAccount

```

Osservazione 5.6. Abbiamo dichiarato la classe interna come privata, rendendola completamente invisibile al mondo esterno. In questo modo, nessun altro codice al di fuori di `BankAccount` può creare un'istanza di `InterestProcessor` o accedere direttamente alla sua logica. Questo è il caso d'uso più comune per le classi interne, ma parleremo comunque a breve anche del caso di classi interne pubbliche.

Abbiamo chiamato `outer` il riferimento nascosto alla classe esterna, ma in realtà la sintassi corretta per denotare il riferimento all'oggetto esterno è una sintassi "all'indietro": per i campi interni ad una classe uso `this.campo`, ma poiché ci si riferisce ad una classe definita all'esterno, la classe esterna va prefissata:

`NomeClasseEsterna.this`

Ad esempio, il riferimento a `balance` all'interno di `InterestProcessor` è in realtà `BankAccount.this.balance`, ma poiché non c'è ambiguità, si può omettere la parte `BankAccount.this` e scrivere semplicemente `balance`. Se invece ci fosse un campo `balance` anche nella classe interna, allora sarebbe necessario usare la sintassi completa per risolvere l'ambiguità (shadowing).

Viceversa, quando la classe esterna vuole riferirsi a quella interna, la sintassi è esattamente la stessa di quella di un suo campo pubblico (anche se la classe interna è privata) e si può eventualmente usare anche la forma con `this` (ma quasi sempre è ridondante); anche il costruttore si può pensare come un normale metodo:

Listing 5.12: Creazione di un'istanza della classe interna

```

1 InterestProcessor proc = new InterestProcessor();
2 // Oppure, per essere espliciti (ma ridondanti):
3 InterestProcessor proc = this.new InterestProcessor();

```

Infine, nel caso in cui la classe interna sia pubblica, dal punto di vista di un codice esterno alla classe esterna, la classe interna va pensata come un campo pubblico della classe esterna; in tal caso la classe interna diventa una classe a cui non è possibile accedere direttamente, ma solo attraverso un'istanza della classe esterna.

Listing 5.13: Riferimento alla classe interna da un contesto esterno

```

1 Outer esterna = new Outer();
2 Outer.Inner interna = esterna.new Inner();

```

5.4.2 Local Inner Classes

Si possono dichiarare classi interne anche dentro un metodo (o più in generale qualsiasi blocco di codice); queste sono chiamate **Local Inner Classes** (le classi interne "normali" sono dette invece **Member Inner Classes**). Le classi interne locali hanno le seguenti caratteristiche:

- **Nessun modificatore di accesso:** Non possono essere `public` o `private`, perché la loro visibilità è già limitata al blocco di codice in cui si trovano.
- **Invisibilità alla classe esterna:** hanno accesso a tutti i campi e metodi della classe esterna ma, a differenza delle member inner classes, la loro visibilità è limitata al blocco di codice in cui sono dichiarate.
- **Variable Capture:** Come le Lambda, possono accedere alle variabili locali del metodo in cui sono dichiarate, ma solo se sono **effectively final**.

Oggi le *Local Classes* sono rare perché le Lambda coprono quasi tutti i loro casi d'uso. Si usano ancora se si ha bisogno di:

1. Definire **più metodi** o un costruttore complesso (cosa impossibile per una Lambda).
2. Implementare **più interfacce** contemporaneamente.

5.4.3 Meccanica della Variable Capture

Il termine **Variable Capture** descrive il processo mediante il quale una classe locale (o anonima) accede alle variabili del blocco di codice in cui è definita. Il meccanismo **si aggiunge** a quello del passaggio del riferimento alla classe esterna (descritto in precedenza): il compilatore non solo passa il riferimento all'oggetto esterno (**this**), ma "inietta" nel costruttore anche una copia del valore di ogni variabile locale catturata.

Infatti, quando si scrive una classe locale, il compilatore Java fa un lavoro nascosto di "sartoria"; per ogni variabile locale che si "cattura":

- Crea un **campo privato e final** dentro la classe interna per memorizzare quel valore (o quel riferimento, se si tratta di un oggetto).
- Aggiunge un parametro al costruttore per ricevere tale dato.
- Nel momento in cui si invoca `new MiaClasseLocale()`, passa silenziosamente i valori correnti al costruttore.

Se il metodo potesse modificare la variabile dopo che è stata catturata, si avrebbe un problema di **incoerenza dei dati**: le variabili locali risiedono nello *stack*, dunque cessano di esistere al termine del metodo, mentre l'oggetto della classe interna risiede nello *heap* e potrebbe sopravvivere più a lungo. Poiché l'oggetto lavora su una **copia locale** (il campo interno generato dal compilatore), se l'originale nello stack cambiasse, l'oggetto non se ne accorgerebbe mai. Java impone il vincolo **effectively final** proprio per garantire che l'originale e la copia siano sempre identici.

Immaginiamo che Java ci permettesse di modificare le variabili catturate. Ecco il caos che ne deriverebbe:

Listing 5.14: Perché non possiamo modificare le variabili locali

```
1 public void createLogger() {
2     int level = 1; // Variabile locale
3
4     class Logger {
5         void log() {
6             // Se potessi leggere level, leggerei la COPIA fatta
7             // al momento del "new"
8             System.out.println("Livello: " + level);
9         }
10    }
11
12    Logger myLogger = new Logger(); // Qui viene fatta la "
13    // fotocopia" di level (1)
14
15    level = 2; // Supponiamo di poterlo cambiare...
16
17    myLogger.log(); // Cosa dovrebbe stampare? 1 o 2?
18 }
```

5.4.4 Classi Anonime

Usando una classe locale, se si vuole creare un solo oggetto di quella classe, si può evitare di darle un nome. In questo caso, si parla di **Classi Anonime**. La sintassi generale è la seguente:

Listing 5.15: Sintassi di una classe anonima

```
1 new SuperTipo(parametri_costruttore) {  
2     // campi e metodi della classe anonima  
3 };
```

`SuperTipo` può essere un'interfaccia o una classe:

- Se `SuperTipo` è un'interfaccia, la classe anonima la implementa (estendendo implicitamente `Object`). Le parentesi devono essere vuote, poiché si sta richiamando il costruttore senza argomenti di `Object`.
- Se `SuperTipo` è una classe, la classe anonima ne diventa una sottoclasse. Poiché la classe anonima non ha nome, non è possibile definirne un costruttore esplicito; i parametri passati a `new SuperTipo(...)` vengono quindi usati per invocare il costruttore della superclasse. Di conseguenza, se la classe madre non possiede un costruttore vuoto (no-args), è obbligatorio fornire i parametri necessari per la sua inizializzazione.

Tecnicamente, un'anonima definita in un metodo è una *local class* senza nome; tipicamente è proprio in questo contesto che vengono utilizzate. Tuttavia, è possibile dichiarare classi anonime anche a livello membro, o addirittura come argomenti di un metodo (anche se quest'ultimo caso è piuttosto raro). Vale anche la pena osservare che in ogni caso hanno le seguenti caratteristiche:

- **Cattura delle variabili:** Segue esattamente la stessa meccanica di "Variable Capture" descritta per le classi locali (necessità di variabili *effectively final*).
- **Limite di Estensione:** Una classe anonima può fare solo una cosa: o estendere una classe (una sola) o implementare un'interfaccia (una sola). Non può fare entrambe le cose contemporaneamente, né implementare più interfacce.

5.4.5 Classi Statiche

Talvolta si ha bisogno di una classe interna che non abbia un riferimento all'oggetto esterno; in questo caso, si può dichiarare la classe interna come **static**.

Uno dei motivi per dichiarare una classe interna come statica è perché la si vuole istanziare in un contesto statico, come ad esempio un metodo statico: come ben sappiamo, i metodi statici possono usare solo membri statici; le classi interne non fanno eccezione. Esempio:

Listing 5.16: Esempio di classe interna statica

```
1 public class Outer {  
2     private static class Inner {...}  
3  
4     public static void doSomething() {  
5         Inner inner = new Inner(); // OK: Inner e' statica  
6     }  
7 }
```

In questo esempio, se **Inner** non fosse statica, ne risulterebbe un errore in fase di compilazione sul tentativo di istanziare la classe interna: questa, come ormai ben sappiamo, contiene un riferimento implicito alla classe esterna, ma di quest'ultima non esiste ancora un'istanza. Dichiarando **Inner** come statica, si elimina il legame con l'istanza esterna, permettendo così la sua istanziazione anche in contesti statici.

In realtà, c'è un motivo molto più valido per dichiarare una classe interna come statica; ne parleremo nel prossimo paragrafo, perché si tratta di un argomento di per sé importante.

Osservazione 5.7. Le classi statiche non sono "vere" classi interne, ma piuttosto classi normali che vengono dichiarate all'interno di un'altra classe per motivi di organizzazione del codice. Non hanno accesso ai membri d'istanza della classe esterna e non mantengono alcun riferimento ad essa. Con modificatore di accesso pubblico, dichiararle in un file a parte e usarle come campo nella classe esterna sarebbe perfettamente equivalente, ma di solito si preferisce tenerle "sotto lo stesso tetto" per ragioni di coesione e incapsulamento logico.

In effetti, la distinzione ufficiale della documentazione Java è:

- **Nested Classes:** termine generico che include tutti i tipi di classe dichiarate dentro un'altra.
- **Inner Classes:** tutte le classi non-statiche dichiarate all'interno di un'altra classe.
- **Static Classes:** classi statiche, necessariamente "nested".

Osservazione 5.8. :

- Records, enumeratori e interfacce dichiarati all'interno di classi o interfacce, sono implicitamente **static**;
- classi dichiarate all'interno di interfacce sono implicitamente **static** e **public**.

5.4.6 Il Pericolo del Memory Leak nelle Inner Classes

Esiste un problema importante che nasce dal riferimento implicito di una classe interna (non statica) a quella esterna. Immaginiamo questo scenario:

- hai una classe esterna molto "pesante" (magari un `DatabaseManager` o una `Activity` con molte immagini).
- all'interno crei una `Inner Class` (non-statica) e ne generi un'istanza.
- passi questo piccolo oggetto a qualcuno che "vive" a lungo, ad esempio un servizio che gira in background o una lista statica.

Il disastro: Quando il `DatabaseManager` non serve più e dovrebbe essere eliminato dal Garbage Collector (GC), il GC vede che l'oggetto interno è ancora vivo. E poiché l'oggetto interno ha il riferimento invisibile `Outer.this`, il GC è costretto a tenere in memoria anche il `DatabaseManager`. In pratica, un piccolo oggetto di pochi byte tiene in ostaggio un oggetto enorme, impedendo a Java di liberare memoria. È come se non potessi buttare via un intero camion solo perché c'è un portachiavi attaccato al cruscotto che vuoi tenere.

Listing 5.17: Esempio di memory leak

```
1 public class Pesante {
2     private byte[] dati = new byte[1024 * 1024 * 10]; // 10 MB di
      dati
3
4     public void avviaTask() {
5         // Classe anonima (inner) passata a un thread che dura
      ore
    }
```

```

6      new Thread(new Runnable() {
7          @Override
8          public void run() {
9              // Fai qualcosa per 2 ore...
10             // Anche se l'oggetto 'Pesante' finisce il suo
              lavoro,
11             // resta in memoria finche' questo thread non
              muore!
12         }
13     }).start();
14 }
15 }

```

! Meccanica del Leak

Ogni istanza di una classe interna (non-statica) contiene un riferimento nascosto a quella esterna. Se l'istanza interna viene passata a un contesto a lungo termine (es. Thread, Static Collection, Singleton), l'intera istanza `OuterClass` rimane bloccata nello *Heap*.

💡 Strategie di prevenzione del leak

1. **Preferire Static Classes:** Se non e' richiesto l'accesso ai campi d'istanza della classe esterna, usare `static class`.
2. **Weak References:** In casi estremi, se si ha bisogno dell'accesso ma si vuole evitare il leak, si puo' usare un `WeakReference<OuterClass>` all'interno di una classe statica. (parleremo di questo argomento altrove)
3. **Pulizia dei Callback:** Assicurarsi di rimuovere listener o annullare task in background quando l'oggetto `Outer` viene distrutto.

⚠ Attenzione

Le **classi anonime** sono le più pericolose, poiché è facile dimenticare che esse mantengono un riferimento all'oggetto circostante.

5.5 Approfondimento: le Chiusure (Closures)

Sebbene il termine non compaia spesso nella documentazione ufficiale Java (che preferisce *Variable Capture*), in informatica si definisce **Closure** (o chiusura) un blocco di codice (come una Lambda o una Inner Class) insieme al **contesto** (lo scope) in cui è stato creato.

💡 Definizione Formale

Una **closure** è una funzione che "racchiude" (da cui il nome) le variabili libere del suo ambiente circostante, permettendo loro di sopravvivere anche dopo che il blocco di codice originale ha terminato l'esecuzione.

In Java, ogni volta che una Lambda Expression o una Local Inner Class accede a una

variabile definita all'esterno del suo corpo, stiamo creando una chiusura.

Le chiusure sono fondamentali nel backend moderno perché permettono di "personalizzare" un comportamento con dei dati specifici al momento della creazione, per poi eseguirlo in un secondo momento.

Listing 5.18: Esempio pratico di Closure

```
1 public class ChiusuraEsempio {
2     public static void main(String[] args) {
3         // Creiamo un'azione "personalizzata" con un dato locale
4         Runnable azione = creaSaluto("Studente Java");
5
6         // Eseguiamo l'azione: il metodo creaSaluto e' gia'
7         // terminato,
8         // ma la Lambda "ricorda" ancora il valore di 'nome'.
9         azione.run();
10    }
11
12    public static Runnable creaSaluto(String nome) {
13        String prefisso = "Benvenuto, "; // Variabile locale
14
15        // Questa e' una CLOSURE:
16        // Il codice della lambda + le variabili 'prefisso'
17        // e 'nome'
18        return () -> System.out.println(prefisso + nome);
19    }
20 }
```

È importante notare una distinzione fondamentale che spesso emerge nei colloqui tecnici:

- **In linguaggi come JavaScript o Python:** Le chiusure catturano il *referimento* alla variabile. Se la variabile cambia fuori, cambia anche dentro la chiusura.
- **In Java:** Le chiusure catturano il **valore** (una copia). Per evitare l'incoerenza tra l'originale e la copia (come spiegato nel paragrafo 5.4.3), Java impone che la variabile sia **effectively final**.



Se qualcuno ti chiede: "Java supporta le chiusure?", la risposta corretta è: *"Sì, Java supporta le chiusure tramite le Lambda e le Inner Classes, ma con il vincolo che le variabili catturate devono essere immutabili (effectively final) per garantire la thread-safety e la coerenza dello stack."*

Riepilogando:

1. **Lambda/Inner Class:** È il contenitore fisico del codice.
2. **Variable Capture:** È il processo tecnico (fatto dal compilatore) di copiare le variabili.
3. **Closure:** È il concetto teorico che descrive l'unione tra il codice e le variabili catturate.

Eccezioni, Asserzioni e Logging

6.1 Gestione delle Eccezioni

Il passaggio dalla programmazione puramente didattica a quella professionale coincide con la consapevolezza che il codice deve operare in un ambiente ostile (dati errati, bug, risorse esterne indisponibili).

In presenza di un'anomalia, il software deve garantire tre requisiti minimi:

1. **Notifica:** Informare l'utilizzatore del problema occorso.
2. **Persistenza:** Salvare lo stato corrente per evitare la perdita di dati.
3. **Terminazione Controllata:** Consentire una chiusura del programma che rilasci correttamente le risorse impegnate.

Storicamente, i metodi segnalavano il fallimento tramite valori sentinella (`null`, `-1`, `false`). Questo approccio è oggi considerato **pericoloso** e **poco informativo**.

In Java, un'anomalia durante l'esecuzione (runtime) non provoca necessariamente il crash immediato del sistema operativo, ma attiva un meccanismo strutturato chiamato **Gestione delle Eccezioni** che si basa sull'uso di classi preposte.

Illustriamo questo meccanismo in un caso specifico. Supponiamo che il processore incontri l'istruzione `int x = 10 / 0;` la Java Virtual Machine interrompe il flusso lineare e segue i seguenti passaggi:

1. **Creazione dell'Oggetto Eccezione:** Non appena si verifica l'errore, la JVM crea istantaneamente un oggetto di una classe specifica. Nel caso della divisione per zero, viene istanziata la classe:

```
java.lang.ArithmeticException
```

Questo oggetto contiene:

- Il **tipo** di eccezione.
 - Il **messaggio di errore** (es. `"/ by zero"`).
 - Lo **Stack Trace**: una fotografia dei metodi che erano in esecuzione al momento del guasto.
2. **Lancio (Throwing):** Il metodo in cui si è verificato l'errore "lancia" (*throws*) l'oggetto eccezione al sistema di runtime. L'esecuzione del codice normale nel metodo corrente viene immediatamente sospesa.
 3. **Ricerca del Gestore (Catching):** Il sistema di runtime cerca nel **Call Stack** (lo stack delle chiamate ai metodi) un blocco di codice in grado di gestire l'eccezione.
 - (a) Il sistema controlla se il punto dell'errore è gestito da un apposito blocco di codice (`try`).
 - (b) In caso positivo, verifica se esiste un blocco `catch` associato che sia compatibile con il tipo di eccezione lanciata; se lo trova, l'esecuzione continua: viene eseguito il blocco `catch`.
 - (c) Se non viene trovato nel metodo corrente, il sistema passa al metodo "chiamante" e così via, risalendo la gerarchia.

4. **Il Default Exception Handler:** Se la ricerca arriva al metodo `main` senza aver trovato alcun gestore, l'eccezione viene catturata dal **Default Exception Handler**. Questo esegue due azioni finali:

- Stampa a video lo Stack Trace (il testo rosso in console).
- Termina forzatamente il thread in cui si è verificata l'eccezione.

Ecco un confronto tra un codice non gestito (che causa il crash) e uno gestito correttamente. Nella seconda parte è anche illustrata la sintassi del `try-catch` in una forma elementare.

Listing 6.1: Esempio di crash

```
1 public class Test {
2     public static void main(String[] args) {
3         int a = 10;
4         int b = 0;
5         int result = a / b; // Il programma termina qui
6         System.out.println("Risultato: " + result);
7     }
8 }
```

Listing 6.2: Gestione dell'anomalia e sintassi base del `try-catch`

```
1 public class Test {
2     public static void main(String[] args) {
3         try {
4             int result = 10 / 0;
5         } catch (ArithmeticException e) {
6             System.err.println("Errore: Impossibile dividere per
7                                 zero!");
8         }
9         System.out.println("Il programma continua l'esecuzione...
10                            ");
11     }
12 }
```

La gestione delle eccezioni trasforma dunque un errore fatale in un evento che può essere previsto e mitigato, permettendo al software di rimanere robusto e di fornire feedback utili all'utente o allo sviluppatore tramite lo Stack Trace.

Confrontiamo l'evoluzione delle API Java per capire il vantaggio delle eccezioni:

- **Vecchio Stile (Legacy):** `file.delete()` restituisce `false` se fallisce.
 - *Problema:* Perché e' fallito? Permessi negati? File inesistente? Disco protetto? Non lo sapremo mai.
- **Nuovo Stile (NIO):** `Files.delete(path)` lancia un'eccezione, ad esempio:

`NoSuchFileException`, `AccessDeniedException`, ecc...

- *Vantaggio:* L'errore è auto-esplicativo e costringe il programmatore a decidere come gestirlo.

6.1.1 La Gerarchia delle Eccezioni

In Java, ogni oggetto eccezione è un'istanza di una classe derivata da `Throwable`; la gerarchia si divide immediatamente in due rami: `Error` ed `Exception`.

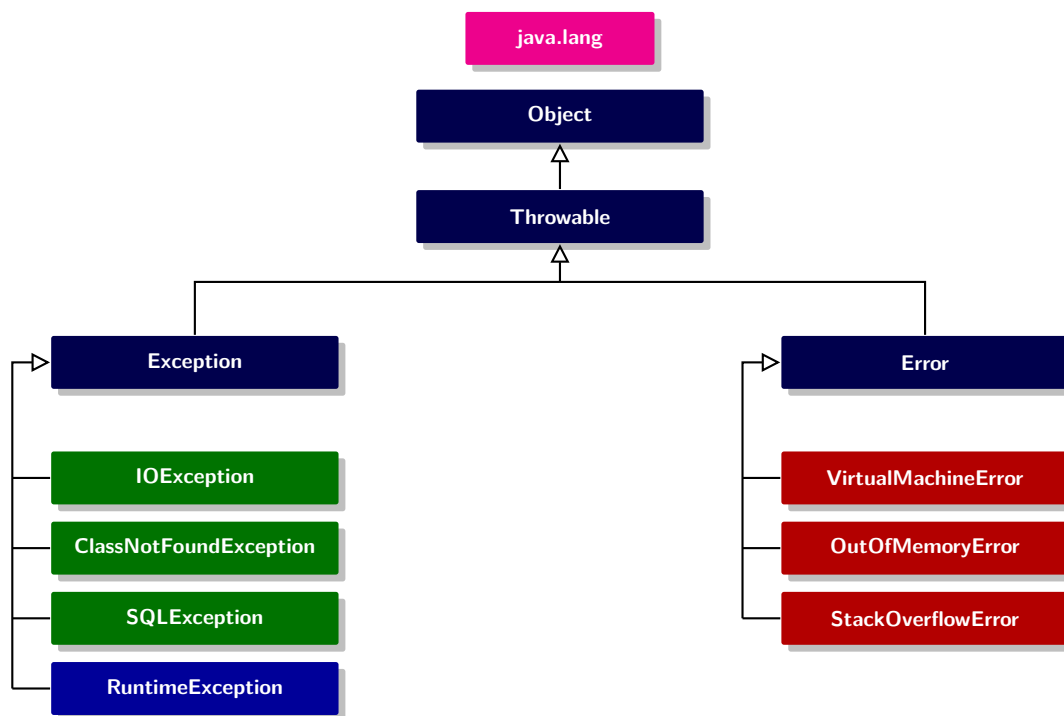


Figura 6.1: Gerarchia delle eccezioni in Java

È fondamentale capire che in Java la differenza tra le eccezioni non è solo gerarchica, ma riguarda soprattutto il modo in cui il compilatore impone di gestirle. La distinzione principale è tra *checked* e *unchecked*:

- **Checked:** il compilatore obbliga il programmatore a gestirle (vedremo a breve in dettaglio cosa questo voglia dire). In genere rappresentano problemi esterni o situazioni recuperabili, come errori di I/O, rete o database.
- **Unchecked:** il compilatore non obbliga a gestirle né a dichiararle. Sono spesso dovute a errori logici del programmatore (ad esempio `NullPointerException`, indici fuori range, argomenti non validi), oppure guasti gravi della JVM o del sistema.

Tutti i `Throwable` che derivano da `Error` o da `RuntimeException` sono *unchecked*, tutti gli altri sono *checked exceptions*.

Possiamo liquidare rapidamente il ramo della gerarchia relativa agli `Error`: si tratta di errori talmente critici (come errori interni e situazioni di esaurimento delle risorse all'interno del sistema di runtime Java) che non si dovrebbe mai tentare di gestire. In questi casi, il programmatore ha poche possibilità di intervento: l'unica azione sensata è solitamente informare l'utente e terminare il programma in modo consono. Fortunatamente, queste situazioni sono abbastanza rare nella pratica.

Da questo momento in poi ci occuperemo quindi del ramo `Exception` della gerarchia, ovvero della classe dei problemi gestibili.

6.1.2 Dichiarare e Lanciare Eccezioni

Un metodo Java può terminare in due modi: restituendo un valore o "esplodendo" con un'eccezione. Se un metodo non è in grado di gestire una situazione anomala, deve dichiarare questa possibilità al mondo esterno. La sintassi per dichiarare e *lanciare* un'eccezione è illustrata nel seguente esempio elementare:

Listing 6.3: Esempio di metodo che lancia un'eccezione

```
1 public void setAge(int age) throws IllegalArgumentException {  
2     if (age < 0) {  
3         // Creiamo l'oggetto eccezione e lo lanciamo  
4         throw new IllegalArgumentException("age cannot be  
5             negative: " + age);  
6     }  
7     this.age = age;  
}
```

È fondamentale non confondere queste due parole chiave, simili ma con scopi diversi:

- **throws**: Si usa nella **firma del metodo** per avvertire il compilatore e il chiamante che quel metodo può lanciare determinate eccezioni.
- **throw**: Si usa nel **corpo del metodo** per lanciare effettivamente l'oggetto eccezione.

Quando viene eseguita l'istruzione **throw**:

1. L'esecuzione del metodo si interrompe **immediatamente**.
2. Le istruzioni successive (come l'assegnamento `this.age = age`) non vengono eseguite.
3. Il controllo passa al chiamante, cercando il primo blocco **try-catch** disponibile che sappia gestire `IllegalArgumentException`.

💡 Quando l'eccezione lanciata è *checked*, è **obbligatorio** dichiararla nella testata del metodo. In questo esempio abbiamo invece usato `IllegalArgumentException`, che è una **Unchecked Exception**, quindi la clausola **throws** nella firma del metodo è in tal caso opzionale.

In generale, intercettare eccezioni *unchecked* (e ancor più gli **Error**) non dovrebbe essere la strategia ordinaria: spesso tali eccezioni segnalano errori di programmazione o uso scorretto delle API, che andrebbero prevenuti scrivendo codice robusto e validando correttamente gli input. Tuttavia, in alcuni contesti può essere sensato intercettarle per effettuare logging, traduzione dell'errore o gestione centralizzata. Horstmann riassume efficacemente l'idea con una frase suggestiva: *"se si verifica un'eccezione unchecked, spesso il problema è nel codice, non in un evento imprevedibile"*.

Osservazione 6.1. Ciò che diremo per i metodi varrà anche per i costruttori. Un metodo o un costruttore può lanciare più di un tipo di eccezione; in questo caso basta elencarle dopo lo specificatore **throws** separate da virgole (se necessario).

6.1.3 Il Blocco **try-catch**: gestione locale

Come abbiamo visto, se si verifica un'eccezione che non viene catturata in alcun luogo, il programma termina e stampa un messaggio sulla console (lo *stack trace*). Per evitare

questo e permettere al programma di continuare o chiudersi in maniera controllata, usiamo il blocco `try-catch`.

La struttura base consiste nel racchiudere il codice "pericoloso" in un blocco `try` e definire la gestione del problema in uno o più blocchi `catch`.

Listing 6.4: Sintassi base try-catch-finally

```
1 try {  
2     // 1. Codice che potrebbe lanciare un'eccezione  
3     // Se un'eccezione esplode qui, il resto del blocco viene  
4     // saltato  
5 } catch (TipoEccezione1 e) {  
6     // 2. Codice eseguito solo se si verifica TipoEccezione1  
7     // 'e' e' l'oggetto che contiene i dettagli dell'errore  
8 } catch (TipoEccezione2 e) {  
9     // 3. Si possono avere piu blocchi catch per diversi errori  
10 } finally {  
11     // 4. (Opzionale) Codice eseguito SEMPRE,  
12     // sia che ci sia stata un'eccezione, sia che non ci sia  
13     // stata.  
14 }
```

Il flusso di esecuzione è il seguente:

1. **Se non ci sono errori:** Viene eseguito tutto il blocco `try`, i blocchi `catch` vengono saltati, viene eseguito il `finally` (se presente) e si continua con l'esecuzione del metodo.
2. **Se si verifica un errore:**
 - L'esecuzione nel `try` si ferma alla riga che ha generato l'eccezione.
 - La JVM cerca un blocco `catch` compatibile con il tipo di eccezione lanciata.
 - Se lo trova, esegue il codice nel `catch`, passa al `finally` (se presente) e continua con l'esecuzione del metodo.
 - Se NON lo trova, viene eseguito l'eventuale `finally`, il metodo termina e l'eccezione "risale" verso il chiamante.

⚠ Attenzione

L'ordine dei blocchi `catch` è fondamentale. Java esegue il primo `catch` compatibile che trova. Pertanto, si devono sempre mettere le eccezioni ****più specifiche**** (es. `FileNotFoundException`) prima di quelle ****più generiche**** (es. `IOException` o `Exception`). Altrimenti, la più specifica non verrà mai raggiunta.

A partire da Java 7, se si devono gestire diverse eccezioni nello stesso identico modo, non è più necessario scrivere blocchi `catch` duplicati: basta usare la sintassi *multi-catch* separando le classi con l'operatore *pipe* (`|`). Oltre a rendere il codice più leggibile, questa pratica è più efficiente: il compilatore genera un unico blocco di *bytecode* per le eccezioni condivise, riducendo la dimensione del file compilato.

Listing 6.5: Esempio di multi-catch

```
1 catch (FileNotFoundException | UnknownHostException e) {  
2     // Gestione comune per entrambi i problemi  
3     logger.error("Errore di connessione o file: " + e.getMessage()  
4         ());
```

4 }

⚠ Attenzione

Quando si usa il multi-catch con l'operatore `|`, la variabile dell'eccezione (`e`) è implicitamente **final**: non si può riassegnarla all'interno del blocco.

Un altro dettaglio tecnico importante è che il tipo di `e` nel multi-catch non è un "tipo unione" (anche quando l'IDE ci dice che è così). Il compilatore assegna a `e` il tipo della **superclasse comune più vicina** (*LUD: least upper bound*) tra quelle elencate. In genere questo non è un problema ma, in caso contrario, probabilmente è meglio tornare a usare blocchi `catch` separati. Il codice risulterà molto più pulito e meno propenso a errori.

Il blocco `finally` viene utilizzato quasi esclusivamente per la pulizia delle risorse (chiudere file, connessioni database, socket). La sua caratteristica unica è che viene eseguito **anche se** nel blocco `try` o `catch` era presente un'istruzione `return`.

L'unica situazione in cui `finally` potrebbe non essere eseguito è se la JVM si arresta bruscamente (es. `System.exit(0)`) o se si verifica un **Error** catastrofico dell'hardware.

⚠ Attenzione

Sebbene il `finally` sia fondamentale per la chiusura delle risorse, bisogna stare attenti a un errore comune: inserire istruzioni che alterano il flusso di controllo (come `return` o `throw`) all'interno del `finally`. Se un blocco `finally` contiene un `return`, questo "sovrascriverà" qualsiasi valore di ritorno o eccezione lanciata nei blocchi `try` o `catch`. Analogamente per i `throw`.

6.1.4 La clausola `throws`: gestione delegata

Non sempre gestire un'eccezione localmente con un `try-catch` è la scelta migliore. Spesso, il metodo che rileva l'errore non sa come risolverlo (ad esempio, un metodo di basso livello che legge un file non sa se deve mostrare un messaggio di errore all'utente o riprovare). In questi casi, è preferibile **propagare** l'eccezione al metodo chiamante.

Per delegare la gestione, si omette il blocco `try-catch` e si aggiunge la clausola `throws` nella firma (testata) del metodo. Questo avverte il compilatore che chiunque chiamerà questo metodo dovrà assumersi la responsabilità di gestire l'eventuale "esplosione".

Listing 6.6: Esempio di delega dell'eccezione

```

1 // Il metodo non gestisce l'errore, ma "avvisa" il chiamante
2 public void leggiFileConfigurazione(String percorso) throws
   FileNotFoundException {
3     // Se il file non esiste, il costruttore lancia
       FileNotFoundException
4     FileInputStream fis = new FileInputStream(percorso);
5     // ... logica di lettura ...
6 }
```

Questo tipo di delega segue le seguenti regole:

- **Per le Checked Exceptions:** La delega tramite `throws` è **obbligatoria** se non si usa un `try-catch`. Se dimentichi di dichiararla, il codice non compila.
- **Per le Unchecked Exceptions:** La delega è **facoltativa**.
- **Passaggio di consegne:** Se il metodo A chiama il metodo B (che lancia un'eccezione), il metodo A ha di nuovo le stesse due opzioni: catturarla con un `try-catch` o dichiararla a sua volta con `throws` (di nuovo: obbligatorio in caso di eccezione *checked*, opzionale altrimenti), passandola al metodo superiore.

✓ Best Practice

Le eccezioni dovrebbero essere intercettate nel livello più basso che sia effettivamente in grado di gestirle in modo corretto; in caso contrario, è preferibile propagarle verso livelli superiori che dispongano del contesto necessario per decidere come reagire.

Sebbene ormai il meccanismo di gestione locale o delega dovrebbe essere chiaro anche nel caso particolare delle *checked exceptions*, evidenziamo la seguente regola del **Catch or Declare**:

💡 Regola del Catch-or-Declare

Se un metodo X ne invoca un altro che lancia una checked exception, il codice **non compila** finché non si soddisfa una di queste due condizioni:

1. **Catch:** Si intercetta l'eccezione con un blocco `try-catch` per gestirla.
2. **Declare:** Si aggiunge la clausola `throws` alla firma del metodo X per delegare la responsabilità a chiunque lo invochi.

6.1.5 Ereditarietà ed Eccezioni

Se un metodo della sottoclasse sovrascrive un metodo della superclasse, valgono le seguenti regole:

- **Non può** dichiarare eccezioni checked più generiche di quelle previste nel metodo originale.
- **Non può** aggiungere nuove eccezioni checked incompatibili con la testata del metodo ereditato.
- **Può** dichiarare eccezioni più specifiche (sottoclassi di quelle originali).
- **Può** decidere di non dichiarare alcuna eccezione.

In particolare, se il metodo della superclasse non dichiara alcuna eccezione checked, allora il metodo sovrascritto nella sottoclasse **non può propagarne alcuna che sia checked**. Può comunque usare codice che genera checked exceptions, ma in tal caso deve gestirle localmente mediante `try-catch`.

Queste restrizioni riguardano solo le *checked exceptions*. Le *unchecked exceptions* possono invece essere lanciate liberamente anche se non vengono lanciate nel metodo della superclasse.

✓ Best Practice

Quando progetti una gerarchia di classi, valuta con attenzione quali eccezioni

- ✓ inserire nella clausola `throws` della superclasse. Una firma troppo restrittiva può rendere più difficile l'implementazione delle sottoclassi che interagiscono con risorse esterne o operazioni fallibili.

6.1.6 Creare Classi di Eccezione Personalizzate

Sebbene Java offra una vasta gamma di classi di eccezione, potresti imbatterti in problemi che non sono descritti adeguatamente da quelle standard. In questi casi, basta creare un'eccezione personalizzata: è sufficiente estendere la classe `Exception` (per creare una *checked exception*) o una sua sottoclasse (come `IOException`). Se invece si desidera una *unchecked exception*, si deve estendere `RuntimeException`.

Una volta definita, la tua eccezione è pronta per essere lanciata all'interno dei tuoi metodi esattamente come un'eccezione di sistema.

Listing 6.7: Lancio di un'eccezione personalizzata

```
1 public String leggiDati(Scanner in) throws FileFormatException {
2     // ... logica di lettura ...
3     while (...) {
4         if (ch == -1) { // Fine del file in attesa
5             if (n < len)
6                 throw new FileFormatException("Il file termina
7                     inaspettatamente");
8             }
9             // ...
10        }
11        return s;
12    }
```

Tip

Quando estendi `Exception`, stai creando una **Checked Exception**: il compilatore obbligherà chiunque usi il tuo metodo a gestirla o a dichiararla. Se vuoi che la tua eccezione sia più "leggera" e non obbligatoria da gestire, estendi `RuntimeException`.

6.1.7 Exceptions Chaining

A volte, all'interno di un blocco `catch`, potresti voler lanciare una nuova eccezione. Questo si fa solitamente per "astrarre" il problema: chi usa il tuo codice non deve conoscere i dettagli tecnici (come un errore del database), ma deve sapere che il tuo sottosistema ha avuto un problema.

Invece di lanciare una nuova eccezione perdendo traccia di quella originale, la pratica migliore è **incatenarle (chaining)**. In questo modo, l'eccezione originale viene salvata come "causa" della nuova.

Listing 6.8: Esempio di Exception Chaining

```
1 try {
2     // Tentativo di prelievo dal database
3     eseguiQueryPrelievo();
4 } catch (SQLException originale) {
```



```

5 // Creiamo una nuova eccezione di alto livello
6 // Passiamo l'eccezione originale come "causa" (secondo
   parametro)
7 var e = new BankServiceException("Errore durante il prelievo"
   , originale);
8 throw e;
9 }

```

Tip

Quasi tutte le classi di eccezione standard hanno un costruttore che accetta la "causa". Se una classe non lo prevede, puoi usare il metodo `e.initCause(originale)` prima di lanciarla. Chi riceve l'eccezione potrà poi risalire all'errore iniziale con `e.getCause()`.

Un altro uso comune di questa tecnica è quando ti trovi in un metodo che **non può** lanciare checked exceptions (ad esempio a causa di un override), ma il codice al suo interno ne genera una.

Listing 6.9: Da Checked a Unchecked

```

1 public void stampaDocumento() { // Non dichiara throws
2     try {
3         generaPDF(); // Questo metodo lancia IOException (checked
   )
4     } catch (IOException e) {
5         // La impacchettiamo in una RuntimeException (unchecked)
6         // Così il compilatore è soddisfatto e non perdiamo l'
   errore
7         throw new RuntimeException("Fallimento critico durante la
   stampa", e);
8     }
9 }

```

Best Practice

L'incatenamento è caldamente raccomandato. Permette di mantenere un'interfaccia pulita verso l'alto (eccezioni di alto livello) senza sacrificare i dettagli tecnici necessari per il debugging (lo *stack trace* mostrerà tutta la catena delle cause).

6.1.8 try-with-resources

La chiusura delle risorse (file, connessioni, socket) è una delle operazioni più critiche. Se una risorsa non viene chiusa, si rischia un *resource leak* che può bloccare il sistema.

È possibile usare il blocco `finally` anche senza il `catch`. Questo serve a garantire che una risorsa venga chiusa indipendentemente dal fatto che si verifichi un'eccezione o meno.

Listing 6.10: Utilizzo di try-finally per la chiusura risorse

```

1 InputStream in = ...;
2 try {

```

```
3 // codice che potrebbe lanciare eccezioni
4 } finally {
5     in.close(); // Eseguito sempre
6 }
```

Per una gestione perfetta, è preferibile separare le responsabilità usando blocchi `try` annidati: uno interno per garantire la chiusura e uno esterno per gestire/segnalare gli errori.

Listing 6.11: un tipico pattern di `try` annidati

```
1 InputStream in = . . . ;
2 try
3 {
4     try
5     {
6         code that might throw exceptions
7     }
8     finally
9     {
10        in.close();
11    }
12 }
13 catch (IOException e)
14 {
15     show error message
16 }
```

Per evitare la complessità dei blocchi annidati, esiste il `try-with-resources`. Questa sintassi può essere usata con qualsiasi classe che implementi l'interfaccia **AutoCloseable** (o la sua sottointerfaccia **Closeable**); essa ha un unico metodo `void close()` che può lanciare una `IOException`.

Listing 6.12: Sintassi del `try-with-resources`

```
1 // La risorsa viene dichiarata tra le parentesi del try
2 try (var in = new Scanner(Path.of("input.txt"))) {
3     while (in.hasNext())
4         System.out.println(in.next());
5 } // in.close() viene chiamato AUTOMATICAMENTE qui
```

Tip

Puoi gestire più risorse contemporaneamente separandole con un punto e virgola: `try (var in = ...; var out = ...) { ... }`. Verranno chiuse tutte in ordine inverso rispetto alla dichiarazione.

Tip

Usa sempre il `try-with-resources` quando lavori con oggetti che richiedono una chiusura. Scrivere manualmente la logica di chiusura con i `finally` annidati è complicato e porta spesso a bug silenziosi.

Un blocco `try-with-resources` può avere i propri blocchi `catch` e `finally`. L'ordine di esecuzione è:

1. Viene eseguito il corpo del `try`.
2. Vengono chiuse le risorse dichiarate nell'intestazione.
3. Vengono eseguiti eventuali blocchi `catch` e il blocco `finally`.

Questo significa che i `catch` possono catturare anche le eccezioni avvenute durante la chiusura automatica delle risorse.

Eccezioni Soppresse

Uno dei vantaggi principali del `try-with-resources` è come gestisce il conflitto tra eccezioni. Come abbiamo visto, nel `finally` classico un'eccezione durante la chiusura "nasconde" quella principale.

Anche quando usiamo il `try-with-resources`, può capitare un "conflitto di eccezioni": un errore durante l'esecuzione del codice (*primary exception*) e uno o più errori durante la chiusura automatica delle risorse (*suppressed exceptions*). Il costrutto però gestisce in modo automatico il conflitto.

Un'eccezione si dice **soppressa** quando viene catturata silenziosamente dalla JVM e allegata all'eccezione principale invece di essere lanciata autonomamente. Questo evita che l'errore di chiusura "mascheri" l'errore originale che ha causato il fallimento del programma.

Con il `try-with-resources`, se sia il blocco `try` che il metodo `close()` lanciano un'eccezione:

1. L'eccezione originale del `try` viene lanciata regolarmente.
2. Le eccezioni lanciate dal metodo `close()` vengono "soppresse" e aggiunte a quella principale.
3. È possibile recuperarle in un secondo momento usando il metodo `e.getSuppressed()`.

Listing 6.13: Recupero delle eccezioni soppresse

```
1 try (var out = new PrintWriter("file.txt")) {
2     throw new IllegalStateException("Errore nel corpo del try");
3 } catch (IllegalStateException e) {
4     System.err.println("Errore primario: " + e.getMessage());
5
6     // Recuperiamo l'array delle eccezioni "nascoste"
7     Throwable[] suppressed = e.getSuppressed();
8     for (Throwable t : suppressed) {
9         System.err.println("Eccezione soppressa: " + t.getMessage
10             ());
11     }
```

Tip

Non devi preoccuparti di gestire manualmente le eccezioni soppresse nel 99% dei casi. La cosa fondamentale è che, guardando lo *stack trace* nella console, Java te le mostrerà automaticamente sotto la dicitura **Suppressed:** ..., permettendoti di vedere l'intera catena del disastro.

6.1.9 Costruttori e Metodi di Eccezioni

Ogni volta che scrivi `catch (Exception e)`, la variabile `e` è un riferimento a un oggetto. Poiché le eccezioni ereditano da `java.lang.Throwable`, ereditano un set di metodi preziosi per la diagnostica.

- **`String getMessage()`**: Restituisce la stringa descrittiva passata al costruttore. È il "cosa è successo" in formato leggibile.
- **`Throwable getCause()`**: Restituisce l'eccezione interna che ha scatenato quella attuale (fondamentale per l'*incatenamento* visto nei paragrafi precedenti).
- **`StackTraceElement[] getStackTrace()`**: Permette di accedere programmaticamente alla "pila" delle chiamate. Ogni elemento contiene il nome del file, la classe, il metodo e la riga di codice.
- **`void printStackTrace()`**: Stampa l'intera traccia dell'errore sullo standard error.
- **`void addSuppressed(Throwable t)`**: Aggiunge un'eccezione alla lista delle sopresse (usato automaticamente dal `try-with-resources`).

Per quanto riguarda i costruttori, creando una tua eccezione, dovresti quasi sempre implementare questi quattro per garantire massima flessibilità:

Listing 6.14: I 4 costruttori "canonici" di un'eccezione

```

1 public class MiaException extends Exception {
2     // 1. Vuoto: per errori generici senza dettagli
3     public MiaException() { super(); }
4
5     // 2. Con messaggio: per spiegare cosa e' andato storto
6     public MiaException(String message) { super(message); }
7
8     // 3. Con causa: per "impacchettare" un altro errore
9     public MiaException(Throwable cause) { super(cause); }
10
11    // 4. Con messaggio e causa:
12    public MiaException(String message, Throwable cause) {
13        super(message, cause);
14    }
15 }
```

Tip

Evita di usare `e.printStackTrace()` nel codice di produzione backend. Questo metodo scrive direttamente sulla console del server, che spesso viene ignorata o sovrascritta. Usa sempre un logger (`logger.log(Level.SEVERE, "msg", e)`) che invece salva lo stack trace in modo ordinato su file.

6.2 Le Asserzioni (Assertions)

Le asserzioni servono a verificare presupposti che il programmatore ritiene debbano essere sempre veri (*invarianti*) in un determinato punto del codice.

Java mette a disposizione la parola chiave `assert` con due forme:

1. `assert condizione;`
2. `assert condizione : espressione;`

Se la condizione è `false`, viene lanciato un `AssertionError`. Nella seconda forma, l'espressione viene convertita in stringa e usata come messaggio di errore.

Listing 6.15: Esempio di utilizzo delle asserzioni

```
1 double y = Math.sqrt(x);
2 // Sono certo che x non sia negativo per logica del programma
3 assert x >= 0 : "x era negativo: " + x;
```



L'unico scopo dell'espressione dopo i due punti è fornire dettagli per il debugging. Non cercare di usarla per "recuperare" l'errore: le asserzioni servono a far fallire il programma subito se qualcosa di "impossibile" accade. È un'applicazione del principio di *Fail Fast*: è molto meglio che un programma vada in crash subito durante i test, piuttosto che continui a girare con dati corrotti, rischiando di salvare informazioni sbagliate nel database o generare bug "silenziosi" difficilissimi da scovare mesi dopo.

Perché usare `assert` invece di `if-throw`? A differenza delle normali istruzioni di controllo, le asserzioni hanno un vantaggio prestazionale unico:

- **In Sviluppo:** Vengono abilitate per trovare bug logici.
- **In Produzione:** Vengono disabilitate di default. Il bytecode relativo alle asserzioni non viene nemmeno eseguito, garantendo che il programma sia il più veloce possibile.

6.2.1 Attivare/Disattivare Asserzioni

Le asserzioni sono dunque disabilitate di default; per abilitarle, bisogna inserire i parametri `-enableassertions` (oppure la forma abbreviata `-ea`) :

```
1 java -enableassertions MyApp
```

Java permette di filtrare l'attivazione delle asserzioni per singola classe o per pacchetto:

- **Singola Classe:** `-ea:NomeClasse` attiva le asserzioni solo per quella classe.
- **Pacchetto e Sottopacchetti:** `-ea:com.azienda.libreria...` attiva le asserzioni per tutte le classi del pacchetto specificato e di tutti i suoi sottopacchetti.
- **Unnamed Package:** L'opzione `-ea:...` attiva le asserzioni per tutte le classi che non appartengono a un pacchetto dichiarato (il cosiddetto *unnamed package*).

Allo stesso modo, è possibile disabilitare le asserzioni in punti specifici del codice utilizzando il flag `-disableassertions` (o `-da`). Questo è particolarmente utile quando si vuole monitorare tutto il progetto tranne una classe o un modulo specifico che sappiamo essere già stabile o che genera troppi errori di asserzione irrilevanti.

Listing 6.16: Esempio di abilitazione specifica

```
1 # 1. Abilitare le asserzioni per una SINGOLA CLASSE
2 java -ea:com.azienda.progetto.ValidatorDati MyApp
3
4 # 2. Abilitare le asserzioni per un SINGOLO PACKAGE (e i suoi
   sottopacchetti)
5 # Nota: i tre punti (...) sono fondamentali
6 java -ea:com.azienda.progetto.servizi... MyApp
```

```
7
8 # 3. Abilitare in un PACKAGE, ma disabilitarle in una CLASSE
   specifica
9 # Utile se quella classe e' vecchia (legacy) e lancia errori che
   vuoi ignorare
10 java -ea:com.azienda.progetto.logica... -da:com.azienda.progetto.
   logica.VecchioAlgoritmo MyApp
11
12 # 4. Abilitare in un PACKAGE, ma disabilitarle in un suo
   SOTTOPACKAGE
13 # Utile per testare la parte stabile e ignorare quella
   sperimentale/instabile
14 java -ea:com.azienda.core... -da:com.azienda.core.sperimentale...
   MyApp
15
16 # Attiva le asserzioni nell'unnamed package, ma disabilitale in
   MyClass
17 java -ea:... -da:MyClass MyApp
```

È possibile combinare più flag. In caso di conflitto, la regola è: **il filtro più specifico vince**.

Listing 6.17: Esempio di configurazione mista

```
1 # Abilita tutto il progetto, ma tieni spente le asserzioni nel
   pacchetto "legacy"
2 java -ea -da:com.vecchio.codice... MyApp
3
4 # Abilita solo una classe specifica mentre tutto il resto e'
   spento
5 java -da -ea:com.mio.AlgoritmoCritico MyApp
```

⚠ Attenzione

Le classi di sistema (come quelle in `java.util.*`) sono caricate dal *Bootstrap Class Loader* e ignorano i flag `-ea` e `-da`. Se sospetti un bug interno al JDK e vuoi attivare le sue asserzioni, devi usare obbligatoriamente `-esa`.

Osservazione 6.2. Sebbene sia possibile attivare le asserzioni da riga di comando, nella pratica quotidiana si utilizzano le impostazioni dell'IDE. E' fondamentale inserire il flag `-ea` nel campo delle **VM Options** (parametri per la Virtual Machine) e non tra i *Program Arguments*, altrimenti il comando verrebbe ignorato.

Le asserzioni non richiedono la ricompilazione del codice per essere attivate o disattivate; questa decisione viene presa dal **Class Loader** (il sottosistema della JVM che cerca i file `.class` sul disco, ne verifica l'integrità e li carica nella memoria della JVM.) al momento dell'esecuzione. Quando sono disattivate, il Class Loader ignora tutte le asserzioni, che quindi non pesano sulle prestazioni.

⚠ Effetti Collaterali

La condizione di un'asserzione non deve **mai** alterare lo stato del programma. Poiché il codice dentro l'`assert` potrebbe non essere eseguito in produzione, inserire operazioni logiche (come eliminare un file o incrementare un contatore) porterebbe a bug diversi tra l'ambiente di test e quello di produzione.

Listing 6.18: Esempio di Side-Effect da evitare

```

1 // SBAGLIATO: l'elemento viene rimosso solo se le asserzioni sono
  attive
2 assert lista.remove(oggetto);
3
4 // CORRETTO:
5 boolean rimosso = lista.remove(oggetto);
6 assert rimosso : "L'oggetto doveva essere presente nella lista!";

```

Se si lavora su sistemi estremamente critici dove anche lo spazio occupato dai file `.class` è un problema, si può usare una costante `static final`:

```

1 public static final boolean DEBUG = true;
2 // ...
3 if (DEBUG) assert x >= 0;

```

💡 Tip

Il compilatore Java è intelligente: se `DEBUG` è `false`, l'intero blocco `if` viene rimosso dal bytecode finale perché considerato "unreachable code".

6.2.2 Quando scegliere le Asserzioni

Java offre tre strumenti per gestire i fallimenti del sistema: le **Eccezioni**, il **Logging** (che vedremo nella prossima sezione) e le **Asserzioni**. Scegliere quella corretta dipende dalla natura dell'errore.

- **Eccezioni:** Per condizioni recuperabili o errori comunicati all'utente/chiamante.
- **Logging:** Per tenere traccia di eventi e diagnosticare problemi nel tempo.
- **Asserzioni:** Per individuare errori logici interni durante lo sviluppo.

Le asserzioni hanno due caratteristiche fondamentali:

1. Sono destinate a errori **fatali e non recuperabili**.
2. Sono attive **solo durante lo sviluppo**. C'è una battuta famosa nel settore: "Usare le asserzioni è come indossare un giubbotto di salvataggio finché sei vicino a riva, per poi buttarlo via quando sei in mezzo all'oceano".

Il dubbio più comune è: "Devo controllare i parametri di un metodo con un `if-throw` o con un `assert`?". La risposta dipende dal **contratto** del metodo.

Se la documentazione del metodo dichiara esplicitamente che verrà lanciata un'eccezione in caso di parametri errati (usando i tag Javadoc `@throws`), quel comportamento fa parte del **contratto**. In questo caso **non** si devono usare le asserzioni.

Listing 6.19: Esempio di contratto con eccezioni

```
1 /**
2  * @throws IllegalArgumentException se fromIndex > toIndex
3  */
4 static void sort(int[] a, int fromIndex, int toIndex) {
5     if (fromIndex > toIndex)
6         throw new IllegalArgumentException("fromIndex > toIndex")
7         ;
8     // ...
9 }
```

Se invece la documentazione pone una **precondizione** (es. "il parametro *a* non deve essere `null`") senza promettere un'eccezione specifica, il chiamante è avvisato: se viola la precondizione, il comportamento è indefinito. Qui l'asserzione è lo strumento perfetto per il programmatore.

Tip

In sintesi: se il controllo deve sopravvivere nel prodotto finale perché l'errore può capitare anche se il codice è perfetto (es. input utente), usa le **Eccezioni**. Se il controllo serve a beccare il programmatore mentre scrive codice sbagliato, usa le **Asserzioni**.

Generics

7.1 Introduzione

Prima dell'introduzione dei *Generics*, la programmazione generica in Java veniva realizzata esclusivamente tramite l'ereditarietà. La classe `ArrayList`, ad esempio, gestiva semplicemente un array di riferimenti di tipo `Object`. Questo approccio presentava due problematiche critiche:

- **Casting Obbligatorio:** Ogni volta che si recuperava un valore dalla lista, era necessario un cast esplicito al tipo desiderato.

```
1 ArrayList files = new ArrayList();  
2 String filename = (String) files.get(0); // Cast necessario
```

- **Assenza di Controllo sui Tipi:** Era possibile inserire oggetti di qualsiasi classe nella stessa collezione senza che il compilatore segnalasse errori.

```
1 files.add(new File("...")); // Compila correttamente, ma  
   causera' un errore al runtime
```

I *Generics* offrono una soluzione basata sui parametri di tipo. La sintassi permette di indicare esplicitamente il tipo di elementi che la collezione deve gestire.

- **Sintassi Diamond:** Se il tipo è dichiarato esplicitamente a sinistra, si può omettere nel costruttore usando l'operatore diamante `<>`.

```
1 ArrayList<String> files = new ArrayList<>();
```

- **Inferenza con var:** Introdotta nelle versioni recenti, permette di dedurre il tipo dalla parte destra dell'assegnazione.

```
1 var files = new ArrayList<String>();
```

L'appello dei parametri di tipo risiede in due pilastri fondamentali:

1. **Leggibilità:** È immediatamente chiaro quali oggetti siano contenuti nella lista (es. solo `String`).
2. **Sicurezza (Type Safety):** Il compilatore impedisce l'inserimento di oggetti di tipo errato.



Un errore segnalato dal compilatore (*compile-time error*) è di gran lunga preferibile a un'eccezione durante l'esecuzione (*runtime ClassCastException*). I *Generics* spostano il controllo della correttezza del codice dalla fase di esecuzione alla fase di scrittura.

La padronanza dei *Generics* si sviluppa su tre livelli distinti, spostando l'attenzione dall'utilizzo alla progettazione.

1. **Utilizzo (Livello Base):** Uso delle classi fornite dal JDK (es. `ArrayList<String>`) come se fossero tipi nativi del linguaggio.
2. **Risoluzione Problemi (Livello Intermedio):** Comprensione dei meccanismi interni per gestire messaggi di errore complessi o interoperabilità con codice legacy.

3. **Implementazione (Livello Avanzato):** Creazione di proprie classi e metodi generici, anticipando i potenziali utilizzi futuri degli utenti.

Un compito cruciale del progettista è bilanciare sicurezza e usabilità. Consideriamo, ad esempio, il metodo `addAll`:

- Deve essere possibile aggiungere `ArrayList<Manager>` a `ArrayList<Employee>` perché un `Manager` è un `Employee`.
- Non deve essere possibile il contrario.

Vedremo presto che per risolvere queste asimmetrie, Java introduce i **wildcard types** (tipi jolly), che permettono di definire relazioni di sottotipo tra tipi parametrizzati.

Tip

La programmazione generica è particolarmente utile quando si scrive codice che altrimenti richiederebbe numerosi cast da tipi generali come `Object`. Per la maggior parte delle applicazioni comuni, le classi fornite dal JDK sono sufficienti.

7.2 Struttura di una Classe Generica

Una classe generica è definita dall'introduzione di una o più **variabili di tipo** posizionate dopo il nome della classe. Queste fungono da segnaposto che verranno sostituiti dai tipi reali al momento dell'istanziamento.

Listing 7.1: Esempio: la classe `Pair`

```
1  public class Pair<T>{
2      private T first;
3      private T second;
4
5      public Pair() { first = null; second = null; }
6      public Pair(T first, T second) { this.first = first;
7                                         this.second = second; }
8
9      public T getFirst() { return first; }
10     public T getSecond() { return second; }
11
12     public void setFirst(T newValue) { first = newValue; }
13     public void setSecond(T newValue) { second = newValue; }
14 }
```

La classe `Pair` dimostra come la variabile `T` sia utilizzata coerentemente in tutta la definizione:

- **Campi:** `private T first;` — Il tipo è determinato dall'utente.
- **Costruttore:** `public Pair(T first, T second)` — Accetta parametri del tipo specificato.
- **Metodi:** `public T getFirst()` — Restituisce un oggetto del tipo corretto senza necessità di cast.

Per mantenere il codice leggibile e coerente con le API di Java, si utilizzano lettere singole maiuscole:

È possibile definire classi con più variabili di tipo, ad esempio `public class Pair<T, U>`, permettendo alla classe di gestire coppie di oggetti di natura differente in modo *type-safe*.

Lettera	Significato
E	Elemento (usato nelle collezioni)
K	Chiave (Key)
V	Valore (Value)
T, U, S	Tipi generici qualsiasi

7.3 Metodi Generici

Un metodo generico può essere definito sia all'interno di classi ordinarie che di classi generiche. La sua caratteristica principale è la presenza di parametri di tipo indipendenti dalla classe.

La variabile di tipo deve essere dichiarata tra parentesi angolate *dopo* i modificatori e *prima* del tipo di ritorno:

```
modificatori <T> tipo_ritorno nomeMetodo(parametri)
```

Il compilatore Java è solitamente in grado di dedurre il tipo del parametro analizzando gli argomenti passati alla chiamata del metodo.

- **Chiamata esplicita:** `ArrayAlg.<String>getMiddle("A", "B");`
- **Chiamata con inferenza:** `ArrayAlg.getMiddle("A", "B");`

Quando vengono passati tipi eterogenei (es. `Double` e `Integer`), il compilatore cerca il *common supertype*. Se esistono più supertipi validi (come `Number` e `Comparable`), l'inferenza fallisce.

Il trucco dell'errore indotto

Per scoprire quale tipo il compilatore sta inferendo "sotto il cofano", prova ad assegnare il risultato del metodo a un tipo palesemente errato (es. `JButton`). L'errore del compilatore rivelerà l'esatta gerarchia di tipi (intersezione di tipi) che ha trovato, come `Object & Serializable & Comparable`.

7.4 Variabili di Tipo Vincolate (Bounded Type Variables)

Talvolta è necessario limitare i tipi che possono essere sostituiti a una variabile di tipo `T`. Questo permette di invocare metodi specifici (come `compareTo`) sull'oggetto generico.

La clausola `<T extends BoundingType>` indica che `T` deve essere un **sottotipo** del tipo vincolante.

- **Esempio:** `<T extends Comparable>` garantisce che `T` implementi l'interfaccia `Comparable`.
- **Nota Linguistica:** Si usa sempre `extends`, anche se il vincolo è un'interfaccia, per denotare il concetto generale di "sottotipo" senza aggiungere nuove keyword: in tale contesto, la parola chiave `implements` **non esiste**.

È possibile specificare più vincoli separandoli con l'operatore `&` (la virgola è già usata per separare le variabili di tipo).

```
T extends Class1 & Interface1 & Interface2
```

Regole di Precedenza

1. Si può specificare al massimo una classe come vincolo.



2. Se presente, la classe deve essere indicata per **prima** nella lista dei vincoli.
3. Le interfacce seguono la classe, separate da **&**.

Ecco come appare il codice di un metodo `min` che calcola il minimo di un array di oggetti generici, purché siano comparabili:

```

1 public static <T extends Comparable> T min(T[] a) {
2     if (a == null || a.length == 0) return null;
3     T smallest = a[0];
4     for (int i = 1; i < a.length; i++) {
5         if (smallest.compareTo(a[i]) > 0) {
6             smallest = a[i];
7         }
8     }
9     return smallest;
10 }
11

```

Osservazione 7.1. L'interfaccia `Comparable` dell'esempio è essa stessa di tipo generico; ciò comporta che la sintassi che abbiamo usato genera dei warnings dovuta all'uso del *raw type*. Chiariremo questo dettaglio in [7.9.3](#)

7.5 Funzionamento interno dei Generics

Vediamo ora come la JVM gestisce internamente i Generics.

7.5.1 Type Erasure (Cancellazione del Tipo)

La *Type Erasure* è il processo mediante il quale il compilatore Java traduce il codice generico in codice compatibile con le versioni precedenti della JVM (pre-Java 5).

Per ogni tipo generico, il compilatore genera un **Raw Type** (tipo grezzo) seguendo queste regole:

- Le variabili di tipo senza vincoli (`<T>`) vengono sostituite da `Object`.
- Le variabili con vincoli (`<T extends B1 & B2>`) vengono sostituite dal **primo vincolo** (`B1`).

Listing 7.2: Esempio: la classe `Interval`

```

1 public class Interval<T extends Comparable & Serializable>
2     implements Serializable{
3     private T lower;
4     private T upper;
5     . . .
6     public Interval(T first, T second)
7     {
8         if (first.compareTo(second) <= 0) { lower = first;
9             upper = second; }
10        else { lower = second; upper = first; }
11    }
12 }

```

Ad esempio, per la classe `Interval<T extends Comparable & Serializable>`:

- Il campo `private T lower` diventa `private Comparable lower`.
- Il metodo `compareTo` può essere chiamato direttamente senza cast.

⚠ Efficienza e Ordine dei Vincoli

Se un tipo ha più vincoli, l'ordine non è indifferente. Il compilatore sostituisce `T` con il primo vincolo della lista. Se il primo vincolo è un'interfaccia "tag" (senza metodi, come `Serializable`), il compilatore dovrà inserire dei cast extra per invocare i metodi degli altri vincoli. Più in generale, per ridurre il *runtime overhead*, conviene posizionare come primo vincolo l'interfaccia i cui metodi vengono invocati più frequentemente all'interno del codice generico.

7.5.2 Traduzione delle Espressioni Generiche

Dopo che il compilatore ha eseguito la *Type Erasure*, le chiamate ai metodi e l'accesso ai campi devono essere adattati per mantenere la coerenza dei tipi. Poiché il tipo di ritorno di un metodo cancellato è solitamente `Object`, il compilatore inserisce automaticamente un **cast** nel codice chiamante. Data la chiamata `Employee buddy = buddies.getFirst();`, la traduzione nel bytecode avviene in due passaggi:

1. Chiamata al metodo *raw* d'istanza `getFirst()` della classe `Pair` (ritorna `Object`).
2. Conversione (*cast*) del valore ritornato da `Object` a `Employee`.

Il medesimo principio si applica all'accesso diretto ai campi, qualora siano visibili. Anche se sintatticamente non appare alcun cast nel codice sorgente `.java`, il bytecode risultante conterrà un'istruzione di cast per garantire che il riferimento sia del tipo corretto.

Il vantaggio dei Generics non è quindi l'eliminazione dei cast a livello di esecuzione (che continuano a esistere nel bytecode), ma il fatto che il compilatore li gestisca **automaticamente** e in modo **sicuro**, sollevando il programmatore dal rischio di errori manuali. Praticamente, i Generics non sono altro che un modo di gestire in modo automatico ciò che prima della loro introduzione si faceva a mano.

7.5.3 Type Erasure e Polimorfismo: i Metodi Ponte (Bridge Methods)

La cancellazione del tipo si applica ovviamente anche ai metodi generici; generalmente, si tende a pensare al metodo:

```
public static <T extends Comparable> T min(T[] a)
```

come a una famiglia di metodi. In realtà, dopo la cancellazione del tipo, ciò che resta è il singolo metodo:

```
public static Comparable min(Comparable[] a)
```

La cancellazione del tipo sui metodi generici comporta però qualche complicazione quando una sottoclasse prova a fare l'override di un metodo ereditato da una classe generica. Consideriamo ad esempio:

```
1  class DateInterval extends Pair<LocalDate>{
2      public void setSecond(LocalDate second)
3      {
4          if (second.compareTo(getFirst()) >= 0)
5              super.setSecond(second);
```

```

6         }
7         . . .
8     }

```

Qui è stato eseguito l'override di `setSecond` per essere sicuri che la seconda data non sia mai precedente alla prima. Il problema è che nella classe `Pair`, dopo la cancellazione del tipo, esiste il metodo `setSecond(Object second)`, che viene ereditato da `DateInterval`; questo è un metodo diverso da `setSecond(LocalDate second)` che abbiamo definito (ha una firma diversa, come se si fosse fatto l'overload al posto di un override).

Per evitare che il polimorfismo fallisca, il compilatore genera automaticamente un *metodo ponte* nella sottoclasse:

```

1     public void setSecond(Object second){
2         setSecond((LocalDate) second);
3     }

```

Q Dettagli: Logica di Rilevamento dei Metodi Ponte

Il compilatore genera un *bridge method* quando rileva una discrepanza tra l'intento del programmatore e i vincoli della *Type Erasure*.

Il compilatore esegue i seguenti passaggi logici durante l'analisi della sottoclasse:

1. **Analisi delle Firme:** Identifica un metodo nella sottoclasse che corrisponde a un metodo della superclasse generica (es. `setSecond(LocalDate)` vs `setSecond(T)`).
2. **Previsione della Cancellazione:** Calcola come appariranno le firme nel bytecode (es. `LocalDate` rimane tale, ma `T` diventa `Object`).
3. **Rilevamento del Conflitto:** Se le firme cancellate non coincidono, il metodo non è più un *override* valido per la JVM.
4. **Sintesi:** Genera automaticamente un metodo con la firma cancellata della superclasse che delega il lavoro al metodo della sottoclasse.

Il compilatore "capisce" la necessità del ponte perché possiede la tabella dei tipi completi prima che la cancellazione dei dati abbia luogo. Il ponte è il meccanismo che permette di mappare una chiamata generica *raw* (basata su `Object`) alla specifica implementazione *type-safe* della sottoclasse.

La situazione diventa ancora più strana con i metodi che restituiscono valori; consideriamo ad esempio:

```

1     class DateInterval extends Pair<LocalDate>{
2         public LocalDate getSecond() { return (LocalDate) super.
3             getSecond(); }
4         . . .
5     }

```

Nella classe `DateInterval` avremmo a questo punto due metodi con lo stesso nome `getSecond()` e nessun parametro: uno che restituisce un `Object` e uno che restituisce un `LocalDate`; come ben sappiamo, in Java questo sarebbe illegale, perché non si possono avere due metodi con la stessa firma che restituiscono tipi diversi. Tuttavia, nella Virtual Machine, un metodo è identificato univocamente dalla combinazione di nome, parametri e tipo di ritorno. Il compilatore può quindi generare bytecode che la JVM gestisce correttamente, anche se noi non potremmo scriverlo a mano. Il compilatore Java si

comporta in questi casi quasi come un "hacker" che sfrutta le libertà della Virtual Machine per aggirare i limiti del linguaggio stesso.

⚠ Firma del Metodo: Java vs JVM

Esiste una distinzione fondamentale tra come il linguaggio Java e la JVM identificano i metodi:

- **Java Language:** Un metodo è identificato univocamente dalla sua *signature* (nome e tipi dei parametri). Il tipo di ritorno viene ignorato per evitare ambiguità durante l'invocazione.
- **JVM Bytecode:** Un metodo è identificato dal nome, dai parametri e dal **tipo di ritorno**.

Questa differenza è ciò che permette al compilatore di sintetizzare i *bridge methods*. Il compilatore genera due metodi con la stessa firma Java ma con ritorni diversi, permettendo alla JVM di risolvere correttamente le chiamate polimorfiche dopo la *type erasure*.

Osservazione 7.2 (Ritorno Covariante e Bridge Methods). I *bridge methods* non sono limitati ai tipi generici; vengono impiegati anche per gestire i **tipi di ritorno covarianti**. Come abbiamo visto nel capitolo su ereditarietà e polimorfismo, si ha covarianza quando una sottoclasse effettua l'override di un metodo specificando un tipo di ritorno più restrittivo rispetto a quello della superclasse.

Consideriamo ad esempio:

```

1      class Animal {
2          public Animal getIdentity() { return this; }
3      }
4
5      class Cat extends Animal {
6          @Override
7          public Cat getIdentity() { return this; } // Ritorno
8              covariante
          }

```

Nella classe `Cat`, il compilatore sintetizza effettivamente due metodi nel bytecode per garantire la compatibilità:

- `Cat getIdentity()`: Il metodo definito dal programmatore.
- `Animal getIdentity()`: Il *bridge method* che sovrascrive il metodo di `Animal` e chiama internamente la versione specifica.

7.6 Interoperabilità con Codice Legacy

Uno degli obiettivi primari dei *Generics* era la compatibilità con il codice scritto prima di Java 5. Questo avviene attraverso l'uso dei **Raw Types** (tipi grezzi). Un *Raw Type* è il nome di una classe generica utilizzata senza parametri di tipo (es. `ArrayList` invece di `ArrayList<T>`).

- **da codice moderno a legacy:** Passando una collezione generica a un metodo che accetta un *Raw Type*, il compilatore emette un *warning*. Il rischio è che il vecchio metodo inserisca oggetti di tipo errato, rompendo l'integrità della collezione.

```

1      // CODICE LEGACY (Pre-2004)

```

```

2      public class VecchioSistema {
3          public static void stampaLibri(ArrayList lista) {
4              // Nota: non c'è <String>
4              // Fa qualcosa con la lista...
5          }
6      }
7      //CODICE MODERNO
8      ArrayList<String> iMieiLibri = new ArrayList<>();
9      iMieiLibri.add("Il Signore degli Anelli");
10
11     // Quando passi la tua lista "sicura" al vecchio
12     metodo:
13     VecchioSistema.stampaLibri(iMieiLibri); // <-- QUI
14     SCATTA IL WARNING

```

- **da legacy a codice moderno:** Assegnando un *Raw Type* a una variabile generica, si ottiene un *warning* di "unchecked assignment", poiché il compilatore non può garantire il contenuto della collezione.

```

1      // Ricevi una lista "misteriosa" (Raw Type)
2      ArrayList listaDalPassato = VecchioSistema.
3          getTuttiILibri();
4
5      // Provi a metterla in una variabile moderna
6      ArrayList<String> libriModerni = listaDalPassato; //
7      <-- ALTRO WARNING

```



Gestione dei Warning

Quando si è certi che l'operazione sia sicura (es. sappiamo che la vecchia libreria legge solo i dati senza modificarli), si può utilizzare l'annotazione `@SuppressWarnings("unchecked")` per istruire il compilatore a ignorare l'avviso su una variabile o su un intero metodo.

```

1      @SuppressWarnings("unchecked")
2      ArrayList<String> libriModerni = VecchioSistema.
3          getTuttiILibri();

```

7.7 Record Patterns e Generics

Nelle versioni più recenti di Java (Java 21+), i record possono essere definiti in modo generico e utilizzati con il **pattern matching** per semplificare l'estrazione dei dati. Un record generico condensa la definizione di una classe *data-carrier* in un'unica riga:

```

1 record Pair<T>(T first, T second) {}

```

L'operatore `instanceof` può ora "de-costruire" un record generico, estraendo i suoi componenti in variabili locali in un unico passaggio:

```

1 if (p instanceof Pair(var a, var b)) {
2     // a e b vengono estratti e tipizzati automaticamente
3 }

```


Il compilatore utilizza le informazioni dei Generics per inferire il tipo delle variabili estratte. Se `p` è dichiarato come `Pair<String>`, le variabili `a` e `b` nel pattern verranno trattate come `String`. È possibile anche specificare i tipi esplicitamente: `p instanceof Pair<String>(String a, String b)`.

I record patterns funzionano non solo con `instanceof`, ma anche all'interno delle espressioni `switch`, rendendo la gestione di diverse combinazioni di dati estremamente pulita e sicura.

7.8 Ereditarietà nei Tipi Generici

Una delle regole più importanti (e spesso meno intuitive) dei Generics è che la gerarchia dei tipi delle classi **non si riflette** sui tipi generici corrispondenti (**Principio di Invarianza nei Generics**): nonostante `Manager` sia una sottoclasse di `Employee`, `Pair<Manager>` **non** è una sottoclasse di `Pair<Employee>`.

⚠ Contaminazione del Tipo

Se fosse permessa la conversione:

```
1 Pair<Manager> managers = new Pair<>();
2 Pair<Employee> employees = managers; // Errore di
   compilazione
3 employees.setFirst(new Employee()); // Questo corromperebbe
   la lista di Manager
```

L'invarianza garantisce che una collezione non possa essere "contaminata" con oggetti di un supertipo non compatibile con l'originale.

🔍 Confronto con gli Array

A differenza dei Generics, gli array in Java sono *covarianti*: `Employee[] staff = new Manager[10]` è legale. Tuttavia, questa scelta è considerata un difetto di progettazione del Java originale, poiché sposta gli errori dal tempo di compilazione al tempo di esecuzione (`ArrayStoreException`).

```
1 String[] s = new String[10];
2 Object[] o = s; // consentito: covarianza
3
4 o[0] = 42; // compila, ma a runtime
5 // lancia ArrayStoreException
```

Abbiamo dunque stabilito che `Pair<Manager>` non è imparentato con `Pair<Employee>`. Tuttavia, entrambi sono figli del *Raw Type* `Pair`. Questa conversione è permessa per un motivo pragmatico: se non si potesse passare un `Pair<String>` a un vecchio metodo che accetta solo `Pair`, non si potrebbe usare quasi nessuna libreria scritta prima del 2004. Il problema è che, una volta che si "declassa" l'oggetto a Raw Type, si perde la "protezione" del compilatore: l'errore di tipo viene ignorato in fase di compilazione, portando alla "contaminazione" della struttura dati originale.

```
1 Pair<Manager> managers = new Pair<>(ceo, cfo);
2 Pair raw = managers; // Legale (con warning)
3 raw.setFirst(new File("../")); // Compila!
```

La sicurezza della JVM non è tuttavia compromessa. La cancellazione dei tipi (*Type Erasure*) non impedisce alla JVM di sollevare una `ClassCastException` al momento del recupero del dato incoerente. Si perde solo il vantaggio del controllo preventivo offerto dai Generics.

7.9 Wildcard Types (Tipi Jolly)

Le *wildcards* permettono di rendere flessibili i parametri di tipo, introducendo una forma di **covarianza** controllata. Supponiamo, ad esempio, di voler scrivere un metodo che stampa una coppia di `Employee`:

```

1  public static void printBuddies(Pair<Employee> p){
2      Employee first = p.getFirst();
3      Employee second = p.getSecond();
4      System.out.println(first.getName() + " and " + second.
5                          getName() + " are buddies.");
6  }
```

Come abbiamo visto nel paragrafo precedente, non possiamo passare una coppia di `Manager` al metodo; per aggirare questa limitazione, possiamo usare una wildcard:

```
public static void printBuddies(Pair<? extends Employee> p)
```

Il tipo `Pair<? extends Employee>` indica una coppia il cui tipo parametrizzato è `Employee` o una sua qualsiasi sottoclasse (es. `Manager`, `Executive`). A differenza dei tipi generici standard, le wildcard stabiliscono una relazione di ereditarietà:

$$\text{Pair<Manager>} \subset \text{Pair<? extends Employee>}$$

Questo permette di scrivere metodi polimorfici che accettano collezioni di sottotipi diversi.

7.9.1 Usi consentiti

Le wildcard sono ammesse ovunque si stia dichiarando il tipo di una variabile. In particolare, si possono usare nei seguenti contesti:

- parametri di un metodo (l'uso più comune):

```
public void stampa(List<? extends Persona> lista) ...
```

- campi di una classe o variabili locali:

```
List<? extends Persona> lista = iMieiManager;
```

- tipi di ritorno (lecito, ma sconsigliato):

```
public List<? extends Persona> getTeam() ...
```

Questo uso non è considerato una buona pratica, perché costringi chi usa il metodo a gestire la wildcard, rendendogli la vita difficile

È abbastanza intuitivo che l'uso di questa sintassi non è invece consentito laddove provochi ambiguità non risolvibile dal compilatore:

- definizione di classi/metodi: E' necessario utilizzare un identificatore (es. `T`, `E`, `K`).

- Operatore **new**: L'allocazione di memoria richiede un tipo concreto.

Nel paragrafo precedente abbiamo esaminato i limiti dell'ereditarietà sui Generics, imposti per evitare contaminazioni di tipo. Le wildcards permettono questo tipo di contaminazione? La risposta è NO: l'utilizzo della wildcard `? extends T` crea una struttura dati in **sola lettura** (o quasi). Questo garantisce che la struttura originale non venga "corrotta" con tipi incompatibili.

Infatti, dato un riferimento `Pair<? extends Employee>`, il compilatore interpreta i metodi così:

- `? extends Employee getFirst()`: **FUNZIONA**. Qualunque sia il tipo reale, è un sottotipo di `Employee`. L'assegnazione a una variabile `Employee` è sicura.
- `void setFirst(? extends Employee)`: **ERRORE**. Il compilatore non può verificare se l'oggetto che stiamo passando sia compatibile con il tipo *specifico* (ma ignoto) della coppia reale, quindi vieta la modifica.



Con `? extends T` puoi **leggere** oggetti e trattarli come `T`, ma non puoi **scrivere** nulla (tranne `null`).

Esempio

Consideriamo la seguente gerarchia: `Manager` $\subset$ `Dipendente` $\subset$ `Persona`.

```

1 // Definizione delle classi e del contesto
2 class Persona { ... }
3 class Dipendente extends Persona { ... }
4 class Manager extends Dipendente { ... }
5
6 Pair<Manager> iMieiManager = new Pair<>(new Manager("Alice"), new
   Manager("Bob"));
7 Pair<? extends Dipendente> jolly = iMieiManager; // Riferimento
   con wildcard
8
9 Dipendente unDipendente = new Dipendente("Carlo");
10 Manager unManager = new Manager("Diana");

```

Di seguito l'esito della compilazione per diverse operazioni di lettura e scrittura:

- `Dipendente d = jolly.getFirst();` → **SI**. Il "tetto massimo" garantito dalla wildcard è `Dipendente`.
- `Persona p = jolly.getFirst();` → **SI**. Poiché ogni `Dipendente` è una `Persona`.
- `Manager m = jolly.getFirst();` → **NO. Errore di compilazione**: il compilatore sa che l'oggetto è un `Dipendente`, ma non ha la certezza che sia un `Manager`.
- `jolly.setFirst(unDipendente);` → **NO**. Il compilatore blocca la scrittura perché non conosce il tipo specifico sottostante (che potrebbe essere `Pair<Manager>`).
- `jolly.setFirst(unManager);` → **NO**. Anche se in questo caso l'oggetto reale è proprio un `Pair<Manager>`, il compilatore non può saperlo e nega l'accesso per sicurezza.
- `jolly.setFirst(null);` → **SI**. `null` è l'unico valore accettato in scrittura con `? extends`.
- `Object o = jolly.getFirst();` → **SI**. Qualsiasi oggetto è un'istanza di `Object`.

7.9.2 Wildcard con vincolo inferiore: ? super T

La wildcard ? super T (Lower Bound) è lo strumento speculare a extends, ottimizzato per le operazioni di **scrittura**. Ad esempio, dato un riferimento Pair<? super Manager>, il compilatore ragiona così:

- **In Scrittura:** Accetta Manager (o sottotipi come Executive). E' sicuro che il contenitore reale sia in grado di ospitarli, poiché è almeno un Pair<Manager> o un suo supertipo.
- **In Lettura:** Non garantisce nulla sul tipo estratto. L'oggetto restituito deve essere trattato come un Object, poiché il contenitore potrebbe essere un Pair<Object>.

Esempio

Consideriamo la gerarchia: Manager $\subset$ Dipendente $\subset$ Persona.

```

1 Pair<? super Dipendente> jolly = new Pair<Persona>(...);
2
3 Dipendente unDipendente = new Dipendente("Carlo");
4 Manager unManager = new Manager("Diana");
5 Persona unaPersona = new Persona("Zoe");

```

- **Dipendente d = jolly.getFirst();** → NO. In lettura, la wildcard super non garantisce nulla se non Object.
- **jolly.setFirst(unDipendente);** → SI. Dipendente è il tipo base accettato.
- **jolly.setFirst(unManager);** → SI. Poiché un Manager è un Dipendente, può essere inserito in sicurezza.
- **jolly.setFirst(unaPersona);** → NO. Il compilatore teme che il contenitore reale sia un Pair<Dipendente>, dove una Persona non potrebbe entrare.
- **Object o = jolly.getFirst();** → SI. Sempre legale.

Tip

Mentre ? extends limita la **scrittura**, ? super limita drasticamente la **lettura**. Quest'ultima è utile quasi esclusivamente per i metodi che devono "ricevere" dati da elaborare o salvare.

7.9.3 Applicazioni in Supertipi Generici

In un paragrafo precedente avevamo usato il seguente esempio, che qui riportiamo per comodità:

```

1
2 public static <T extends Comparable> T min(T[] a) {
3     if (a == null || a.length == 0) return null;
4     T smallest = a[0];
5     for (int i = 1; i < a.length; i++) {
6         if (smallest.compareTo(a[i]) > 0) {
7             smallest = a[i];
8         }
9     }
10    return smallest;
11 }

```

Avevamo anticipato che la sintassi usata genera un warning (dovuta all'uso di un raw type); in effetti, essendo `Comparable` essa stessa un'interfaccia generica avremmo potuto scrivere più precisamente:

```
1 public static <T extends Comparable<T>> T min(T[] a)
```

Questo tipo di sintassi funzionerebbe senza problemi nella maggior parte dei casi, ma non sempre; infatti, la firma `<T extends Comparable<T>>` è troppo restrittiva: se una classe (es. `LocalDate`) eredita l'implementazione di `Comparable` da una superclasse (es. `ChronoLocalDate`), essa non risulterà compatibile con il vincolo `Comparable<T>`.

🔍 Dettagli: Analisi di `LocalDate`

- `LocalDate` estende `ChronoLocalDate`.
- `ChronoLocalDate` implementa `Comparable<ChronoLocalDate>`.
- Pertanto, `LocalDate` implementa `Comparable<ChronoLocalDate>`, ma **non** `Comparable<LocalDate>`.

In questi casi occorre usare le wildcards e la sintassi corretta da usare è:

```
1 public static <T extends Comparable<? super T>> T min(T[] a)
```

✅ Best Practice

Quando si definisce un metodo generico che richiede un vincolo su un'interfaccia a sua volta generica (come `Comparable` o `Comparator`), è sempre preferibile utilizzare la wildcard `? super T` per massimizzare la compatibilità con le gerarchie di classi esistenti.

L'uso di `? super` è estremamente comune nelle interfacce funzionali di Java (come `Predicate`, `Function` o `Consumer`), poiché permette di riutilizzare logica generica su tipi specifici.

Ad esempio, il metodo `removeIf` della collezione `Collection<E>` è dichiarato come:

```
default boolean removeIf(Predicate<? super E> filter)
```

Grazie al vincolo `? super E`, possiamo passare un filtro che lavora su un supertipo di `E`. Supponiamo allora di voler rimuovere tutti gli elementi con hashcode dispari da una collezione:

```
1 ArrayList<Employee> staff = ...;
2 // Un predicato che lavora su Object (supertipo di Employee)
3 Predicate<Object> logic = obj -> obj.hashCode() % 2 != 0;
4
5 // Legale grazie a ? super E
6 staff.removeIf(logic);
```

Se la firma fosse stata `Predicate<E>`, il compilatore ci avrebbe costretto a scrivere un nuovo predicato specifico per `Employee`, anche se la logica era già disponibile per `Object`.

7.9.4 Il Principio PECS (Producer Extends, Consumer Super)

Le *wildcards* con vincoli non sono così banali da capire e usare correttamente; per agevolarne comprensione e uso, esiste una regola mnemonica fondamentale per decidere quale tipo di jolly utilizzare: il principio **PECS**.

Mantra PECS

- **Producer Extends**: se la struttura dati deve *fornire* elementi (produzione), usa `? extends T`.
- **Consumer Super**: se la struttura dati deve *ricevere* elementi (consumo), usa `? super T`.

Il compilatore Java applica queste restrizioni per garantire la *Type Safety* a runtime, evitando che una collezione venga "corrotta" con oggetti di tipo errato.

1. **Il Producer (extends)**: Quando dichiari `List<? extends Number>`, stai dicendo: "Questa è una lista di *qualcosa* che estende `Number` (es. `Integer`, `Double`, etc.)".
 - Puoi **leggere** con sicurezza perché qualsiasi cosa esca dalla lista sarà almeno un `Number`.
 - Non puoi **scrivere** perché il compilatore non sa se la lista reale sia una `List<Double>` e tu stia cercando di inserire un `Integer`.
2. **Il Consumer (super)**: Quando dichiari `List<? super Integer>`, stai dicendo: "Questa lista può ospitare `Integer` perché è una lista di `Integer` o di un suo supertipo (es. `Number`, `Object`)".
 - Puoi **scrivere** con sicurezza perché un `Integer` "entra" correttamente in una lista di `Object` o `Number`.
 - Non puoi **leggere** (se non come `Object`) perché non sai se la lista contiene `Object`, rendendo rischioso l'assegnamento a variabili più specifiche.

L'esempio più celebre di PECS nel JDK è il metodo per copiare elementi da una lista all'altra. Analizziamo la sua firma:

Listing 7.3: Applicazione di PECS in `Collections.copy`

```

1 public static <T> void copy(List<? super T> dest, List<? extends
  T> src) {
2     for (int i = 0; i < src.size(); i++) {
3         dest.set(i, src.get(i));
4     }
5 }
```

In questo scenario:

- **src** è il **Producer (extends)**: fornisce gli elementi da copiare. Vogliamo essere in grado di leggere da una lista di `Manager` se il tipo `T` è `Employee`.
- **dest** è il **Consumer (super)**: riceve gli elementi. Vogliamo essere in grado di scrivere i nostri oggetti in una lista che potrebbe essere più generica (es. scrivere `Manager` in una lista di `Employee` o `Object`).

Best Practice

Usa PECS ogni volta che progetti un metodo che accetta collezioni generiche come

- ✓ parametri. Senza questa flessibilità, il tuo codice risulterà inutilmente rigido e costringerà gli utenti a cast manuali o a conversioni costose tra tipi di collezioni.

⚠ Eccezione alla regola

Se hai bisogno sia di leggere che di scrivere in una struttura dati all'interno dello stesso metodo, non usare i wildcards. In quel caso, devi utilizzare un tipo generico esatto (es. `List<T>`).

7.9.5 Unbounded Wildcards

La sintassi `Pair<?>` indica una coppia di un tipo completamente ignoto. Sebbene possa somigliare a un *Raw Type*, la gestione della sicurezza è radicalmente diversa. La differenza fondamentale risiede nella protezione dei dati:

- **Raw Type (`Pair`):** Permette di chiamare `setFirst(Object)`, esponendo al rischio di *type corruption*.
- **Unbounded (`Pair<?>`):** **Proibisce** qualsiasi chiamata a metodi di scrittura (tranne per il passaggio di `null`). Non sapendo quale sia il tipo atteso, il compilatore blocca ogni inserimento.

Le wildcard senza vincoli sono ideali per metodi "agnostici" rispetto al tipo, che devono solo compiere operazioni strutturali (come il controllo della presenza di valori nulli o il conteggio degli elementi).

```
1 public static boolean hasNulls(Pair<?> p) {
2     return p.getFirst() == null || p.getSecond() == null;
3 }
```

Osservazione 7.3. Il metodo sopra potrebbe essere scritto come:

```
<T> boolean hasNulls(Pair<T> p).
```

Tuttavia, la versione con `?` è preferibile perché più concisa e comunica immediatamente che il tipo `T` non viene mai utilizzato nel corpo del metodo.

7.9.6 Wildcard Capture

Esistono situazioni in cui l'uso delle wildcard blocca operazioni lecite, come lo scambio di elementi, perché `?` non è un nome di tipo valido per dichiarare variabili locali.

Per risolvere il problema, si delega l'operazione a un metodo *helper* generico. Il parametro di tipo `<T>` del metodo helper "cattura" la wildcard, dandole un nome temporaneo utilizzabile nel codice.

```
1 public static void swap(Pair<?> p) {
2     swapHelper(p); // T cattura la wildcard ?
3 }
4
5 private static <T> void swapHelper(Pair<T> p) {
6     T t = p.getFirst();
7     p.setFirst(p.getSecond());
8     p.setSecond(t);
9 }
```


La cattura della wildcard è possibile solo se il compilatore può garantire che la wildcard rappresenti un **singolo tipo definito**. Ad esempio, non è possibile catturare le wildcard in strutture nidificate come `ArrayList<Pair<?>>`, poiché ogni elemento della lista potrebbe avere un tipo di wildcard differente.

7.10 Restrizioni e Limitazioni dei Generics

La maggior parte delle limitazioni dei *Generics* in Java deriva direttamente dalla **Type Erasure** (cancellazione dei tipi). Ad esempio, la prima (ovvia) limitazione è che non è possibile utilizzare tipi primitivi come parametri di tipo: `Pair<double>` è illegale, mentre `Pair<Double>` è corretto. Dopo la cancellazione dei tipi il compilatore sostituisce `T` con `Object`, la struttura risultante può ospitare solo **referimenti a oggetti**.

7.10.1 Verifiche di Tipo a Runtime

A causa della *Type Erasure*, tutte le verifiche sui tipi effettuate durante l'esecuzione riguardano esclusivamente i **Raw Types**.

- **instanceof**: Non è possibile testare se un oggetto appartiene a un tipo parametrizzato specifico.

```

1      if (a instanceof Pair<String>) // ERRORE DI
      COMPILAZIONE
2      if (a instanceof Pair<?>)      // LEGALE (testa solo
      se e' un Pair)

```

- **Cast**: Il cast verso un tipo generico genera un *warning*. Il compilatore avvisa che non può garantire l'integrità del contenuto a runtime.
- **getClass()**: Il metodo `getClass()` restituisce sempre il *raw type*.

Attenzione

Due istanze di una stessa classe generica con parametri di tipo diversi (es. `Pair<String>` e `Pair<Double>`) condividono la stessa identica classe a runtime. Il confronto `p1.getClass() == p2.getClass()` sarà sempre `true`.

7.10.2 Divieto di Creazione di Array Generici

In Java, non è possibile istanziare array di tipi parametrizzati (es. `new Pair<String>[10]`). Questa restrizione protegge l'integrità del sistema dei tipi.

Il divieto esiste perché gli array sono **covarianti** e **reificati** (controllano il tipo a runtime), mentre i Generics usano la **Type Erasure**.

Se fosse permesso, potremmo:

1. Assegnare un array `Pair<String>[]` a un riferimento `Object[]`.
2. Inserire un `Pair<Employee>` nell'array (per la cancellazione del tipo, la JVM vedrebbe solo un `Pair` generico e lo accetterebbe).
3. Causare una `ClassCastException` al momento della lettura.

Best Practice

Se hai bisogno di una collezione di oggetti generici, non usare gli array. La soluzione corretta e sicura è utilizzare una `ArrayList`:



```
ArrayList<Pair<String>> lista = new ArrayList<>();
```

Le liste generiche sono invarianti e gestite interamente dal compilatore, evitando ogni rischio a runtime.

7.10.3 Limiti sui Varargs

Come sappiamo dai primi capitoli, i metodi con un numero variabile di argomenti (*varargs*) utilizzano internamente gli array. Quando si usano tipi generici con i *varargs*, si crea un conflitto con il divieto di istanziare array generici visto nel paragrafo precedente. Per non bloccare del tutto l'utilità dei *Varargs*, Java ha fatto un'eccezione: permette la creazione dell'array in questo caso specifico, ma ti "tempesta" di avvisi sul rischio di *Heap Pollution* (inquinamento dell'heap); quest'ultimo rappresenta il rischio che l'array (la cui componente è cancellata a *Object* o al *bound*) venga "inquinato" con oggetti di tipo errato.

Per silenziare questi warnings (sia nella dichiarazione che nella chiamata) è stata introdotta l'annotazione **@SafeVarargs**; questa è una sorta di promessa solenne che fai al linguaggio. Quando aggiungi **@SafeVarargs**, stai dicendo: "Giuro che in questo metodo userò l'array solo per leggerne i valori e non farò nulla di strano (come scriverci dentro o passarlo a metodi che potrebbero corromperlo)".



L'annotazione **@SafeVarargs** può essere applicata solo a metodi **static**, **final** o **private**. Rappresenta una garanzia dello sviluppatore che l'array dei parametri sarà usato solo in modalità "sola lettura" e non verrà mai esposto all'esterno.



Il pericolo del "Generic Array Creator"

È possibile usare i *varargs* per creare un metodo che restituisce un array generico (operazione normalmente vietata). Tuttavia, questa pratica è fortemente sconsigliata poiché reintroduce tutti i rischi di *ClassCastException* a runtime discussi precedentemente.

Esiste un comportamento particolare dei *varargs* generici quando vengono passati degli array. Sebbene un array possa essere passato direttamente a un parametro *varargs*, il meccanismo di "distribuzione" (*spread*) fallisce con i tipi primitivi.

1. **Meccanismo Standard (Spreading):** Quando si passa un array a un metodo con *varargs*, Java lo tratta come se gli elementi fossero stati passati singolarmente.

```
1 String[] moreStrings = { "A", "B", "C" };
2 List<String> list = List.of(moreStrings); // OK:
   lista di 3 elementi
```

2. **Il Fallimento con i Primitivi:** Poiché i parametri di tipo (come *E*) non possono essere tipi primitivi (es. *int*), un array di tipo *int[]* non può essere convertito in un array di tipo *Integer[]* o *Object[]*.

```
1 Collection<Integer> numbers = . . . ;
2 int[] moreNumbers = new int[] { 11, 12, 13, 14,
3                               15 };
   addAll(numbers, moreNumbers); // ERRORE
```

Un'altra insidia che riguarda i *varargs* con tipi primitivi è illustrata nel seguente esempio. Consideriamo il metodo `public static <E> List<E> of(E... elements)`. Se proviamo a passare un array di primitivi:

```
1 int[] moreNumbers = { 1, 2, 3 };
2 var numbers = List.of(moreNumbers);
```

In questo caso, il compilatore non può far corrispondere `E` a `int`. Tuttavia, poiché un array è esso stesso un oggetto, il compilatore fa corrispondere `E` al tipo `int[]`. Il risultato è una `List<int[]>` che contiene **un solo elemento**: l'intero array di numeri, invece di una lista di tre numeri interi.

Questo comportamento è, ancora una volta, una diretta conseguenza della *Type Erasure*: dato che `E` viene cancellato a `Object`, e un `int` non è un `Object`, l'unico modo che la JVM ha per gestire la chiamata è trattare l'intero array `int[]` come l'unico oggetto che soddisfa il parametro *varargs*.

7.10.4 Divieto di Istanziamento di Variabili di Tipo

Un'altra limitazione significativa è l'impossibilità di utilizzare variabili di tipo in espressioni di creazione di oggetti con `new T(...)`.

Il compilatore blocca queste operazioni perché, a causa della *Type Erasure*, la variabile `T` verrebbe sostituita dal suo *bound* (solitamente `Object`). Di conseguenza, l'istruzione `new T()` verrebbe compilata come `new Object()`, il che raramente produce il risultato desiderato dallo sviluppatore.

Per risolvere questo problema, il progettista deve fare in modo che il chiamante fornisca un modo per costruire l'oggetto. L'approccio più moderno e pulito consiste nel passare al metodo una *constructor expression* tramite l'interfaccia funzionale `Supplier<T>`.

Listing 7.4: Istanziamento tramite Supplier

```
1 public static <T> Pair<T> makePair(Supplier<T> constr) {
2     // Utilizziamo il metodo get() per ottenere nuove istanze di
3     T
4     return new Pair<>(constr.get(), constr.get());
5 }
6 // Chiamata al metodo tramite Method Reference
7 Pair<String> p = Pair.makePair(String::new);
```

Osservazione 7.4. Esiste un altro approccio più tradizionale ma anche più complesso da gestire basato sull'uso della *Reflection*, che qui omettiamo.

Collections

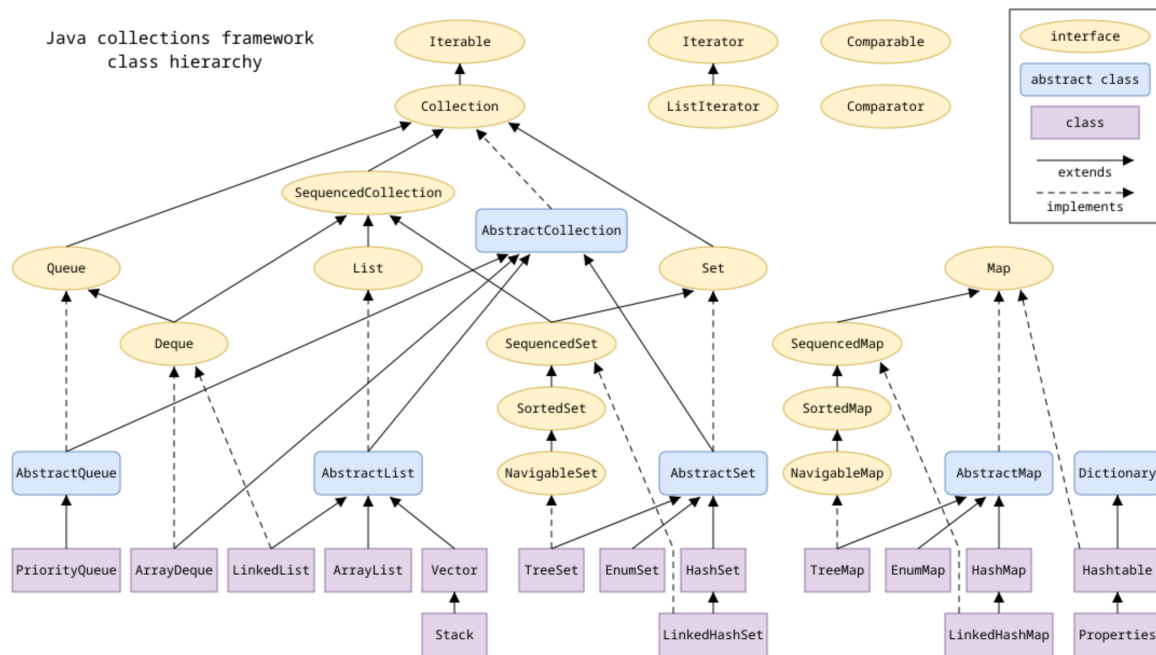


Figura 8.1: Gerarchia delle interfacce del Java Collection Framework

Il *Java Collections Framework* (JCF) si presenta come un'architettura gerarchica vasta e articolata. Per navigare in questo "groviglio" di interfacce e classi, adotteremo un **approccio top-down**: inizieremo l'analisi dalle astrazioni più elevate per poi scendere progressivamente verso le specializzazioni e le implementazioni concrete.

Il punto di partenza è rappresentato dalle interfacce radice, **Iterable** e **Collection**, che stabiliscono i requisiti minimi e i comportamenti comuni a quasi tutti i contenitori di dati in Java.

8.1 Separazione tra Interfaccia e Implementazione

Il *Java Collections Framework* adotta un approccio fondamentale della programmazione a oggetti: la netta separazione tra le **interfacce** (che definiscono il comportamento) e le **implementazioni** concrete (che definiscono le strutture dati e le prestazioni). Per illustrare bene questo concetto e i suoi vantaggi, esaminiamo il seguente esempio.

Una **Queue** (coda) specifica un comportamento **FIFO** (*First In, First Out*): aggiunta in coda, rimozione in testa e monitoraggio della dimensione.

Listing 8.1: Interfaccia minimale di una Queue

```
1 public interface Queue<E> {
```

```

2     void add(E e);
3     E remove();
4     int size();
5 }

```

L'interfaccia non dice nulla su come i dati siano memorizzati. Esistono due implementazioni principali nel JDK:

1. **ArrayDeque**: Utilizza un *circular array* (array circolare, vedi in appendice A.3).
2. **LinkedList**: Utilizza una lista concatenata di nodi.

Il vantaggio principale di questa separazione è la flessibilità. Nel codice backend, è considerata una *best practice* utilizzare il tipo interfaccia per dichiarare le variabili, limitando l'uso della classe concreta solo al momento della costruzione.

Listing 8.2: Utilizzo corretto delle interfacce

```

1 // Uso l'interfaccia come tipo della variabile
2 Queue<Customer> expressLane = new ArrayDeque<>(100);
3 expressLane.add(new Customer("Harry"));
4
5 // Se cambio idea, devo cambiare solo la riga del costruttore
6 Queue<Customer> expressLane = new LinkedList<>();

```

✓ Coding to an Interface

Usare il tipo dell'interfaccia (es. `Queue`) invece del tipo concreto (es. `ArrayDeque`) protegge il codice da dipendenze eccessive. Se accidentalmente si utilizzasse un metodo specifico di `ArrayDeque` non presente in `Queue`, il compilatore segnalerebbe l'errore, garantendo che il codice rimanga facilmente intercambiabile.

In Java 6, quando fu introdotta `ArrayDeque`, i programmatori che avevano "programmato per interfacce" poterono migliorare istantaneamente le prestazioni delle loro applicazioni cambiando una singola riga di codice (il costruttore), lasciando intatto tutto il resto della logica aziendale.

Perché scegliere l'una o l'altra? L'interfaccia è agnostica riguardo alle prestazioni, ma le implementazioni no:

- **ArrayDeque**: È generalmente più performante. Ha un impatto minore sul *Garbage Collector* perché non deve creare un oggetto "Nodo" per ogni elemento inserito. Richiede la riallocazione dell'array quando la capacità finisce (operazione computazionalmente costosa), ma se la capacità iniziale è ben calibrata, queste riallocazioni sono rare.
- **LinkedList**: Non ha problemi di capacità finita, ma ogni elemento aggiunto comporta la creazione di un nuovo oggetto in memoria, rendendola meno efficiente in scenari ad alto volume di dati.

8.2 L'Interfaccia Collection

L'interfaccia `Collection<E>` è la radice della gerarchia per le strutture dati che gestiscono gruppi di oggetti (ad eccezione delle *Map*). Essa definisce un set di operazioni standard che garantiscono l'interoperabilità tra diverse implementazioni. Di seguito, esaminiamo i metodi di uso più comune.

Operazioni di Query (Ispezione) Questi metodi permettono di ottenere informazioni sullo stato della collezione senza modificarla.

- `int size()`: Restituisce il numero di elementi.
- `boolean isEmpty()`: Equivalente a `size() == 0`.
- `boolean contains(Object o)`: Verifica la presenza di un elemento.

Operazioni di Modifica :

- `boolean add(E element)`: Restituisce **true** se l'aggiunta ha effettivamente modificato la collezione, **false** altrimenti. Ad esempio, in una *List*, il metodo `add` restituirà quasi sempre **true**; invece, in un *Set* (che non ammette duplicati), se provi ad aggiungere un oggetto già presente, il metodo restituirà **false**, segnalando che l'operazione non ha avuto effetto.
- `boolean remove(Object o)`: Rimuove una singola istanza dell'elemento, se presente. Il valore restituito segue la stessa logica di `add`.
- `void clear()`: Rimuove tutti gli elementi, svuotando la collezione.

Operazioni Bulk (di Massa) Questi metodi sono fondamentali per manipolare interi set di dati (es. confrontare liste di ID o filtrare risultati). Per `addAll`, `removeAll` e `retainAll` il valore di ritorno indica se la struttura dati è stata mutata, per `containsAll` il boolean rappresenta l'esito logico del test di inclusione.

- `boolean addAll(Collection<? extends E> c)`: **Unione**. Aggiunge tutti gli elementi della collezione `c` alla collezione corrente.
- `boolean removeAll(Collection<?> c)`: **Differenza**. Rimuove dalla collezione corrente tutti gli elementi che sono presenti anche in `c`.
- `boolean retainAll(Collection<?> c)`: **Intersezione**. Rimuove dalla collezione corrente tutti gli elementi che **non** sono contenuti in `c`, mantenendo solo i comuni.
- `boolean containsAll(Collection<?> c)`: **Inclusione**. Verifica se la collezione corrente contiene tutti gli elementi della collezione `c`.

Conversione in Array Nonostante la potenza delle collezioni, a volte è necessario interfacciarsi con API legacy che richiedono array classici.

- `Object[] toArray()`: Restituisce un array di oggetti.
- `<T> T[] toArray(T[] a)`: Versione tipizzata. Se l'array `a` è abbastanza grande, i dati vengono inseriti lì; altrimenti, viene creato un nuovo array dello stesso tipo.

Metodi Default e Approccio Funzionale (Java 8+) Con l'introduzione delle lambda, l'interfaccia si è arricchita di metodi **default** molto potenti:

- `boolean removeIf(Predicate<? super E> filter)`: Rimuove tutti gli elementi che soddisfano una determinata condizione. È molto più efficiente e pulito di un ciclo manuale con iteratore.
- `Stream<E> stream()`: Il punto d'ingresso per la *Stream API*, essenziale per trasformazioni e filtraggi complessi in stile dichiarativo. Tratteremo gli *Stream* in un capitolo successivo.



Rischio di Errori a Runtime per Parametri Legacy

Un dettaglio tecnico sottile ma cruciale riguarda il tipo dei parametri di alcuni metodi fondamentali come `contains`, `remove` e le operazioni bulk. Sebbene la

⚠ collezione sia generica (`Collection<E>`), questi metodi accettano parametri di tipo `Object` o `Collection<?>`.

Il *Java Collections Framework* è stato progettato prima dell'introduzione dei tipi generici. Per mantenere la compatibilità con il codice legacy, i parametri di `contains` e `remove` sono rimasti di tipo `Object`. Questo significa che il compilatore non segnalerà errori se si tenta di cercare o rimuovere un oggetto di tipo errato.

Listing 8.3: Esempio di errore di tipo non rilevato

```
1  Collection<Path> paths = ...;
2  // Errore logico: cerco di rimuovere una String da una
   collezione di Path
3  paths.remove("/tmp"); // Compila correttamente, ma non
   avra' alcun effetto
```

8.2.1 Il Concetto di Operazioni Opzionali

Quasi tutti i metodi che abbiamo elencato non sono di default e ciò potrebbe indurre a pensare che *tutte* le collezioni concrete in Java implementino tutti questi metodi. Tuttavia, questo può essere fuorviante: il framework Java gestisce la varietà delle strutture dati (es. collezioni a dimensione fissa o non modificabili) attraverso il concetto di **metodi opzionali**.

Invece di creare decine di sotto-interfacce, i progettisti di Java hanno preferito mantenere l'interfaccia `Collection` unica. Se una specifica implementazione non supporta un metodo (ad esempio il metodo `add` in una collezione a sola lettura), il metodo viene comunque implementato, ma il suo corpo consiste nel lancio di una eccezione:

```
1 public boolean add(E e) {
2     throw new UnsupportedOperationException();
3 }
```

⚠ Attenzione a Runtime

Poiché l'obbligo di implementazione è soddisfatto formalmente, il compilatore non segnalerà errori. L'errore emergerà solo a **runtime**. Questo è un motivo in più per cui è fondamentale conoscere bene la documentazione della classe concreta che si sta istanziando.

8.3 Iteratori

Per scorrere gli elementi di una collezione, il Java Collections Framework non si affida agli indici (che sarebbero inefficienti per strutture come le liste concatenate), ma utilizza l'interfaccia `Iterator<E>`; essa definisce il meccanismo standard per visitare gli elementi di una collezione uno alla volta. A livello astratto, si può immaginare l'iteratore come un cursore posizionato **tra** gli elementi (si veda la figura 8.2).

L'interfaccia `Iterator` definisce quattro metodi principali:

```
1 public interface Iterator<E> {
2     boolean hasNext();
3     E next();
```

```

4  default void remove();
5  default void forEachRemaining(Consumer<? super E> action);
6  }

```

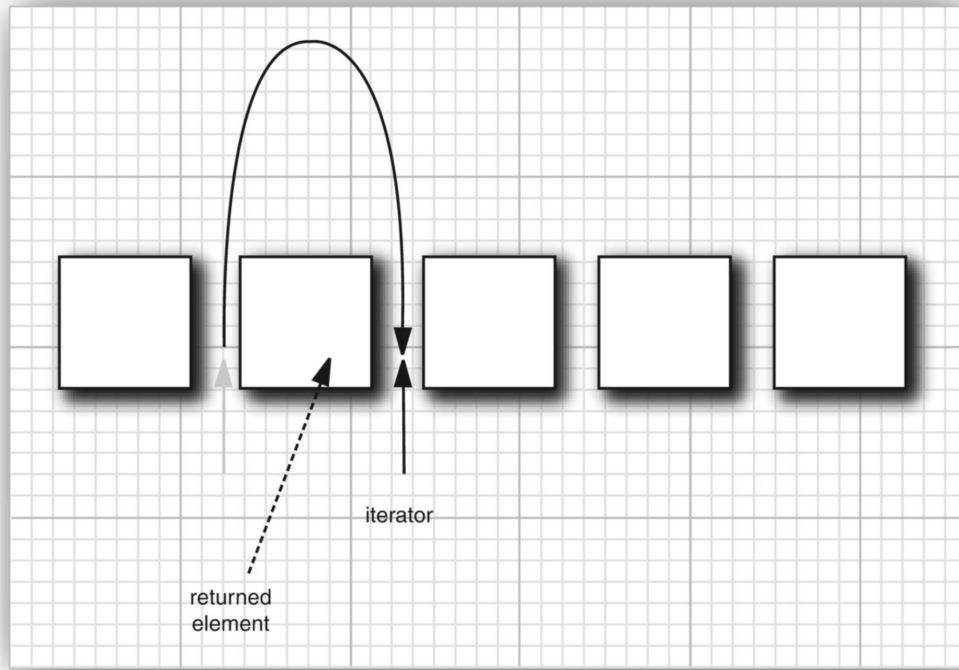


Figura 8.2: l'iteratore come cursore **tra** elementi

- `boolean hasNext()`: Restituisce `true` se ci sono ancora elementi da scorrere.
- E `next()`: Chiamando questo metodo, il cursore "salta" oltre l'elemento successivo e restituisce un riferimento all'elemento che ha appena saltato.
- `void remove()`: Cancella l'elemento restituito dall'ultima chiamata di `next`; esso è soggetto a vincoli: può essere chiamato solo dopo una chiamata a `next` (in concetto è: prima lo devo leggere, poi lo posso cancellare) e solo **una volta** dopo ogni chiamata a `next()` (non si può chiamare due volte di fila senza un `next` in mezzo). Se uno di questi vincoli non viene rispettato, si riceve una `IllegalStateException`.
- `void forEachRemaining(Consumer<? super E> action)`: Esegue un'azione su tutti gli elementi rimanenti.



Da notare che nel codice dell'interfaccia i metodi `remove()` e `forEachRemaining()` sono segnati come default. Questo significa che chi implementa l'interfaccia non è obbligato a scriverne il corpo se non ne ha bisogno.

Nello specifico, `remove()` è un metodo "opzionale": alcune collezioni (come quelle non modificabili) potrebbero lanciare una `UnsupportedOperationException` se provi a usarlo.

Come si vede dalla figura 8.1, `Iterable` rappresenta l'interfaccia padre di `Collection`; ogni classe che la implementa dichiara di poter restituire i propri elementi uno alla volta. Essa contiene un solo metodo astratto e non di default che restituisce un iteratore:

```
1 public interface Iterable<E> {
2     Iterator<E> iterator();
3 }
```

Questo metodo viene dunque ereditato da `Collection`; per ispezionare tutti gli elementi di una collezione, si richiede un iteratore e si richiama ripetutamente il metodo `next()`. Se si chiama `next()` quando si è già raggiunta la fine della collezione, l'iteratore solleva una `NoSuchElementException`. Per questo motivo, è fondamentale verificare sempre la presenza di un elemento successivo tramite il metodo `hasNext()`.

Listing 8.4: Utilizzo classico del ciclo while con Iterator

```
1 Collection<String> coll = ...;
2 Iterator<String> iter = coll.iterator();
3 while (iter.hasNext()) {
4     String element = iter.next();
5     // operazione sull'elemento
6 }
```

Java permette di scrivere il ciclo precedente in modo molto più conciso grazie al costrutto **for each**. Il compilatore traduce automaticamente questo costrutto in un ciclo con iteratore.

Listing 8.5: Sintassi for each

```
1 for (String element : coll) {
2     // operazione sull'elemento
3 }
```

Q Dettagli

```
1 // Scritto dallo sviluppatore:
2 for (String s : coll) { System.out.println(s); }
3
4 // Tradotto dal compilatore:
5 Iterator<String> it = coll.iterator();
6 while (it.hasNext()) {
7     String s = it.next();
8     System.out.println(s);
9 }
```

Il ciclo "for each" può essere utilizzato con qualsiasi oggetto che implementi l'interfaccia `Iterable`.



Poiché l'interfaccia `Collection` estende `Iterable`, ogni collezione del framework Java può essere scorsa usando la sintassi *for each*. Prediligi sempre questa sintassi per la leggibilità, a meno che tu non abbia bisogno di rimuovere elementi durante

✓ l'iterazione.

⚠ Effetti Collaterali

Tieni presente che durante un `forEach`, non puoi modificare la struttura della collezione (es. aggiungere o rimuovere elementi), altrimenti Java lancerà una `ConcurrentModificationException`.

Oltre ai cicli tradizionali, il *Collections Framework* mette a disposizione metodi che accettano **lambda expressions** per elaborare gli elementi in modo più conciso e leggibile. Invece di gestire manualmente l'iteratore, possiamo passare un'azione (sotto forma di lambda) direttamente alla collezione o all'iteratore stesso.

Listing 8.6: Esempi di iterazione funzionale

```
1 // Applica l'azione a ogni elemento della collezione
2 myCollection.forEach(element -> System.out.println(element));
3
4 // Applica l'azione agli elementi rimanenti dell'iteratore
5 myIterator.forEachRemaining(element -> System.out.println(element
  ));
```

Entrambi questi metodi utilizzano l'interfaccia funzionale `Consumer<? super E>`. La lambda viene invocata internamente per ogni elemento della collezione senza che lo sviluppatore debba preoccuparsi di chiamare `next()` o `hasNext()`.

Per quanto riguarda l'ordine in cui gli elementi vengono visitati, il comportamento dipende esclusivamente dal tipo di collezione; ad esempio, per un `ArrayList` l'iteratore è **predicibile**: inizia dall'indice 0 e incrementa di uno a ogni passo; per un `HashSet`, l'iteratore visita gli elementi in ordine **essenzialmente casuale** e non si può fare alcuna assunzione sulla sequenza di uscita.

Se in una collezione l'iteratore è predicibile, si dice che ha un **encounter order** (ordine di incontro).

💡 Scelta della Collezione

Se l'ordine dei dati è irrilevante (es. sommare dei totali o contare occorrenze), l'incertezza sull'ordine di visita non è un problema. Se invece l'ordine è parte della logica di business (es. una lista di transazioni cronologiche), è vitale scegliere una collezione che garantisca l'ordinamento come `ArrayList` o `TreeSet`.

8.3.1 L'Interfaccia `ListIterator`

Mentre `Iterator` permette solo lo scorrimento in avanti, **`ListIterator`** è una specializzazione per le liste che permette la navigazione bidirezionale e la modifica della struttura durante l'iterazione. **Implementazioni:** È fornita da tutte le classi della gerarchia `List` (`ArrayList`, `LinkedList`, `Vector`)

⚠ L'importanza in `LinkedList`

Metodo	Descrizione
<code>next()</code> / <code>previous()</code>	Sposta il cursore e restituisce l'elemento saltato.
<code>hasNext()</code> / <code>hasPrevious()</code>	Verifica la presenza di elementi nelle due direzioni.
<code>add(e)</code>	Inserisce un elemento nella posizione del cursore.
<code>set(e)</code>	Sostituisce l'ultimo elemento restituito da <code>next/previous</code> .
<code>nextIndex()</code> / <code>previousIndex()</code>	Restituisce l'indice dell'elemento che verrebbe restituito.

Tabella 8.1: Metodi principali di `ListIterator`

In una `LinkedList`, l'unico modo efficiente ($O(1)$) per inserire o rimuovere ripetutamente elementi nel mezzo della lista è posizionarsi con un `ListIterator` nella posizione desiderata. Senza di esso, ogni operazione di accesso richiederebbe uno scorrimento lineare $O(n)$ dalla testa della lista ogni singola volta.

**Tip**

Capita raramente di usare `previous()`, ma non è così raro usare `ListIterator` per trasformare una lista mentre la si scorre. L'iteratore standard permette solo di rimuovere (`remove()`), mentre il `ListIterator` permette di sostituire un elemento (`set()`) o aggiungerne uno nuovo (`add()`) esattamente in quel punto, senza mandare in crash la collezione con una `ConcurrentModificationException`.

**Tip**

Sebbene per `ArrayList` sia possibile sostituire elementi tramite indice senza causare eccezioni, l'uso di `ListIterator` rimane fondamentale per ragioni di Sicurezza Strutturale: Permette di aggiungere (`add()`) o rimuovere elementi durante l'iterazione. Mentre `myList.add()` causerebbe una `ConcurrentModificationException` in un ciclo *for-each*, `myIterator.add()` aggiorna correttamente lo stato dell'iteratore.

8.3.2 Sicurezza e Iterazione: Perché l'Indice è Pericoloso

La scelta del ciclo per scorrere una collezione non è solo una questione di stile o di "zucchero sintattico", ma una decisione ingegneristica che impatta sulla robustezza del codice.

L'utilizzo di un indice (`for(int i=0; i < list.size(); i++)`) è considerato **pericoloso** se la lista viene modificata durante il ciclo.

- **Problema:** Quando si rimuove l'elemento all'indice i , tutti gli elementi successivi scalano a sinistra. Al prossimo incremento ($i++$), il ciclo salterà l'elemento che è appena scalato nella posizione i .
- **Comportamento:** Il codice non crasha, ma produce risultati errati e imprevedibili (*silent failure*).

Nonostante la superiorità dell'iteratore per la sicurezza, il ciclo `for` con indice rimane lo strumento d'elezione in scenari specifici.

1. **Sincronizzazione di più Collezioni:** Quando è necessario accedere a due o più strutture dati simultaneamente utilizzando la stessa posizione relativa.
2. **Calcoli basati sulla Posizione:** Quando l'indice i è un parametro della formula matematica o della logica di business.

3. **Accesso Non-Sequenziale:** Algoritmi che richiedono salti ($i += k$), scansioni a ritroso o partizionamento (es. *Binary Search*, *QuickSort*).
4. **Prestazioni su Primitivi:** Negli array di tipi primitivi, evita l'overhead di creazione degli oggetti *Iterator* e favorisce le ottimizzazioni della CPU.

 Tip

Per scorrere qualsiasi tipo di struttura dati, prediligi l'uso di iteratori oppure, per la sola lettura, *enhanced for*.

Usa il ciclo con indice come un **bisturi**: quando la struttura dati è un array semplice o quando la logica dell'algoritmo non è una semplice "visita" di tutti gli elementi, ma una manipolazione basata sulla loro coordinata spaziale.

8.4 Le Classi Astratte "Ponte"

Per evitare che gli sviluppatori debbano scrivere decine di metodi ogni volta che creano una nuova collezione, il JDK fornisce delle classi astratte chiamate **Abstract Skeleton Implementations**:

- `AbstractCollection`
- `AbstractList`
- `AbstractSet`

Queste classi implementano già la maggior parte dei metodi (come `containsAll`, `removeAll`, `toString`) basandosi sui metodi fondamentali `iterator` e `size`.

 Tip

Se un giorno dovessi creare una tua collezione personalizzata, non implementare direttamente l'interfaccia `Collection`. Estendi invece `AbstractCollection`: dovrai implementare solo `iterator()` e `size()`, e avrai tutti gli altri metodi (compresi quelli bulk) già pronti e funzionanti.

Per capire come queste classi riescano a implementare metodi complessi senza sapere come sono memorizzati i dati, osserviamo come `AbstractCollection` implementa il metodo `contains(Object o)`.

Listing 8.7: Logica di `AbstractCollection.contains`

```
1 public boolean contains(Object o) {
2     Iterator<E> it = iterator(); // Metodo astratto!
3     if (o == null) {
4         while (it.hasNext())
5             if (it.next() == null) return true;
6     } else {
7         while (it.hasNext())
8             if (o.equals(it.next())) return true;
9     }
10    return false;
11 }
```

Osservazione 8.1. `AbstractCollection` non ha idea se la collezione sia un array o una lista concatenata. Tuttavia, sa che ogni collezione **deve** fornire un `Iterator`. Usando quel-

l'unico metodo (che rimane astratto nella classe `skeleton`), può fornire un'implementazione funzionante di `contains`, `toArray`, `toString` e molti altri.

Quando una classe concreta estende `AbstractCollection`, il carico di lavoro dello sviluppatore è ridotto, ma non nullo.

- **Metodi Obbligatori:** La classe concreta deve fornire le implementazioni di `iterator()` e `size()`.
- **Collezioni Mutabili:** Se la collezione deve permettere la modifica, è necessario implementare anche il metodo `add()`. In caso contrario, il metodo `add()` ereditato lancerà una `UnsupportedOperationException`.
- **Ottimizzazione:** Sebbene `AbstractCollection` fornisca già le implementazioni di molti metodi concreti, la classe figlia è libera di sovrascriverli se può offrire un'implementazione più efficiente (ad esempio, sfruttando un indice o una tabella hash interna per evitare lo scorrimento lineare).

Q Dettagli

L'approccio basato sulle "classi ponte" è oggi considerato parzialmente superato dal punto di vista del design. Sarebbe stato preferibile avere questi metodi direttamente nell'interfaccia `Collection` come metodi `default`, ma questa funzionalità non esisteva quando il framework fu progettato.

Tuttavia, nelle versioni recenti sono stati aggiunti diversi metodi `default`:

- **Stream API:** Metodi per l'elaborazione dei dati (trattati nel Volume II).
- **Conversione:** Metodi `toArray` più flessibili.
- **Rimozione Condizionale:** Il metodo `removeIf`.

8.5 Interfacce della Gerarchia `Collection`

Esaminiamo ora alcune specifiche interfacce della parte del framework che si dirama da `Collection`.

8.5.1 L'Interfaccia `List`

Una `List` è una collezione ordinata in cui ogni elemento ha una posizione precisa. L'accesso agli elementi può avvenire in due modi:

1. **Accesso Sequenziale:** Utilizzando un iteratore per visitare gli elementi uno dopo l'altro.
2. **Accesso Casuale (Random Access):** Utilizzando un indice intero per saltare direttamente a una posizione specifica (come in un array).

Per l'accesso casuale, l'interfaccia definisce metodi specifici per manipolare la lista tramite indici:

- `void add(int index, E element)`: Inserisce un elemento in una posizione specifica.
- `E remove(int index)`: Rimuove e restituisce l'elemento alla posizione indicata.
- `E get(int index)`: Restituisce l'elemento alla posizione indicata.
- `E set(int index, E element)`: Sostituisce l'elemento in una posizione e restituisce quello precedente.

⚠ L'ambiguità di `remove` con `List<Integer>`

Esiste un'insidia comune quando si usa una lista di interi (`List<Integer>`). Esistono due versioni del metodo `remove`:

- `remove(int index)`: Rimuove l'elemento all'indice specificato.
- `boolean remove(Object o)`: Rimuove la prima occorrenza dell'oggetto specificato.

Se passi un valore `int`, Java sceglierà **sempre** la versione che accetta l'indice, senza eseguire l'autoboxing a `Integer`. Per rimuovere l'elemento con valore 0 (e non l'elemento all'indice 0), devi scrivere: `list.remove(Integer.valueOf(0))`.

🔍 Dettagli

Horstmann evidenzia un difetto strutturale: l'interfaccia `List` raggruppa collezioni basate su array (veloci nell'accesso casuale) e collezioni basate su liste concatenate (lente nell'accesso casuale). Probabilmente sarebbe stato meglio usare due interfacce diverse per i due tipi.

Per permettere agli algoritmi di distinguere queste due tipologie a runtime, Java 1.4 ha introdotto l'interfaccia `RandomAccess`. Si tratta di un'*interfaccia marcatore* (*tagging interface*) che non contiene metodi e serve solo a "etichettare" classi come `ArrayList` per segnalare che supportano l'accesso a tempo costante. È possibile testarla con l'operatore `instanceof`:

```

1      if (list instanceof RandomAccess) {
2          // Posso usare un ciclo for classico con get(i)
3          //      -> O(1)
4      } else {
5          // Meglio usare un iteratore o for-each -> O(n)
6          //      complessivo
7      }

```

8.5.2 L'Interfaccia Set (Insiemi)

L'interfaccia `Set<E>` definisce una collezione che non ammette duplicati. Sebbene dichiari gli stessi metodi di `Collection`, la distinzione è puramente **concettuale** e **semantica** (cambia il *contratto*):

1. Permette al programmatore di scrivere metodi che accettano esplicitamente *solo* insiemi, forzando l'unicità dei dati a livello di tipo.
2. Definisce un criterio di uguaglianza diverso da quello della `List`: per un set l'ordine è un dettaglio irrilevante per l'identità della collezione.

Per oggetti di tipo `Set` devono quindi valere:

- **Rifiuto dei duplicati**: Il metodo `add(E element)` deve garantire che l'elemento venga aggiunto solo se non è già presente. Se l'elemento esiste già, il metodo restituisce `false` e non modifica la collezione.
- **Uguaglianza (`equals`)**: Due set sono considerati uguali se contengono gli stessi elementi, indipendentemente dall'ordine in cui sono stati inseriti.
- **Coerenza Hash**: Il metodo `hashCode()` deve essere implementato in modo che due set con gli stessi elementi generino lo stesso codice hash.

 Il test di uguaglianza

Il metodo `equals` di una classe che implementa `Set` deve verificare che l'oggetto passato come argomento sia a sua volta un `Set`. Non è sufficiente che sia una `Collection` generica; questo protegge dalla possibilità che un `Set` e una `List` (che seguono contratti di uguaglianza diversi) vengano erroneamente considerati uguali.

8.5.3 SequencedCollection (Java 21+)

A partire da Java 21, il framework introduce l'interfaccia `SequencedCollection<E>` per fornire un accesso uniforme al primo e all'ultimo elemento di una collezione che possiede un ordine di incontro (*encounter order*), cioè per cui un iteratore sia **predicibile**.

Prima di questa interfaccia, le operazioni sulla testa e sulla coda della collezione erano frammentate in metodi diversi tra `List`, `Deque` e `SortedSet`. `SequencedCollection` raggruppa queste operazioni sotto un unico contratto.

Listing 8.8: Metodi di `SequencedCollection`

```
1 interface SequencedCollection<E> extends Collection<E> {
2     // Accesso e modifica
3     void addFirst(E e);
4     void addLast(E e);
5     E getFirst();
6     E getLast();
7     E removeFirst();
8     E removeLast();
9
10    // Vista invertita
11    SequencedCollection<E> reversed();
12 }
```

Osservazione 8.2. Il metodo `reversed()` non crea una copia della collezione, ma fornisce una **vista** in ordine inverso. Qualsiasi modifica alla collezione originale si riflette nella vista e viceversa, con un'efficienza di $O(1)$.

L'interfaccia `SequencedSet<E>` estende sia `Set` che `SequencedCollection`.

- È implementata da classi che mantengono l'ordine di inserimento (es: `LinkedHashSet`) o l'ordine di ordinamento (es: `TreeSet`).
- Il metodo `reversed()` in questa interfaccia restituisce un `SequencedSet` anziché una `Collection` generica, permettendo di mantenere le proprietà di unicità anche nella vista invertita.

 Se devi scrivere un metodo che accetta qualsiasi cosa sia "ordinata" (lista o set ordinato), considera `SequencedCollection` come tipo del parametro. Questo potrà rendere il codice molto più flessibile.

Da `SequencedSet` si diramano inoltre versioni più sofisticate dell'interfaccia `Set` che vengono utilizzate quando non ci si accontenta dell'unicità, ma si ha bisogno che gli elementi siano mantenuti in un ordine specifico.

- **SortedSet<E>**: Questa interfaccia espone il **Comparator** utilizzato per l'ordinamento e definisce metodi per ottenere "viste" di sotto-insiemi della collezione (es. tutti gli elementi minori di un certo valore).
- **NavigableSet<E>**: Estende **SortedSet** aggiungendo metodi per la ricerca di elementi "vicini" (prossimo o precedente) rispetto a un valore dato.

Osservazione 8.3. Queste interfacce sono implementate dalla classe **TreeSet** che, di fatto, non è altro che un albero binario di ricerca bilanciato (nello specifico, *Red-Black*)

8.5.4 Queue e Deque

Le code sono fondamentali per la gestione di flussi di dati in cui l'ordine di elaborazione è critico.

La **Queue<E>** è progettata per il processamento di elementi in ordine FIFO. L'approccio di Java per le code è molto pragmatico: ogni operazione ha due varianti, una lancia un'eccezione se l'operazione fallisce per effetto di una coda vuota o piena (alcune implementazioni hanno capacità limitata), l'altra restituisce un valore speciale (**null** o **false**).

I metodi **element()**/**peek()** restituiscono la testa della coda senza rimuoverla.

Operazione	Eccezione	Valore Speciale (null/false)
Inserimento	add(e)	offer(e)
Rimozione	remove()	poll()
Ispezione	element()	peek()

Tabella 8.2: Metodi dell'interfaccia Queue

Tip

Molte librerie di messaggistica (come RabbitMQ o le code interne di Spring) usano questi contratti. Sapere che **poll()** non ti manderà in crash l'applicazione se la coda è vuota (a differenza di **remove()**) è la base per scrivere codice robusto che non "esplode" sotto carico.

La **Deque<E>** (Double-Ended Queue) permette l'accesso ad entrambi i capi della collezione. Eredita da **Queue** e aggiunge versioni **First** e **Last** per ogni metodo (es. **addFirst**, **pollLast**).

Deque come Stack

In Java, la classe **Stack** è considerata obsoleta. Se hai bisogno di una struttura LIFO (Last-In-First-Out), il Java Collections Framework raccomanda l'uso di una **Deque**:

- **push(e)** equivale a **addFirst(e)**.
- **pop()** equivale a **removeFirst()**.

Osservazione 8.4. Con l'introduzione di Java 21, **Deque** estende, oltre a **Queue**, anche **SequencedCollection**, rendendo i suoi metodi **getFirst**, **getLast**, ecc., coerenti con il resto del framework.

8.6 Classi Collection Concrete

In questo paragrafo abbandoniamo l'astrazione delle interfacce per analizzare come Java memorizza fisicamente i dati. La scelta della classe concreta influisce drasticamente sulla complessità algoritmica e sull'efficienza della memoria.

Abbiamo già accennato ad alcune di queste classi, qui riepiloghiamo quanto detto e completiamo con ulteriori classi non ancora menzionate.

8.6.1 Il Motore ad Array (Contiguità Fisica)

Le classi in questo gruppo utilizzano un array interno. La caratteristica principale è la **contiguità della memoria**, che favorisce la velocità di accesso e l'efficienza della cache della CPU. Sia `ArrayList` che `ArrayDeque` si basano su un array che viene ridimensionato quando la capacità massima viene raggiunta. Quando l'array interno è pieno, Java ne alloca uno nuovo più grande e vi sposta gli elementi. Sebbene la singola operazione di copia sia $O(n)$, essa avviene così raramente che, spalmata su migliaia di inserimenti, il costo medio (costo **ammortizzato**) rimane $O(1)$.

ArrayList

È l'implementazione di `List` più utilizzata. Quando la capacità massima è raggiunta, cresce solitamente del 50% della capacità corrente.

- **Accesso Casuale:** $O(1)$ tramite indice.
- **Inserimento/Rimozione:** $O(n)$ nel caso peggiore (perché deve "spostare" gli elementi), ma l'aggiunta in coda ha un costo **ammortizzato** di $O(1)$.

ArrayDeque

Introdotta per superare i limiti di `Stack` e `LinkedList`, è una coda a doppia entrata basata su un array circolare. Quando la capacità massima è raggiunta, cresce solitamente del 100% della capacità corrente.



Best Practice

È la scelta più efficiente per implementare sia **Pile** (Stack) che **Code** (Queue). È sensibilmente più veloce di `LinkedList` perché non deve creare un oggetto "nodo" per ogni elemento inserito.

8.6.2 Il Motore a Nodi: LinkedList

Non è altro che una lista doppiamente concatenata (*doubly linked list*): ogni elemento è memorizzato in un nodo che contiene un riferimento al nodo precedente e a quello successivo.

Nonostante sia un classico accademico, nel Java moderno `LinkedList` è raramente la scelta migliore:

1. **Cache Miss:** I nodi sono sparsi nella RAM, obbligando la CPU a continui e lenti recuperi dalla memoria principale.
2. **Overhead:** Ogni nodo richiede memoria extra per due puntatori (`next` e `prev`).
3. **Accesso:** Anche per le operazioni che sembrano $O(1)$, bisogna spesso scorrere la lista per raggiungere il punto desiderato ($O(n)$).

Se `ArrayList` è quasi sempre superiore, quando dovremmo preferire `LinkedList`? La risposta risiede nel costo delle operazioni strutturali durante l'iterazione.

- **Modifiche durante lo scorrimento:** Se il workflow prevede di scorrere una lista con un `ListIterator` e inserire/rimuovere elementi frequentemente nel mezzo, `LinkedList` opera in $O(1)$, mentre `ArrayList` rimane legata a un costo $O(n)$ per lo spostamento dei dati.
- **Assenza di picchi di latenza:** In `LinkedList` non esiste il concetto di "ridimensionamento". Ogni inserimento ha lo stesso costo costante, evitando il picco $O(n)$ tipico della copia degli array dinamici. Questo può essere rilevante se la struttura dati subisce numerosi ridimensionamenti o se si desidera una latenza più prevedibile.



Attenzione: per inserire un elemento in una posizione specifica i , `LinkedList` deve comunque **trovare** quel punto scorrendo i nodi ($O(n)$). Il vantaggio $O(1)$ esiste **solo** se l'iteratore si trova già sulla posizione corretta.

In conclusione, nella stragrande maggioranza delle applicazioni backend (web server, elaborazione dati, microservizi), i benefici della **Cache Locality** dell'array superano di gran lunga i vantaggi teorici dei puntatori. **Usa `ArrayList` di default.**

8.6.3 Il Motore Hash (Tabelle Hash)

Per richiami sulle tabelle hash in Java, si veda [A.2](#)

- **HashSet:** Implementazione standard di `Set` che non garantisce alcun ordine di iterazione. **Prestazioni:** $O(1)$ per ricerca, inserimento e rimozione (se la funzione hash è buona).
- **LinkedHashSet:** Variante di `HashSet` che mantiene una lista doppiamente concatenata tra gli elementi, combinando la velocità del `HashSet` con la capacità di ricordare l'ordine di inserimento. Dunque l'iterazione è predicibile e segue l'ordine in cui gli elementi sono stati aggiunti. È ideale per cache o sistemi dove l'ordine temporale è importante tanto quanto l'unicità.



Mutabilità degli Elementi e Invarianza dell'Hash

Presta estrema attenzione alla modifica (*mutazione*) dello stato degli oggetti dopo averli inseriti in un `Set` (o come chiavi in una `HashMap`).

Se i campi modificati partecipano al calcolo del metodo `hashCode()`, il valore hash dell'oggetto cambierà a seguito della mutazione. Di conseguenza, l'elemento rimarrà fisicamente intrappolato nel bucket associato al suo **vecchio** hash code, ma qualsiasi successiva operazione di ricerca (`contains`), rimozione (`remove`) o inserimento calcolerà l'indice basandosi sul **nuovo** valore hash.

Questo comportamento corrompe l'integrità della struttura dati: l'oggetto modificato diventerà virtualmente "invisibile" alle successive query della mappa o del set, causando fallimenti logici silenziosi (*silent failures*) ed errori di business incredibilmente difficili da tracciare a runtime. Come regola generale, gli elementi inseriti nelle tabelle hash dovrebbero preferibilmente essere **immutabili**.

8.6.4 EnumSet: Prestazioni a Livello di Bit

`EnumSet` è un'implementazione speciale di `Set` progettata per tipi enumerativi. Non è una classe pubblica con costruttori, ma si ottiene tramite metodi factory statici.

- **Meccanismo Interno:** Utilizza un **bit-vector**. Se l'enum ha ≤ 64 elementi (caso comunissimo), il set è memorizzato in un singolo `long`.
- **Efficienza:** Le operazioni sono tradotte in istruzioni bitwise della CPU. È la collezione più veloce in assoluto per i tipi `enum`, con un consumo di memoria minimo.
- **Null-Safety:** Non permette l'inserimento di elementi `null`.

Listing 8.9: Esempio di `EnumSet`

```

1 // Creazione di un set con elementi specifici
2 Set<Day> weekend = EnumSet.of(Day.SATURDAY, Day.SUNDAY);
3
4 // Creazione di un set con tutti gli elementi dell'enum
5 Set<Day> allDays = EnumSet.allOf(Day.class);

```

Osservazione 8.5. L'ordine di iterazione in un `EnumSet` non dipende dall'ordine di inserimento, ma dall'**ordine di dichiarazione** delle costanti nell'Enum originale.

✓ Best Practice

In passato, per gestire i "flag" (es. permessi READ, WRITE, EXECUTE), si usavano costanti intere e maschere di bit manuali ($2^0, 2^1, 2^2$). In Java moderno, `EnumSet` offre la stessa efficienza dei bit ma con la sicurezza dei tipi (Type Safety) dell'orientamento agli oggetti.

8.6.5 Il Motore ad Albero: `TreeSet`

Come già discusso nei paragrafi precedenti, utilizza un albero Red-Black per mantenere gli elementi costantemente ordinati.

- **Prestazioni:** $O(\log n)$.
- **Utilizzo:** Da preferire quando servono operazioni di "range" (es. estrarre tutti i valori tra x e y) o quando l'ordinamento naturale è un requisito.

8.6.6 Il Motore a Heap: `PriorityQueue`

Rappresenta una *coda a priorità* implementata tramite un array dinamico che funge da *Heap* (per entrambi i concetti, si vedano in appendice i richiami A.4 e A.5).

- **Criterio di Priorità:** Se non viene fornito un `Comparator`, gli elementi devono implementare `Comparable` (ordinamento naturale).
- **Min-Priority di default:** In Java, il valore più piccolo è quello con priorità più alta (testa della coda). Per una Max-Priority Queue, occorre invertire il comparatore: `Collections.reverseOrder()`.

L'efficienza della `PriorityQueue` deriva direttamente dall'uso dello heap come struttura dati sottostante:

✓ Efficienza di Massa

Se devi trasformare una collezione di n elementi in una `PriorityQueue`, non fare n

Operazione	Complessità	Nota Tecnica
<code>peek()</code>	$O(1)$	La radice dello heap è sempre accessibile.
<code>offer()/add()</code>	$O(\log n)$	Richiede il "galleggiamento" (<i>sift-up</i>).
<code>poll()</code>	$O(\log n)$	Richiede l'affondamento (<i>sift-down</i>).
<code>remove(Object)</code>	$O(n)$	Attenzione: richiede la ricerca lineare.
<code>contains(Object)</code>	$O(n)$	Lo heap non è ottimizzato per la ricerca.
<code>heapify</code> (Costruttore)	$O(n)$	Algoritmo di Floyd (costruzione in massa).

Tabella 8.3: Complessità temporale della PriorityQueue

- ✓ inserimenti ($O(n \log n)$). Usa il costruttore che accetta una collezione: Java userà l'algoritmo di **Floyd** (*heapify*) che costruisce lo heap in tempo $O(n)$.

⚠ Gestione dei pareggi

A differenza di quanto si potrebbe pensare, la `PriorityQueue` di Java **non garantisce** un ordine FIFO per elementi con la stessa priorità. Se l'ordine di arrivo è cruciale in caso di parità, è necessario includere un "timestamp" o un contatore nel comparatore. Stesso discorso vale per l'ordine di iterazione.

⚠ Modifica degli elementi

Se un oggetto già presente nella coda viene modificato in modo da cambiare la sua priorità (il risultato di `compareTo` cambia), lo heap **non si riposiziona automaticamente**. Questo corrompe la struttura dati. In questi casi, l'elemento deve essere rimosso e reinserito.

8.7 Mappe

Dopo aver esaminato la parte di framework che si dirama da `Iterable` e `Collection`, procediamo in modo analogo per le mappe. Seguiremo anche qui un approccio top-down, partendo dall'alto della gerarchia e scendendo fino alle classi concrete.

8.7.1 L'Interfaccia Map

L'interfaccia `Map<K, V>` rappresenta una struttura dati che non fa parte della gerarchia `Collection`. Invece di memorizzare elementi singoli, una mappa gestisce associazioni tra **Chiavi** (Key) e **Valori** (Value). Il contratto fondamentale di una mappa stabilisce che le chiavi devono essere **uniche** (non sono ammessi duplicati), mentre i valori possono essere ripetuti. Di seguito, esaminiamo i metodi di uso più comune, raggruppati in parallelo con quelli delle collezioni.

Operazioni di Query (Ispezione) Questi metodi permettono di ottenere informazioni sullo stato della mappa e di cercare elementi senza modificarla. Notare che la ricerca è divisa tra chiavi e valori.

- `int size()`: Restituisce il numero di associazioni chiave-valore presenti.
- `boolean isEmpty()`: Equivalente a `size() == 0`.

- `boolean containsKey(Object key)`: Verifica se la mappa contiene la chiave specificata. È un'operazione estremamente veloce ($O(1)$ nella maggior parte delle implementazioni).
- `boolean containsValue(Object value)`: Verifica se la mappa contiene una o più istanze del valore specificato. Spesso richiede una ricerca lineare ($O(n)$).
- `V get(Object key)`: Restituisce il valore associato alla chiave, oppure `null` se la chiave non è presente.

Operazioni di Modifica A differenza delle collezioni, i metodi di modifica delle mappe restituiscono spesso il valore precedentemente associato alla chiave, fornendo un feedback immediato sulle sostituzioni.

- `V put(K key, V value)`: Inserisce una nuova associazione. Se la chiave era già presente, il vecchio valore viene sovrascritto e **restituito** dal metodo. Se la chiave è nuova, restituisce `null`.
- `V remove(Object key)`: Rimuove l'associazione per la chiave specificata e restituisce il valore che è stato appena rimosso (o `null` se non esisteva).
- `void clear()`: Rimuove tutte le associazioni, svuotando la mappa.

Operazioni Bulk (di Massa) Per le mappe, le operazioni di massa si concentrano principalmente sull'unione di dizionari diversi.

- `void putAll(Map<? extends K, ? extends V> m)`: **Unione**. Copia tutte le associazioni dalla mappa `m` alla mappa corrente. Se ci sono chiavi coincidenti, i valori della mappa `m` sovrascrivono quelli attuali.

Metodi Default e Approccio Funzionale (Java 8+) L'interfaccia `Map` ha beneficiato enormemente dell'introduzione dei metodi `default`, che hanno snellito drasticamente il codice per la gestione dei valori nulli e delle operazioni condizionali.

- `V getOrDefault(Object key, V defaultValue)`: Se la chiave esiste, restituisce il suo valore; altrimenti restituisce `defaultValue` senza inserirlo nella mappa. Elimina la necessità di continui controlli `if (map.containsKey(...))`.
- `V putIfAbsent(K key, V value)`: Inserisce la coppia solo se la chiave non è già mappata a un valore (o se è mappata a `null`).
- `void forEach(BiConsumer<? super K, ? super V> action)`: Permette di iterare su chiavi e valori in modo funzionale e pulito tramite una lambda, ad esempio:

```
map.forEach((k, v) -> System.out.println(k + ":" + v))
```

- `V computeIfAbsent(K key, Function<? super K, ? extends V> mF)` (`mF` sta per "mapping function"): Molto usato per costruire mappe di collezioni o cache. Se la chiave manca, esegue la funzione per calcolare il valore, lo inserisce e lo restituisce in un solo colpo.

8.7.2 Iterazione e le "Viste" della Mappa

Una delle differenze architetturali più marcate tra le Collezioni e le Mappe riguarda l'iterazione. L'interfaccia `Map` **non** estende `Iterable`. Di conseguenza, una mappa non possiede un iteratore proprio e il compilatore non permette di utilizzare il costrutto *for-each* direttamente sull'oggetto mappa.

Il motivo di questa scelta progettuale è semantico: un iteratore è un cursore monodimensionale (scorre una riga di elementi), mentre una mappa è una struttura bidimensionale (associa chiavi a valori). Per scorrere i dati di una mappa, è necessario prima chiederle di proiettare la sua struttura bidimensionale in una forma monodimensionale. Questa proiezione prende il nome di **Vista** (*View*).

Esistono tre viste fondamentali, tutte supportate dalla mappa originale (significa che se rimuovi un elemento dalla vista, questo scompare anche dalla mappa):

- `Set<K> keySet()`: Restituisce l'insieme delle chiavi. Essendo chiavi, la vista è rigorosamente un `Set`.
- `Collection<V> values()`: Restituisce l'insieme dei valori. Poiché i valori possono essere duplicati, la vista è una generica `Collection`.
- `Set<Map.Entry<K, V> entrySet()`: Restituisce un `Set` di oggetti `Map.Entry`. Questa è la vista più potente e utilizzata.

L'Interfaccia Map.Entry `Map.Entry<K, V>` è un'interfaccia nidificata staticamente all'interno dell'interfaccia `Map`. Il suo unico scopo è incapsulare una singola associazione (una coppia inseparabile chiave-valore) in un unico oggetto.

Espone tre metodi fondamentali:

- `K getKey()`: Restituisce la chiave della coppia.
- `V getValue()`: Restituisce il valore della coppia.
- `V setValue(V value)`: Sostituisce il valore attualmente associato alla chiave in questa specifica *entry*, aggiornando automaticamente la mappa sottostante.

Pattern di Iterazione Grazie alle viste, possiamo utilizzare i normali costrutti di iterazione visti per le collezioni.

Listing 8.10: Iterazione tradizionale con for-each

```

1 Map<String, Integer> anagrafica = ...;
2
3 // 1. Scorrere solo le chiavi
4 for (String nome : anagrafica.keySet()) {
5     System.out.println("Nome: " + nome);
6 }
7
8 // 2. Scorrere chiavi e valori (Approccio Consigliato)
9 for (Map.Entry<String, Integer> entry : anagrafica.entrySet()) {
10     System.out.println("Nome: " + entry.getKey() + ", Eta': " +
11         entry.getValue());
12 }
```

✓ L'Anti-Pattern del KeySet

Un errore comune tra i programmatori junior è quello di iterare sul `keySet()` e richiamare il metodo `get()` all'interno del ciclo per ottenere il valore:

```

1 // LENTO E INEFFICIENTE!
2 for (String key : mappa.keySet()) {
3     Integer value = mappa.get(key);
4     // Usa key e value
5 }
```



Questo approccio è inefficiente perché costringe la mappa a ricalcolare l'hash e a cercare fisicamente l'elemento ($O(1)$ o $O(\log n)$) a ogni iterazione. Iterando direttamente sull'`entrySet()`, invece, la JVM ti consegna la coppia già "confezionata", abbattendo a zero il costo di ricerca del valore.

Iterazione Funzionale (Java 8+) L'aggiunta dei metodi `default` ha introdotto un'alternativa sintattica estremamente pulita per scorrere le mappe, mascherando del tutto la necessità di estrarre manualmente l'`entrySet`. Il metodo `forEach` accetta un `BiConsumer` (un'azione che richiede due parametri di input).

Listing 8.11: Iterazione dichiarativa tramite Lambda

```
1 anagrafica.forEach((nome, eta) -> {
2     System.out.println("Nome: " + nome + ", Eta': " + eta);
3 });
```

Questo approccio è oggi considerato lo standard per la pura lettura sequenziale dei dati, mentre il ciclo *for-each* su `Map.Entry` rimane insostituibile quando è necessario modificare i valori tramite `setValue()` durante lo scorrimento o uscire anticipatamente dal ciclo tramite `break`.

8.7.3 Pattern Operativi di Aggiornamento

Una delle operazioni più delicate quando si lavora con le mappe è l'aggiornamento di un valore esistente, a causa del rischio intrinseco di incappare in chiavi non ancora registrate. La gestione manuale di queste casistiche produce codice verboso e pronò a `NullPointerException`. Per questo motivo, Java fornisce metodi funzionali progettati per risolvere specifici e frequentissimi scenari di business.

Pattern 1: Il Contatore (Frequenze e Statistiche) Uno scenario tipico è il conteggio delle occorrenze (es. contare quante volte una parola appare in un file). L'approccio ingenuo:

```
1 counts.put(word, counts.get(word) + 1); // PERICOLO: Se 'word' e'
    nuova, get() restituisce null
```

Per evitare l'eccezione, si potrebbe usare `getOrDefault` oppure `putIfAbsent`, ma Java offre una soluzione definitiva e concisa con il metodo `merge()`. Questo metodo inserisce un valore iniziale se la chiave non esiste, altrimenti applica una funzione per combinare il vecchio valore con quello nuovo:

Listing 8.12: Uso ottimale di `merge` per i contatori

```
1 // Se 'word' non esiste, inserisce 1.
2 // Se esiste, somma il valore precedente a 1 tramite Integer::sum
3 counts.merge(word, 1, Integer::sum);
```

Pattern 2: Il Multi-Dizionario (Mappe di Collezioni) Un altro scenario frequentissimo è l'associazione di molteplici valori a una singola chiave (es. un indice in cui a ogni "termine" corrisponde un set di "pagine" in cui compare). In questo caso, il tipo della mappa sarà `Map<String, Set<Integer>`.

Il modo migliore per aggiornare questa struttura è concatenare il metodo `computeIfAbsent()` con l'aggiunta alla collezione. Questo metodo, a differenza di `putIfAbsent`, è utilissimo perché restituisce sempre la collezione associata alla chiave (creandola al volo se mancante), permettendo di chiamare `.add()` nella stessa istruzione.

Listing 8.13: Costruzione di una Multimap con `computeIfAbsent`

```

1 var index = new TreeMap<String, TreeSet<Integer>>();
2 // ...
3 // Costruisce il TreeSet al volo solo se la parola e' nuova, poi
  // vi aggiunge la pagina
4 index.computeIfAbsent(term, k -> new TreeSet<>()).add(pageNumber)
  ;

```

⚠ La Trappola dei Method Reference nei Costruttori

Nel codice precedente, si potrebbe essere tentati di sostituire la lambda `k -> new TreeSet<>()` con il method reference `TreeSet::new`. **Questo è un errore logico pericoloso.**

Il metodo `computeIfAbsent` passa sempre la *chiave* come argomento alla funzione di creazione. Poiché `TreeSet` non ha un costruttore che accetta una `String`, il compilatore bloccherebbe il codice. Il vero pericolo si nasconde quando i tipi coincidono. Supponiamo di avere una `Map<Integer, ArrayList<String>` e di scrivere:

```

1 // GRAVE ERRORE LOGICO
2 mappa.computeIfAbsent(numero, ArrayList::new);

```

La classe `ArrayList` possiede un costruttore che accetta un `int` per definire la sua capacità iniziale. Invece di creare semplicemente una lista vuota, la JVM istanzerà un array interno con una capacità pari al valore della chiave `numero`, spreca un'enorme quantità di memoria in modo totalmente involontario.

🔍 L'incoerenza nella gestione dei valori null

Le API dell'interfaccia `Map` presentano un'incoerenza storica riguardo al trattamento dei valori `null`, che è bene tenere a mente per il debugging: alcune funzioni considerano `null` come un valore valido, altre lo trattano come equivalente a "chiave assente".

- **getOrDefault:** Appartiene al primo gruppo. Se una chiave è esplicitamente associata al valore `null`, il metodo restituirà `null` senza attivare il valore di default fornito.
- **putIfAbsent:** Appartiene al secondo gruppo. Se una chiave è presente ma il suo valore è `null`, il metodo considererà lo spazio "libero" e sovrascriverà il `null` con il nuovo valore.

8.7.4 La Classe Astratta "Ponte" per le Mappe

Esattamente come avviene per le collezioni, il JDK fornisce una **Abstract Skeleton Implementation** per le mappe, allo scopo di ridurre al minimo lo sforzo richiesto per implementare l'interfaccia `Map<K, V>`: la classe **AbstractMap**.

Questa classe fornisce un'implementazione di base per quasi tutti i metodi dell'interfaccia (come `containsKey`, `containsValue`, `get`, `isEmpty`), basando la propria logica su un unico metodo fondamentale che la classe concreta dovrà fornire: `entrySet()`.

Tip

Se un giorno dovessi creare una mappa personalizzata, non implementare direttamente l'interfaccia `Map`. Estendi invece **AbstractMap**: ti basterà fornire l'implementazione del metodo `entrySet()` per ereditare una mappa funzionante (in sola lettura).

Per capire il livello di astrazione di questa architettura, osserviamo come **AbstractMap** riesca a implementare un metodo complesso come `containsValue(Object value)` senza avere la minima idea di come la mappa memorizzi fisicamente le coppie:

Listing 8.14: Logica di `AbstractMap.containsValue`

```
1 public boolean containsValue(Object value) {
2     Iterator<Map.Entry<K,V>> i = entrySet().iterator(); // Metodo
      astratto!
3     if (value == null) {
4         while (i.hasNext()) {
5             Entry<K,V> e = i.next();
6             if (e.getValue() == null) return true;
7         }
8     } else {
9         while (i.hasNext()) {
10            Entry<K,V> e = i.next();
11            if (value.equals(e.getValue())) return true;
12        }
13    }
14    return false;
15 }
```

AbstractMap ignora se la struttura sottostante sia una tabella hash o un albero. Sfrutta semplicemente il fatto che ogni mappa **deve** poter restituire il set delle sue *Entry*. Iterando su quel set, può risolvere la ricerca del valore.

Quando una classe concreta estende **AbstractMap**, le regole di ingaggio cambiano leggermente rispetto alle collezioni generiche, soprattutto per una questione di **prestazioni**:

- **Metodi Obbligatori**: La classe figlia deve fornire l'implementazione di `entrySet()` (che deve restituire un set di istanze `Map.Entry`).
- **Mappe Mutabili**: Per permettere l'inserimento di nuovi dati, è necessario sovrascrivere il metodo `put(K key, V value)`, che altrimenti lancerà una `UnsupportedOperationException`.
- **Ottimizzazione (Critica)**: Sebbene **AbstractMap** fornisca già un'implementazione per `get(Object key)` e `containsKey(Object key)`, queste implementazioni di

default eseguono una **ricerca lineare** ($O(n)$) iterando su tutto l'`entrySet`. Poiché lo scopo primario di una mappa è l'accesso istantaneo tramite chiave, la classe figlia **deve assolutamente sovrascrivere** questi metodi sfruttando la propria struttura fisica (es. il calcolo dell'hash o la discesa nell'albero) per riportare le prestazioni a $O(1)$ o $O(\log n)$.

Q La Classe Dictionary: Un reperto fossile

Consultando la gerarchia originale di Java o alcuni vecchi diagrammi, è possibile imbattersi nella classe astratta **Dictionary**.

Si tratta di un artefatto obsoleto risalente a Java 1.0, precedente alla nascita del *Collections Framework*. Fu creata come superclasse per la vecchia struttura **Hashtable**. Nel codice Java moderno, **Dictionary** è considerata *deprecated* a livello concettuale: non deve **mai** essere estesa o utilizzata nei nuovi progetti. L'unica astrazione corretta per i dizionari di dati è l'interfaccia **Map** e la sua scheletro **AbstractMap**.

8.7.5 SequencedMap e Mappe Ordinate (Java 21+)

Prima di esplorare le interfacce specifiche, è utile fissare un concetto teorico fondamentale: a livello logico e matematico, **una mappa può essere pensata come una funzione definita su un dominio finito**. In questo parallelismo, il "dominio" della funzione è costituito dall'insieme delle chiavi. Poiché le chiavi non ammettono duplicati, questo dominio logico non è altro che un **Set**. Di conseguenza, il modo in cui una mappa mantiene (o non mantiene) l'ordine dei suoi elementi dipende fisicamente e logicamente dal tipo di set che funge da struttura portante per le sue chiavi.

Seguendo l'evoluzione introdotta con Java 21, il framework definisce l'interfaccia **SequencedMap<K, V>** per uniformare l'accesso alle estremità delle mappe dotate di un ordine di incontro (*encounter order*) predicibile (come **LinkedHashMap** o **TreeMap**).

Listing 8.15: Metodi principali di SequencedMap

```

1 interface SequencedMap<K, V> extends Map<K, V> {
2     // Accesso e modifica alle estremità
3     Map.Entry<K,V> firstEntry();
4     Map.Entry<K,V> lastEntry();
5     Map.Entry<K,V> pollFirstEntry();
6     Map.Entry<K,V> pollLastEntry();
7     V putFirst(K k, V v);
8     V putLast(K k, V v);
9
10    // Viste ordinate (covarianza dei tipi di ritorno)
11    SequencedSet<K> sequencedKeySet();
12    SequencedCollection<V> sequencedValues();
13    SequencedSet<Map.Entry<K,V>> sequencedEntrySet();
14
15    // Vista invertita
16    SequencedMap<K,V> reversed();
17 }
```

Osservazione 8.6. In perfetta simmetria con le collezioni, il metodo **reversed()** non clona la mappa, ma ne restituisce una **vista** speculare con efficienza $O(1)$. Inoltre, i metodi

che restituiscono le "viste" dei dati (es. `sequencedKeySet()`) sfruttano la covarianza per restituire strutture che implementano a loro volta le interfacce **Sequenced**, garantendo la corretta propagazione dell'ordine al dominio della mappa.

Esattamente come avviene per i **Set**, da **SequencedMap** si diramano interfacce più sofisticate, pensate per scenari in cui la mappa non deve solo ricordare l'ordine di inserimento, ma deve mantenere il dominio delle chiavi costantemente **ordinato secondo un criterio logico** (es. ordine alfabetico o numerico).

- **SortedMap<K, V>**: Lo specchio di **SortedSet**. Espone il **Comparator** utilizzato per l'ordinamento delle chiavi e fornisce metodi per ottenere "sotto-mappe" (viste parziali, come `subMap`, `headMap`, `tailMap`) basate su un range di chiavi.
- **NavigableMap<K, V>**: Estende **SortedMap** (in modo analogo a **NavigableSet**), aggiungendo metodi per la ricerca di prossimità. Permette di trovare la chiave o l'**Entry** "strettamente maggiore" (`higherEntry`) o "minore o uguale" (`floorEntry`) rispetto a una chiave target.

Osservazione 8.7. Queste interfacce avanzate sono implementate dalla classe concreta **TreeMap** che, analogamente a **TreeSet**, si appoggia a un *albero binario di ricerca bilanciato* (Red-Black Tree) per mantenere il set delle chiavi costantemente ordinato con un costo algoritmico di $O(\log n)$.

8.7.6 Classi Map Concrete

Dal punto di vista logico e matematico, come abbiamo stabilito, una mappa è una funzione definita su un **Set** (il suo dominio). Sarebbe quindi naturale dedurre che una **HashMap** sia strutturalmente costruita "utilizzando" un **HashSet**.

Nella realtà fisica del framework Java, tuttavia, accade l'**esatto opposto**. Per ragioni di ottimizzazione del codice, gli architetti del JDK hanno sviluppato prima i complessi motori fisici delle mappe, e hanno poi implementato i **Set** come semplici "involucri" (*wrapper*) attorno ad esse.

- **HashMap**: È il motore fisico reale (basato sull'array di bucket e l'hashing). Un **HashSet** non è altro che una **HashMap** in cui il "valore" di ogni coppia è un oggetto fittizio ignorato dal sistema.
- **LinkedHashMap**: Aggiunge la lista doppiamente concatenata alla tabella hash per mantenere l'ordine. Il **LinkedHashSet** si appoggia internamente a questa esatta struttura.
- **TreeMap**: Implementa fisicamente l'albero Red-Black. Il **TreeSet** è solo un adattatore che delega la gestione dei dati a una **TreeMap** dietro le quinte.

Avendo già studiato approfonditamente il comportamento fisico dei motori ad array, hash e alberi nel capitolo precedente, non è necessario ripeterne l'analisi prestazionale in questa sede. La complessità algoritmica (es. $O(1)$ per l'hash, $O(\log n)$ per gli alberi), le dinamiche di collisione e i meccanismi di ordinamento **sono letteralmente gli stessi**, poiché le classi **Map** rappresentano le reali fondamenta strutturali dell'intero framework.

Null-Safety nelle Mappe

La gestione dei valori **null** non è uniforme nel framework ed è una fonte comune di **NullPointerException** a runtime:

- **HashMap** e **LinkedHashMap** sono totalmente tolleranti: ammettono **una** chiave **null** e un numero arbitrario di valori **null**.



- `TreeMap` non permette chiavi `null` (lanciando un'eccezione non appena l'albero tenta di chiamare `compareTo()`), ma ammette valori `null`.
- `EnumMap` non permette chiavi `null` a causa del vincolo sul tipo enumerativo.



Mappe Specializzate: `WeakHashMap` e `IdentityHashMap`

Il framework fornisce due ulteriori implementazioni concrete di `Map`, progettate per risolvere problemi architetturali molto specifici nel backend, legati alla gestione della memoria e all'identità degli oggetti.

- **`WeakHashMap`:** In una normale `HashMap`, l'inserimento di un oggetto come chiave crea un *referimento forte* (*strong reference*), impedendo al *Garbage Collector* (GC) di liberare la memoria occupata da quell'oggetto finché la mappa non viene svuotata. La `WeakHashMap` utilizza invece *referimenti deboli*. Se una chiave non è più referenziata da nessun'altra parte dell'applicazione, il GC è autorizzato a distruggerla; quando ciò accade, la mappa rimuove automaticamente e silenziosamente l'intera *Entry*. È la struttura d'elezione per implementare sistemi di **caching** sicuri, prevenendo i *memory leak*.
- **`IdentityHashMap`:** Questa mappa viola intenzionalmente il contratto standard del framework. Per stabilire se due chiavi sono identiche, **non usa** il metodo `equals()`, ma utilizza l'operatore di uguaglianza fisica `==` (confrontando i puri indirizzi di memoria). Anche per il calcolo del bucket ignora il metodo `hashCode()` sovrascritto dall'utente e usa il `System.identityHashCode()`. È utilizzata quasi esclusivamente dagli sviluppatori di framework (es. serializzatori JSON come Jackson o ORM come Hibernate) per tenere traccia degli oggetti visitati ed evitare cicli infiniti durante l'attraversamento di grafi di oggetti complessi.

8.8 Copie e Viste

Come abbiamo già accennato analizzando i metodi di ispezione delle mappe (ad esempio `keySet()` o `entrySet()`), il *Java Collections Framework* fa un uso estensivo e fondamentale del concetto di **vista** (*view*).

Guardando l'intricata gerarchia delle interfacce e delle classi astratte (come `AbstractCollection` o `AbstractMap`), si potrebbe erroneamente dedurre che tale architettura sia sovradimensionata rispetto al modesto numero di classi concrete fisiche (come `ArrayList` o `HashMap`). In realtà, questa flessibilità strutturale serve proprio a supportare la creazione e la manipolazione delle viste.

Una **vista** è un oggetto che restituisce l'implementazione di un'interfaccia standard del framework (come `Collection`, `Set` o `List`), ma che **non possiede** una propria struttura dati indipendente e contigua per memorizzare fisicamente gli elementi. Al contrario, è un oggetto "leggero" i cui metodi manipolano, ispezionano o proiettano in tempo reale i dati di una collezione originale sottostante.

Per illustrare il meccanismo, riconsideriamo l'esempio del metodo `keySet()` di una mappa. A un primo sguardo, si potrebbe pensare che questo metodo allochi nuova memoria per istanziare un `HashSet`, esegua un ciclo per copiarvi tutte le chiavi della mappa e infine lo restituisca. **Fisicamente, non accade nulla di tutto questo.**

Il metodo restituisce in tempo costante ($O(1)$) un oggetto proxy che implementa l'interfaccia `Set`. Quando su questo `Set` si chiama il metodo `contains(k)`, l'oggetto delega la chiamata invocando `containsKey(k)` sulla mappa originale.

Osservazione 8.8 (Vantaggi Ingegneristici). L'impiego delle viste risolve due problemi cruciali nello sviluppo backend:

1. **Efficienza Spaziale:** Evita la duplicazione massiva dei dati in RAM. Estrarre una vista non richiede mai la copia lineare ($O(n)$) degli elementi.
2. **Sincronizzazione e Coerenza:** Essendo la vista solo una "lente" attraverso cui guardare la collezione originale, qualsiasi modifica strutturale apportata tramite la vista si riflette istantaneamente e obbligatoriamente sulla collezione base, e viceversa.

8.8.1 Factory Method per Collezioni Leggere e Immodificabili

A partire da Java 9, il framework mette a disposizione una serie di metodi statici (factory) estremamente concisi per la creazione rapida di liste, set e mappe. Sebbene a livello logico queste vengano percepite dallo sviluppatore come normali collezioni, fisicamente esse sono istanze di speciali classi interne, fortemente ottimizzate e strettamente regolamentate.

Collezioni rapide con `of()` e `ofEntries()` Le interfacce `List`, `Set` e `Map` espongono il metodo factory `of(...)` per inizializzare collezioni al volo. Per le mappe con più di 10 associazioni, si utilizza invece `Map.ofEntries(...)` in combinazione con `Map.entry()`.

Listing 8.16: Creazione di collezioni tramite factory

```
1 // Creazione di una Lista e di un Set
2 List<String> names = List.of("Peter", "Paul", "Mary");
3 Set<Integer> numbers = Set.of(1, 2, 3);
4
5 // Creazione di una Mappa (fino a 10 coppie, alternate Chiave/
6 // Valore)
7 Map<String, Integer> scores = Map.of("Peter", 2, "Paul", 3);
8
9 // Creazione di una Mappa estesa (numero arbitrario di
10 // associazioni)
11 Map<String, Integer> extendedScores = Map.ofEntries(
12     Map.entry("Peter", 2),
13     Map.entry("Paul", 3),
14     Map.entry("Mary", 5)
15 );
```

Il contratto architetturale di queste collezioni è rigidissimo e differisce da quello delle collezioni classiche:

1. **Immodificabilità assoluta:** Qualsiasi tentativo di aggiungere, rimuovere o modificare gli elementi (`add`, `remove`, `set`, `put`) solleverà istantaneamente una `UnsupportedOperationException`.
2. **Intolleranza ai null:** Non è permesso inserire valori, chiavi o elementi `null`. Il passaggio di un `null` ai factory genererà una `NullPointerException`.
3. **Rifiuto alla creazione dei duplicati:** A differenza di un `HashSet` che semplicemente ignora l'inserimento di un duplicato restituendo `false`, chiamare `Set.of(1, 1)` lancia una `IllegalArgumentException` a runtime.

Questi vincoli permettono alla JVM di allocare oggetti estremamente compatti e privi dell'overhead tipico delle strutture espandibili. Ad esempio, `List.of("A")` non istanzia un `ArrayList` o un array sottostante sovradimensionato, ma un oggetto dedicato che occupa in memoria esattamente lo spazio per quel singolo riferimento.

Rendere le collezioni modificabili

Se hai bisogno di inizializzare rapidamente una collezione, ma la logica di business richiede che questa possa essere modificata in seguito, puoi sfruttare queste viste come argomento per il costruttore di una collezione fisica mutabile. Il passaggio clonerà gli elementi nella nuova struttura dinamica:

```
1 // namesList sara' una normale ArrayList modificabile
2 List<String> mutableNames = new ArrayList<>(List.of("Peter",
    "Paul", "Mary"));
```

Liste ripetute con `Collections.nCopies()` Un altro strumento peculiare è il metodo `Collections.nCopies(int n, T o)`, utilizzato per generare una lista contenente n ripetizioni dello stesso elemento.

Da un punto di vista dell'implementazione (la verità dietro l'astrazione), questo metodo è uno degli esempi più eleganti di **vista**.

L'istruzione `Collections.nCopies(1000, "Java")` **non alloca** un array da 1000 posizioni e **non copia** 1000 volte il riferimento alla stringa. Sarebbe uno spreco catastrofico di memoria.

Al contrario, la chiamata istanzia un minuscolo oggetto (una speciale classe anonima immutabile) che in RAM occupa solo lo spazio per due variabili:

1. Un intero per la dimensione logica (1000).
2. Un singolo puntatore all'oggetto da ripetere ("Java").

Quando il codice chiamante esegue il metodo `get(i)`, l'oggetto semplicemente verifica che i sia un indice valido per le dimensioni dichiarate e, in tal caso, restituisce quell'unico puntatore memorizzato.

Questa vista è l'ideale quando si deve inizializzare massivamente una vera collezione con un valore di default, sfruttando l'allocazione a tempo costante $O(1)$:

Listing 8.17: Inizializzazione massiva tramite `nCopies`

```
1 // Crea un'ArrayList vera di 1000 elementi, tutti inizializzati a
   "DEFAULT"
2 List<String> defaultSettings = new ArrayList<>(Collections.
    nCopies(1000, "DEFAULT"));
```

8.8.2 Copie e Viste Immutabili

Nello sviluppo di applicazioni robuste, sorge frequentemente la necessità di passare strutture dati a metodi di terze parti o di esporre lo stato interno di un oggetto al mondo esterno. In questi scenari, esporre la collezione originale è estremamente pericoloso: qualsiasi componente esterno potrebbe modificarne il contenuto. Per proteggere i dati, Java offre due approcci strutturalmente e semanticamente opposti: le **copie indipendenti** e le **viste wrapper**.

Copie Indipendenti con `copyOf` A partire da Java 10, le interfacce `List`, `Set` e `Map` forniscono il metodo statico `copyOf(Collection)` per generare una collezione immutabile a partire da una esistente.

A differenza delle viste, questo metodo crea uno **snapshot** (un'istantanea indipendente) dei dati. Le modifiche successive alla collezione originale **non** avranno alcun effetto sulla copia restituita. Per questioni di ottimizzazione, se la collezione passata come argomento è *già* una collezione immuticabile (ad esempio creata con `List.of()`), il framework evita di sprecare RAM: non alloca nulla di nuovo, ma restituisce semplicemente il riferimento all'oggetto originale ($O(1)$).

✓ Il Caso d'Uso Tipico: La Copia Difensiva

L'uso principale di `copyOf` è la **Defensive Copy** (Copia Difensiva) per la creazione di classi di dominio immutabili.

Immagina un record `Fattura` che riceve una `List<Articolo>` nel costruttore. Se il costruttore si limitasse a salvare la lista o a creare una vista, il codice chiamante manterrebbe il riferimento alla lista originale e potrebbe aggiungere nuovi articoli *dopo* che la fattura è stata emessa, violando la logica di business. Usando `List.copyOf()` nel costruttore, la `Fattura` "congela" lo stato della lista al momento della creazione, recidendo ogni legame con la collezione esterna e garantendo l'immutabilità architetturale.

Viste Wrapper con `Collections.unmodifiable...` La classe di utilità `Collections` fornisce una serie di metodi storici (come `unmodifiableList`, `unmodifiableSet`, `unmodifiableMap`) che generano delle **viste**.

A differenza di `copyOf`, questi metodi non copiano i dati. Creano un oggetto proxy che funge da "lente" (*look at*) per guardare la collezione originale. La vista delega ogni operazione di lettura alla collezione sottostante, ma intercetta e blocca ogni tentativo di modifica (`add`, `remove`, `set`), lanciando una `UnsupportedOperationException`.

Poiché la vista non possiede dati propri, **ogni modifica apportata direttamente alla collezione originale si rifletterà immediatamente sulla vista**.

Listing 8.18: Comportamento dinamico della vista `unmodifiable`

```
1 List<String> staff = new ArrayList<>();
2 staff.add("Harry");
3
4 // Creiamo una vista in sola lettura (wrapper)
5 List<String> readOnlyStaff = Collections.unmodifiableList(staff);
6
7 // readOnlyStaff.add("Ron"); // ERRORE:
   UnsupportedOperationException
8
9 // Ma se modifichiamo l'originale...
10 staff.add("Ron");
11
12 // La vista "vede" la modifica! readOnlyStaff ora contiene 2
   elementi.
13 System.out.println(readOnlyStaff.size()); // Stampa: 2
```


! L'Anomalia di equals in unmodifiableCollection

Esiste un'insidia architetturale molto pericolosa quando si utilizza il metodo generico `Collections.unmodifiableCollection(c)`.

Le interfacce `List` e `Set` hanno regole di uguaglianza incompatibili (una lista non può **mai** essere uguale a un set, anche se contengono gli stessi elementi). L'oggetto restituito da `unmodifiableCollection` implementa solo la radice `Collection`; non potendo sapere se la collezione che sta "avvolgendo" sia una lista o un set, **non ridefinisce il metodo equals**.

Invece di delegare il controllo di uguaglianza alla collezione sottostante, eredita l'`equals` della classe `Object`, che si basa unicamente sull'identità in memoria (`==`).

```
1 List<String> myList = new ArrayList<>(List.of("A", "B"));
2 Collection<String> myView = Collections.
   unmodifiableCollection(myList);
3
4 // Restituira' FALSE! (Invece del TRUE che ci si
   aspetterebbe)
5 System.out.println(myView.equals(myList));
```

Per mantenere l'integrità del contratto `equals`, è categorico utilizzare i wrapper specifici (`unmodifiableList` per le liste o `unmodifiableSet` per i set), i quali delegano correttamente l'uguaglianza alla struttura sottostante.

8.8.3 Viste di Sotto-Intervalli (Sub-Ranges)

Oltre a creare viste dell'intera collezione, il *Java Collections Framework* permette di estrarre viste parziali che limitano l'accesso a uno specifico intervallo di elementi. Conformemente al pattern architetturale delle viste, non viene allocata alcuna nuova struttura dati o array: le operazioni sul sotto-intervallo si riflettono direttamente in tempo reale sulla collezione originale.

Sottoliste (subList) L'interfaccia `List` espone il metodo `subList(int fromIndex, int toIndex)`. Come quasi tutte le API del JDK che gestiscono indici, l'intervallo specificato è matematicamente **chiuso a sinistra e aperto a destra** (`[fromIndex, toIndex)`). Questo significa che l'elemento in posizione `fromIndex` è incluso, mentre quello in `toIndex` è escluso.

Listing 8.19: Uso di `subList` per estrarre e manipolare una porzione

```
1 List<String> staff = new ArrayList<>(List.of("A", "B", "C", "D",
   "E"));
2
3 // Crea una vista degli elementi agli indici 1 e 2 ("B", "C")
4 List<String> group2 = staff.subList(1, 3);
5
6 // Qualsiasi modifica alla vista impatta la lista originale
7 group2.clear();
8 // Ora 'staff' contiene solo ["A", "D", "E"]
```

 Cancellazione Massiva

L'idioma `list.subList(from, to).clear()` è in assoluto il modo più efficiente ed elegante per rimuovere un intero blocco contiguo di elementi da una lista con una singola istruzione.

Sottoinsiemi e Sottomappe (`subSet`, `subMap`) Per le collezioni ordinate (`SortedSet` e `SortedMap`), non avendo a disposizione gli indici posizionali, le viste vengono generate fornendo direttamente gli elementi (o le chiavi) che fungono da delimitatori di intervallo. Anche in questo caso, per le interfacce base, l'intervallo rispetta il pattern `[from, to)`.

- `subSet(E fromElement, E toElement)`
- `headSet(E toElement)`: Restituisce la vista di tutti gli elementi strettamente minori di `toElement`.
- `tailSet(E fromElement)`: Restituisce la vista di tutti gli elementi maggiori o uguali a `fromElement`.

Osservazione 8.9 (Inclusività in `NavigableSet` e `NavigableMap`). Le interfacce storiche `SortedSet` e `SortedMap` vincolavano lo sviluppatore al pattern standard. Per ottenere intervalli totalmente chiusi, si ricorreva a espedienti non ottimali. Con l'introduzione di `NavigableSet` e `NavigableMap`, i metodi sono stati sovraccaricati con l'aggiunta di flag booleani che permettono di specificare esplicitamente l'inclusione o l'esclusione di entrambi gli estremi:

```
1 // Crea un set con gli elementi tra "A" (incluso) e "D" (incluso)
2 SortedSet<String> sub = words.subSet("A", true, "D", true);
```

 La fragilità strutturale delle Viste

Essendo le viste dei puri "proxy" posizionali che puntano alla collezione sottostante, la loro validità logica dipende strettamente dall'integrità della struttura dati originale.

Se si modifica strutturalmente la collezione originale (aggiungendo o rimuovendo elementi) **senza passare attraverso la vista**, l'indice logico della vista si corrompe. Qualsiasi successivo tentativo di utilizzare il `subList` (o `subSet`) causerà istantaneamente il lancio di una `ConcurrentModificationException`.

Regola d'oro: non appena si estrae una vista parziale, si deve utilizzare **esclusivamente** quella vista per manipolare la relativa porzione di dati; in alternativa, la vista deve essere considerata un oggetto usa-e-getta, da scartare immediatamente se la collezione madre subisce alterazioni dirette.

Appendice A: Nozioni Accessorie

A.1 Il Concetto di Hashing

L'hashing è uno dei pilastri fondamentali dell'informatica moderna. Non è una tecnologia esclusiva di Java, ma un concetto matematico e logico universale utilizzato per la sintesi e l'identificazione rapida dei dati.

Un **hash** (o *digest*) è il risultato di una funzione che prende un input di lunghezza arbitraria (come un testo, un file o un oggetto complesso) e lo trasforma in una stringa o in un numero di **lunghezza fissa**.

L'analogia più calzante è quella dell'**impronta digitale**:

- Un libro di 1000 pagine è un dato complesso.
- Il suo valore hash è un numero (es. 42) che rappresenta in modo quasi univoco quel libro.
- È molto più facile confrontare due numeri (42 vs 43) che confrontare due libri parola per parola.

Affinché una funzione di hash sia utile, deve rispettare tre criteri:

1. **Deterministicità**: Lo stesso input deve generare *sempre* lo stesso identico output.
2. **Efficienza**: Il calcolo dell'hash deve essere estremamente rapido, indipendentemente dalla dimensione del dato in ingresso.
3. **Effetto Valanga (*Avalanche Effect*)**: Una minima modifica all'input (anche un singolo bit) deve produrre un hash completamente diverso e non correlato al precedente.

Ambiti di Applicazione

L'hashing risolve problemi critici in diversi settori:

- **Sicurezza delle Password**: I sistemi non memorizzano le password in chiaro, ma solo il loro hash. In caso di furto del database, l'attaccante non può risalire alla password originale.
- **Integrità dei Dati**: Attraverso algoritmi come SHA-256 o MD5, è possibile verificare se un file scaricato è integro o se è stato corrotto durante il trasferimento.
- **Strutture Dati (Hash Tables)**: Permette di trovare un elemento in un insieme di milioni di record in tempo costante ($O(1)$), usando l'hash come indice per un "armadietto" (*bucket*) specifico.

Il Problema delle Collisioni

Poiché l'insieme dei possibili input è infinito, mentre l'insieme dei possibili valori hash (numeri interi o stringhe di lunghezza fissa) è finito, esiste la possibilità teorica che due input diversi producano lo stesso hash. Questo evento è chiamato **collisione**. Un buon algoritmo di hash riduce drasticamente la probabilità di collisioni, e i sistemi informatici (come la JVM di Java) sono progettati per gestire correttamente questi rari casi di "omonimia digitale".

A.2 Tabelle Hash in Java

In Java, una tabella hash è implementata come un **array di liste concatenate** (o *array di nodi*). Ogni singolo slot (o cella) di questo array prende il nome di **bucket** (secchio).

Per determinare l'esatta posizione di un oggetto all'interno della tabella, il sistema esegue, a livello concettuale, due operazioni:

1. Calcola l'hash code dell'oggetto (chiamando il metodo `hashCode()`).
2. Riduce questo numero intero *modulo* la capacità totale dell'array (il numero di bucket).

Il resto della divisione rappresenta l'indice del bucket che ospiterà l'elemento. Ad esempio, se un oggetto ha un hash code pari a 76268 e la tabella ha 128 bucket, l'oggetto verrà posizionato nel bucket all'indice 108 (poiché $76268 \pmod{128} = 108$).

Se si è fortunati, il bucket calcolato risulta vuoto e l'elemento viene semplicemente inserito. Tuttavia, è frequente imbattersi in un bucket già occupato da un altro elemento: questo evento prende il nome di **collisione hash** (*hash collision*). In caso di collisione, il nuovo elemento viene aggiunto alla lista concatenata presente in quel bucket. Durante la ricerca, la JVM dovrà scorrere i nodi di questa lista e utilizzare il metodo `equals()` per confrontare il nuovo oggetto con quelli già presenti, verificandone l'effettiva presenza. Se gli hash code sono distribuiti in modo sufficientemente casuale e il numero di bucket è adeguato, ogni lista conterrà pochissimi elementi, mantenendo il tempo di ricerca medio a $O(1)$.

🔍 La Verità sull'Indice: Bitwise AND e Potenze di 2

La spiegazione matematica tramite l'operatore modulo (%) è corretta sul piano logico, ma **non** corrisponde a ciò che accade fisicamente nella JVM. L'operazione di divisione è computazionalmente costosa per la CPU.

Nel codice sorgente reale del JDK, l'indice non viene calcolato con il modulo, bensì tramite un'operazione binaria estremamente veloce (**Bitwise AND**):

```
1 index = (n - 1) & hash; // dove n e' il numero totale di
   bucket
```

Affinché questa operazione bit a bit restituisca esattamente lo stesso risultato di `hash % n`, esiste un vincolo ingegneristico assoluto: **la capacità dell'array deve sempre essere una potenza di 2** (16, 32, 64, 128, ecc.).

Se in fase di costruzione si passa al costruttore una capacità iniziale arbitraria (es. `new HashMap<>(30)`), Java non creerà un array di 30 elementi, ma lo arrotonderà automaticamente e silenziosamente alla successiva potenza di due più vicina (in questo caso, 32). La capacità di default è fissata a 16.

Ottimizzazioni Interne e Fattore di Carico Sebbene l'hashing garantisca, in media, prestazioni eccellenti ($O(1)$), esistono situazioni critiche che la JVM deve saper gestire, come funzioni hash mal progettate o veri e propri attacchi informatici volti a generare collisioni intenzionali per bloccare il sistema.

A partire da Java 8, se il numero di collisioni in un singolo bucket supera una specifica soglia (attualmente fissata a 8 elementi) e la tabella è sufficientemente grande, la lista concatenata si trasforma automaticamente in un albero binario bilanciato (*Red-Black Tree*). Questo processo degrada le prestazioni in modo controllato a $O(\log n)$, scongiurando il caso peggiore di $O(n)$.

Tuttavia, affinché la JVM possa costruire questo albero in modo efficiente, è fondamentale che gli oggetti utilizzati come chiavi implementino l'interfaccia `Comparable`. In caso contrario, in assenza di un ordinamento naturale esplicito, la JVM dovrà ripiegare su complessi meccanismi di *fallback* basati sugli indirizzi di memoria, penalizzando le prestazioni.

Oltre alla gestione delle singole collisioni, è necessario monitorare il riempimento globale della tabella. Se l'array interno si riempie eccessivamente, la probabilità di collisioni aumenta vertiginosamente. Per evitare questo scenario, interviene un parametro chiamato **Fattore di Carico** (*Load Factor*).

Il fattore di carico è un valore decimale che determina la soglia massima di riempimento prima che la tabella venga ridimensionata. In Java, il valore di default è **0.75**. Ciò significa che quando la tabella è piena al 75% della sua capacità, scatta un'operazione computazionalmente costosa detta **Rehashing**:

1. Viene allocato un nuovo array interno con il doppio della capacità rispetto al precedente.
2. Tutti gli elementi della vecchia tabella vengono ricalcolati (ri-hashati) e inseriti nel nuovo array.
3. La vecchia tabella viene abbandonata per essere poi gestita dal *Garbage Collector*.

Per la maggior parte delle applicazioni backend, il fattore di carico di default (0.75) rappresenta il perfetto compromesso tra efficienza in termini di tempo di esecuzione e occupazione di memoria. Tuttavia, i continui rehashing possono rallentare l'applicazione se non gestiti correttamente.

✓ Inizializzazione Preventiva

Se si conosce in anticipo il volume approssimativo dei dati da gestire, è buona norma inizializzare la tabella con una capacità corretta fin dall'inizio, evitando inutili e pesanti operazioni di rehashing.

La capacità iniziale ottimale non corrisponde al numero esatto degli elementi attesi, ma deve tener conto del fattore di carico:

$$\text{Capacità Ottimale} = \frac{\text{Elementi Attesi}}{\text{Load Factor}}$$

A partire da **Java 19**, il framework mette a disposizione dei comodi *factory methods* statici che eseguono questo calcolo matematico in automatico al posto dello sviluppatore:

```
1 // Il metodo calcola la capacita' ottimale per 10.000
   elementi
2 HashSet<String> mySet = HashSet.newHashSet(10000);
3 HashMap<String, Integer> myMap = HashMap.newHashMap(5000);
```

A.3 L'Array Circolare (Circular Array)

Nello studio delle collezioni, in particolare quando si analizzano le implementazioni di code (*Queue*) e code a doppia entrata (*Deque*), ci si imbatte nel concetto di **array circolare**.

In un normale array lineare, quando implementiamo una coda, aggiungiamo elementi alla fine e li rimuoviamo dall'inizio.

- Ogni volta che rimuoviamo il primo elemento, dovremmo teoricamente "shiftare" tutti gli altri elementi a sinistra per coprire il buco.
- Questa operazione ha una complessità temporale di $O(n)$, rendendo la struttura inefficiente per grandi moli di dati.

L'array circolare risolve il problema "collegando" virtualmente l'ultima posizione dell'array alla prima. Invece di spostare i dati, spostiamo i **puntatori** (indici) che indicano dove inizia e dove finisce la nostra collezione.

L'implementazione si basa su due indici principali:

1. **head**: Punta alla posizione del primo elemento disponibile (per la rimozione).
2. **tail**: Punta alla posizione successiva all'ultimo elemento (per l'inserimento).

Quando un indice raggiunge la fine dell'array fisico, "salta" alla posizione 0. Matematicamente, questo è gestito tramite l'operatore **modulo**:

$$\text{nuova_posizione} = (\text{posizione_attuale} + 1) \pmod{\text{lunghezza_array}}$$

Questa struttura permette operazioni di inserimento e rimozione in **tempo costante** $O(1)$.

Sebbene circolare, l'array ha comunque una dimensione fisica limitata. Quando l'array è pieno (ovvero quando **head** == **tail**), Java deve allocare un nuovo array più grande e copiare gli elementi. Tuttavia, questa operazione avviene raramente se la capacità iniziale è ben calibrata.

A.4 La Struttura Dati Heap

Lo **Heap** (o cumulo) è una struttura dati basata su albero che soddisfa proprietà specifiche di forma e ordine. Nonostante sia concettualmente un albero binario, la sua implementazione più efficiente e comune avviene tramite un **array dinamico**, eliminando l'overhead dei puntatori.

Un Heap deve rispettare due vincoli strutturali:

1. **Proprietà della Forma**: L'albero deve essere un **albero binario quasi completo**. Ciò significa che tutti i livelli sono completamente riempiti, ad eccezione eventualmente dell'ultimo, che deve essere riempito da sinistra a destra.
2. **Proprietà dello Heap (Ordine)**:
 - **Min-Heap**: Il valore di ogni nodo è maggiore o uguale al valore del suo nodo padre. Di conseguenza, la radice contiene il valore minimo. (Implementazione di default della `PriorityQueue` in Java).
 - **Max-Heap**: Il valore di ogni nodo è minore o uguale al valore del suo nodo padre. La radice contiene il valore massimo.

La genialità dello heap risiede nella sua mappatura implicita in un array. Dato un elemento memorizzato all'indice i (partendo da 0), è possibile calcolare la posizione dei suoi parenti tramite semplici operazioni aritmetiche:

- **Figlio sinistro**: $2i + 1$
- **Figlio destro**: $2i + 2$
- **Padre**: $\lfloor (i - 1) / 2 \rfloor$

Questa rappresentazione garantisce la **contiguità della memoria**, rendendo lo heap estremamente veloce grazie alla *cache locality*.

Operazioni Principali

Le operazioni devono mantenere intatte le due proprietà sopra descritte. Quando un nuovo elemento viene aggiunto:

1. Viene inserito nella prima posizione libera (l'ultima cella dell'array) per mantenere la forma dell'albero.
2. Viene confrontato con il padre. Se la proprietà dell'ordine è violata (es. nel Min-Heap il nuovo elemento è più piccolo del padre), i due vengono scambiati.
3. Il processo continua risalendo l'albero finché la proprietà non è ripristinata.

Complessità: $O(\log n)$, corrispondente all'altezza dell'albero.

Rimuovere la radice (l'elemento con priorità massima) è l'operazione cardine:

1. Si preleva il valore della radice.
2. L'ultimo elemento dell'array (il più a destra dell'ultimo livello) viene spostato nella radice.
3. Si confronta la nuova radice con i suoi figli. Se viola la proprietà dell'ordine, viene scambiata con il figlio minore (nel caso del Min-Heap).
4. L'elemento "affonda" lungo l'albero finché non trova la sua corretta collocazione.

Complessità: $O(\log n)$.

Analisi delle Prestazioni:

Operazione	Complessità	Descrizione
<code>peek()</code>	$O(1)$	Accesso immediato alla radice.
<code>add()</code>	$O(\log n)$	Inserimento con risalita.
<code>poll()</code>	$O(\log n)$	Estrazione con ripristino (heapify).
<code>buildHeap</code>	$O(n)$	Creazione di uno heap da un array disordinato.

Tabella A.1: Complessità computazionale dello Heap

Osservazione A.1. Sebbene la ricerca di un elemento generico sia $O(n)$ (perché lo heap non è un albero binario di ricerca), esso è imbattibile per la gestione delle priorità, dove l'unico interesse è accedere rapidamente all'elemento "più importante".

A.5 La Coda a Priorità (Priority Queue)

La **Coda a Priorità** è un Tipo di Dato Astratto (ADT) che estende il concetto di coda classica. Mentre in una coda standard (FIFO, si pensi ad esempio alla fila al supermercato) l'ordine di uscita è determinato esclusivamente dal tempo di arrivo, in una coda a priorità ogni elemento è associato a un valore di **priorità** (si pensi ad esempio alla fila al pronto soccorso).

In una coda a priorità, l'ordine di estrazione segue queste regole:

1. Un elemento con priorità maggiore viene servito prima di un elemento con priorità minore.
2. Se due elementi hanno la stessa priorità, vengono serviti secondo il loro ordine nella coda (solitamente FIFO, sebbene non sia sempre garantito da tutte le implementazioni).

Le operazioni fondamentali definite dall'ADT sono:

- `insert(item, priority)`: Aggiunge un elemento con una data priorità.
- `peek()`: Restituisce l'elemento con la priorità massima (o minima) senza rimuoverlo.

- `pull()` (o `pop`): Rimuove e restituisce l'elemento con la priorità massima (o minima).

La scelta standard per implementare una coda a priorità è un Min-heap (vedi paragrafo precedente) in cui la "priorità" è determinata dal valore stesso dell'elemento (o da un `Comparator`).

Bozze e Argomenti in Sospeso

B.1 Argomenti in sospeso da riprendere

Cap.4:

- **Factory Methods:** par. 4.4.4.
- **Distruzione di oggetti:** par. 4.6.8.
- **seconda parte delle lambdas da fare/rivedere:** a partire da par. 6.2.5.
- **logging**
- **seconda parte delle limitazioni sui generics**
- **capitolo collections a partire dal paragrafo 5.4**

B.2 Il Class Path

Il **Class Path** (percorso delle classi) è la collezione di tutte le locazioni (cartelle o archivi) in cui la JVM e il compilatore cercano i file `.class`. È, di fatto, l'elenco dei punti di partenza per la risoluzione dei package.

Il Class Path è una stringa composta da vari percorsi. Il separatore dipende dal sistema operativo:

Sistema	Esempio di Class Path	Separatore
Fedora / UNIX	/home/user/classdir:./libs/archive.jar	Due punti (:)
Windows	C:\classdir;.;C:\libs\archive.jar	Punto e virgola (;)

Osservazione B.1 (Il simbolo del punto). In entrambi i sistemi, il punto (.) rappresenta la **directory corrente**.

È comune confondere il Classpath con la directory di output. Ecco la distinzione corretta:

- **Per il Compilatore (javac):** Il Classpath serve a **risolvere i riferimenti esterni**. Se la tua classe A usa la classe B (che si trova in un JAR o in un'altra cartella), il compilatore deve "leggere" B per verificare che tu stia usando i suoi metodi correttamente.
 - *Nota:* Per decidere **dove mettere** il file compilato, si usa sempre il flag `-d`, non il Classpath.
- **Per la JVM (java):** Il Classpath serve a **caricare il codice in memoria**. La JVM non sa nulla dei file `.java`; lei cerca solo il bytecode (`.class`) seguendo i percorsi indicati nel Classpath per poter avviare l'esecuzione.

B.2.1 La "Ricerca Intelligente" del Compilatore

Horstmann sottolinea che il compilatore è più sofisticato della JVM nella ricerca. Quando incontra una classe, ad esempio `Employee`:

1. Cerca il file `Employee.class` nel Classpath.
2. Se trova anche il file `Employee.java`, controlla le date di modifica.

3. **Auto-recompile:** Se il sorgente (`.java`) è più recente del compilato (`.class`), il compilatore ricompila automaticamente il sorgente per garantire che tu stia usando la versione più aggiornata.

B.2.2 I file JAR (Java Archive)

Oltre alle normali directory, le classi possono essere raggruppate in file **JAR**.

- Un file JAR è essenzialmente un archivio in formato **ZIP** che contiene classi e strutture di sottocartelle compresse.
- È lo standard per la distribuzione di librerie di terze parti.
- Permette di migliorare le prestazioni (un solo file da aprire invece di centinaia) e risparmiare spazio.

B.2.3 L'Algoritmo di Ricerca

Se la JVM deve caricare la classe `com.horstmann.corejava.Employee`, cercherà in ordine:

1. Nelle classi delle **API di Java** (sempre cercate per prime, non serve includerle nel Class Path).
2. In ogni voce del Class Path, aggiungendo il percorso del package alla base. Ad esempio, se il path contiene `/home/user/classdir`, cercherà:
`/home/user/classdir/com/horstmann/corejava/Employee.class`

Attenzione B.1 (La trappola del punto). Per impostazione predefinita, il Class Path è costituito solo dal punto (`.`). Tuttavia, se specifichi un Class Path personalizzato (tramite la variabile d'ambiente `CLASSPATH` o il flag `-cp`) e **dimentichi** di includere il punto (`.`), la JVM non guarderà più nella cartella corrente.

- **Risultato:** Il programma compila (perché `javac` guarda sempre la cartella corrente), ma non parte (perché `java` non la trova).

B.2.4 Uso delle Wildcard

È possibile includere tutti i file JAR di una cartella usando l'asterisco:

```
1 # Su Fedora (bisogna proteggere l'asterisco dalle espansioni
   della shell)
2 java -cp "/home/user/libs/*:." MiaClasse
```

Nota: L'asterisco include tutti i file `.jar`, ma non i singoli file `.class` sparsi nella cartella.

B.3 La Classe `java.lang.System`

La classe `java.lang.System` rappresenta uno dei pilastri fondamentali del Java Development Kit (JDK). Situata nel pacchetto `java.lang`, essa funge da interfaccia primaria tra l'applicazione Java e l'ambiente in cui viene eseguita (Java Virtual Machine e Sistema Operativo).

B.3.1 Caratteristiche Strutturali

Dal punto di vista dell'architettura software, la classe `System` presenta caratteristiche peculiari:

- **Finalità:** È dichiarata come `public final`, il che impedisce qualsiasi tentativo di ereditarietà.

- **Istanziabilità:** Il costruttore è privato, rendendo impossibile creare istanze della classe. Tutte le sue funzionalità sono accessibili esclusivamente tramite metodi e campi `static`.
- **Accessibilità:** Essendo parte di `java.lang`, viene importata automaticamente in ogni file sorgente Java.

B.3.2 I Flussi di I/O Standard

La classe gestisce i tre canali di comunicazione standard definiti a livello di sistema operativo. Questi campi sono istanze di `PrintStream` (per l'output) e `InputStream` (per l'input).

1. `System.out`: Rappresenta lo *standard output*. Viene utilizzato per l'invio di dati alla console o al terminale.
2. `System.err`: Rappresenta lo *standard error*. Sebbene tecnicamente simile a `out`, è destinato specificamente alla diagnostica. Molti IDE e sistemi di logging separano questi due flussi, evidenziando spesso l'errore in rosso.
3. `System.in`: Rappresenta lo *standard input*. Viene solitamente utilizzato in combinazione con la classe `Scanner` per leggere l'input dell'utente dalla tastiera.

Listing B.1: Esempio di utilizzo dei flussi standard

```
1 System.out.println("Messaggio informativo");  
2 System.err.println("Messaggio di errore critico");
```

B.3.3 Gestione del Tempo e Misurazioni

Per il monitoraggio delle prestazioni e la gestione temporale, la classe offre due metodi fondamentali con risoluzioni differenti:

- `currentTimeMillis()`: Restituisce il tempo corrente in millisecondi dall'epoca Unix (1 gennaio 1970). È ideale per calcolare date o timestamp da salvare nel database.
- `nanoTime()`: Restituisce il valore di un timer ad alta risoluzione in nanosecondi. Non è legato all'ora solare, ma è lo strumento standard per misurare quanto tempo impiega un blocco di codice ad essere eseguito (benchmarking).

B.3.4 Interazione con l'Ambiente Operativo

La classe permette di interrogare il sistema operativo e la configurazione della JVM tramite le *System Properties* e le *Environment Variables*.

Proprietà di Sistema

Le proprietà di sistema sono coppie chiave-valore che definiscono il runtime Java.

- `System.getProperty("user.home")`: Restituisce il percorso della cartella utente.
- `System.getProperty("java.version")`: Restituisce la versione di Java in uso.
- `System.lineSeparator()`: Restituisce il separatore di riga specifico dell'OS (`\n` su Unix, `\r\n` su Windows).

Variabili d'Ambiente

Le variabili d'ambiente sono impostate a livello di OS e sono fondamentali nel backend moderno (specialmente con Docker e Kubernetes) per gestire configurazioni sensibili come password o indirizzi di database.

```
1 String dbUrl = System.getenv("DATABASE_URL");
```

B.3.5 Utility di Sistema e Memoria

- **arraycopy()**: Un metodo nativo estremamente performante per copiare dati tra array. A differenza di un ciclo **for**, questo metodo sposta i blocchi di memoria direttamente tramite chiamate a basso livello.
- **gc()**: Suggerisce alla JVM di eseguire il Garbage Collector. È importante notare che è solo un suggerimento e non garantisce l'immediata liberazione della memoria.
- **exit(int status)**: Termina la JVM corrente. Un codice di stato 0 indica una chiusura corretta, mentre un valore diverso da zero indica un fallimento.

B.3.6 Integrazione con il Logging Moderno

Dalla versione 9 di Java, la classe **System** ha assunto un ruolo centrale nella *Platform Logging API* tramite il metodo:

```
1 System.Logger logger = System.getLogger("NomeLogger");
```

Questo permette di astrarre il sistema di logging, delegando a **System** il compito di trovare e utilizzare il miglior backend disponibile (come JUL o Log4j).

Attenzione

Nel contesto dei server e delle applicazioni web, l'uso di **System.exit()** e la manipolazione diretta di **System.out** sono sconsigliati. Si preferisce delegare la chiusura al contenitore dell'applicazione e utilizzare i framework di logging per l'output, garantendo così la stabilità e la tracciabilità del sistema.

B.4 Snippets vari da inserire

Esiste un paradosso apparente: le interfacce non possono estendere le classi (nemmeno **Object**), eppure possiamo chiamare **.toString()** o **.equals()** su qualsiasi variabile di tipo interfaccia.

Questo accade perché Java aggiunge implicitamente le firme dei metodi pubblici di **Object** a ogni interfaccia.

B.5 Logging [BOZZE DA RIPRENDERE]

Tutti i programmatori Java conoscono il rito del "debugging tramite stampa": inserire chiamate a **System.out.println** nei punti critici del codice per capire cosa stia succedendo. Una volta risolto il problema, queste istruzioni vengono rimosse, per poi essere reinserite al bug successivo.

I framework di **logging** sono progettati per superare questo ciclo inefficiente, offrendo una soluzione permanente, professionale e configurabile.

Il logging non è una semplice "stampa a video evoluta", ma un sistema di tracciamento che offre:

- **Persistenza:** Registrare i messaggi su file o database, non solo sulla console.
- **Granularità:** Classificare i messaggi per importanza (es. INFO, WARNING, SEVERE).
- **Configurabilità:** Attivare o disattivare i messaggi senza modificare o ricompilare il codice.

In un'applicazione professionale, l'uso di `System.out.println` è considerato una cattiva pratica. Considera questo scenario: un server subisce un guasto alle 3 di notte mentre processa dei pagamenti. L' "utente" in questo caso non è un essere umano che interagisce in tempo reale, ma un processo automatico che gira in background. Senza un log persistente su file, l'errore svanirebbe nel nulla e sarebbe impossibile ricostruire l'accaduto il mattino seguente. Il logging è la "scatola nera" per quando non si è presenti a guardare la console.

B.5.1 Architettura del Logging: Facciate e Implementazioni

Nel mondo Java, il logging non è un blocco monolitico. Per garantire flessibilità, si utilizza un'architettura divisa in due parti:

- **Frontend (façade/API):** È il codice (es. `logger.log(...)`) che il programmatore scrive. Definisce *come* si inviano i messaggi. Esempi: **SLF4J**, **Platform Logging API (JEP 264)**.
- **Backend (Implementation):** È il motore che decide *dove* vanno i messaggi (file, console, rete) e come formattarli. Esempi: **java.util.logging (JUL)**, **Log4j**, **Logback**.

Utilizzare una "facciata" (come SLF4J o la Platform API) permette di cambiare il motore di logging in futuro senza dover modificare una singola riga di codice nel resto dell'applicazione. Basta cambiare una configurazione esterna.

Anche se nel mondo enterprise si usano spesso Log4j o Logback, studiare il backend standard del JDK (JUL) è importante per tre motivi:

1. **Fondamenta:** Offre una base solida per capire come funzionano tutti gli altri framework.
2. **Semplicità e Sicurezza:** Essendo parte del JDK, è più "snello" e meno soggetto ad attacchi informatici.
3. **Zero Dipendenze:** Non richiede librerie esterne, rendendolo perfetto per piccoli progetti o moduli di sistema.



Sicurezza e Log4j

La complessità eccessiva può essere un pericolo. Il famoso attacco hacker a **Log4j** (Log4Shell) è stato possibile proprio a causa di funzionalità oscure e troppo potenti che permettevano l'esecuzione di codice malevolo tramite un semplice messaggio di log.

Di seguito, salvo avviso contrario, faremo dunque riferimento a JEP 264 per il frontend e a JUL per il backend.

B.5.2 L'interfaccia `System.Logger`

Come abbiamo accennato alla fine di 5.4.5, oltre alle classi, si possono annidare anche le interfacce. `System.Logger` è proprio un esempio di **Nested Interface**: la classe `System` funge da *namespace* per l'interfaccia `Logger` quindi, quando scriviamo `System.Logger`, il punto (`.`) non indica l'accesso a un metodo o a una variabile di istanza, ma l'accesso a un **tipo** definito all'interno dell'ambito (scope) della classe `System`.

Questa scelta di design non è estetica, ma strutturale:

- **Namespacing**: Evita di creare nomi globali troppo generici come `Logger` nel pacchetto `java.lang`, che potrebbero andare in conflitto con altre librerie.
- **Accoppiamento Logico**: Indica chiaramente che l'interfaccia esiste *solo* in funzione dei servizi offerti dalla classe ospite.
- **Incapsulamento**: Permette di raggruppare tipi correlati senza "inquinare" il file `system` con decine di piccoli file `.java`.

Se analizzassimo il codice sorgente del JDK, la struttura apparirebbe qualitativamente così:

Listing B.2: Struttura semplificata di `System.java`

```
1 public final class System {
2     // Campi e metodi statici (out, err, getProperty...)
3
4     /** * Interfaccia annidata statica.
5      * In Java, le interfacce annidate sono implicitamente '
6      * static '.
7      */
8     public interface Logger {
9         enum Level { ALL, TRACE, DEBUG, INFO, WARNING, ERROR, OFF
10             }
11         void log(Level level, String msg);
12         // ... altri metodi abstract ...
13     }
14
15     public static Logger getLogger(String name) {
16         // Restituisce un'implementazione concreta fornita dal
17         backend
18     }
19 }
```

Ricordiamo che per utilizzare un tipo annidato, la sintassi segue la gerarchia di contenimento:

1. **Dichiarazione del tipo**: si usa il nome della classe esterna come prefisso:
`System.Logger mioLogger;`
2. **Accesso ai membri del tipo annidato**: se l'interfaccia annidata contiene a sua volta delle enumerazioni (come `Level`), la catena si allunga:
`System.Logger.Level.INFO.`

Tip

Ricorda: un'interfaccia annidata è sempre **static**, anche se non lo scrivi esplicitamente. Questo significa che può essere utilizzata senza dover creare un'istanza

💡 della classe esterna (cosa che nel caso di **System** sarebbe comunque impossibile, avendo il costruttore privato).

B.5.3 Ottenimento e Utilizzo del Logger

L'ottenimento di un logger non avviene tramite l'operatore **new**, ma attraverso il metodo statico **System.getLogger()**. L'argomento del metodo è una stringa che rappresenta il nome del logger; questo nome può essere arbitrario, ma è considerata una *best practice* utilizzare il nome del package (o della classe) in cui si generano i messaggi.

Questo approccio permette di sfruttare la **gerarchia dei nomi** per una configurazione granulare. Di fatto, il sistema utilizza un meccanismo di **caching** concettualmente analogo a quello della *String Pool*: la prima chiamata al metodo con un determinato nome crea il logger, mentre tutte le chiamate successive con lo stesso nome restituiscono il medesimo oggetto già esistente.

Listing B.3: Recupero di un Logger

```
1 System.Logger logger = System.getLogger("com.myapp");
```

💡 Tip

Poiché i logger con lo stesso nome sono condivisi, è prassi comune dichiararli come **private static final**. Questo garantisce che la variabile non venga riassegnata e che sia disponibile a livello di classe.

Una volta ottenuto il logger, è possibile registrare messaggi specificando il **livello di importanza**. La sintassi base della *Platform Logging API* è la seguente:

Listing B.4: Esempio di registrazione di un messaggio

```
1 logger.log(System.Logger.Level.INFO, "Apertura del file " +
    filename);
```

A differenza di una stampa manuale, il sistema di logging produce un record strutturato. Ecco un esempio di come appare il messaggio precedente in console (usando il backend JUL):

```
1 Apr 28, 2026 10:20:00 AM com.myapp.Main read
2 INFO: Apertura del file dati.txt
```

Il logger si comporta come un segretario scrupoloso. Oltre al messaggio fornito dal programmatore, esso cattura e stampa automaticamente i seguenti **metadati**:

- **Timestamp**: Data e ora precisa dell'evento.
- **Nome della Classe**: Il nome completo (*Fully Qualified Name*) della classe chiamante, nel nostro caso **com.myapp.Main**.
- **Nome del Metodo**: Il nome del metodo all'interno del quale è stata invocata la scrittura del log, nel nostro caso **read**.
- **Livello**: L'etichetta di severità (**INFO**, **WARNING**, ecc.).

Il fatto che il nome della classe e del metodo siano inclusi **automaticamente** è un vantaggio enorme in fase di debug. Non è più necessario scrivere manualmente **"Errore nel metodo read(): ..."**, poiché il logger "sa" già dove si trova grazie all'analisi dello stack trace.